

VarioHyve Language Specification

Ciprian Teodorov

01-10-2026

Contents

1	VarioHyve for Systems-of-Systems and Federated Modeling	1
2	Program Structure	10
3	Literals	15
4	Objects and Classes	24
5	Slots and Accessors	29
6	Message Sends	36
7	Message Send, Invocation, Apply, and Tear-Off	45
8	Syntactic Sugar	50
9	Name Resolution	53
10	Recursion	61
11	Descriptors	65
12	Object Model	68
13	Interpreter Design	90
14	Foreign Semantic Values and the Alien/Expat Boundary	97
15	Tooling Architecture	104
16	Future Directions	108

1 VarioHyve for Systems-of-Systems and Federated Modeling

Most programming languages are designed around a single application: one runtime, one concurrency model, one object model, one team, one semantic universe. Heterogeneity — different languages, different tools, different organizations, different timing models — is treated as an exception to be bridged from the outside.

Systems-of-Systems (SoS) work the opposite way. Multiple partially-autonomous subsystems, implemented by different teams with different tools under different governance regimes, must still compose into a coherent whole. Heterogeneity is not the exception — it is the structure. VarioHyve is designed from first principles around that reality.

This section explains why each architectural choice in VarioHyve maps directly to the demands of SoS design and federated behavioral modeling.

1.1 Heterogeneity as the Default, Not the Exception

The VarioHyve spec opens with a design statement that sets the tone:

“multiple subsystems (potentially implemented in different languages, running on different runtimes, governed by different concurrency models) must compose into a coherent whole.”

This is not a use case. It is the baseline design constraint. Every structural decision — no global environment, constructor-parameter injection, the Alien/Expat boundary model, the AAM-style interpreter — flows from this commitment.

1.2 Explicit Constructor Dependencies

VarioHyve has no global environment and no import statements. Named and anonymous class literals obtain capabilities through superclass inheritance, mixin composition, lexical nesting, and constructor parameters. Normal module evaluation has an empty top activation at its base, with no enclosing application scope. The top-level class inherits only from root `Object` and composes no mixins. Its external dependencies therefore arrive through its constructor parameters. See [Program Structure](#).

Development and debugging tools may explicitly provide richer evaluation contexts under the same language rules. These contexts do not silently enrich normal module roots, and results may capture the supplied tooling objects.

```
class Mission(consoleService, clockService, communicationBus) {  
  console = consoleService;  
  clock = clockService;  
  bus = communicationBus;  
}
```

An instantiation service supplies existing objects it has acquired or already holds. There can be any declared number of arguments. A kernel object may bundle several dependencies, but neither its name nor a single-argument signature is required. The service's remaining capabilities do not become a lexical scope for the module. Access to further objects through a supplied service is part of that service's authority.

In SoS terms, this means:

- **Partitioning follows object reachability.** A module obtains another participant's services only through the object graphs made available to it.
- **Deployment can replace dependencies independently.** Supply a mock clock, a real communication bus, or a recording console without changing lookup.
- **Analysis has explicit inputs.** All constructor arguments and their reachable behavior must be modeled; this is not restricted to one kernel object.

Optional bundles can simplify repeated deployment configurations:

Bundle convention	Example contents	Use case
ConcreteKernel	Real I/O, real clock, real bus	Deployment
SimKernel	Virtual clock, mock sensors, replay bus	Simulation
TestKernel	Recording console, fault injector	Testing
AbstractKernel	Abstract input values and effect models	Future analysis
ModelCheckKernel	State-logging inputs and effect models	Future model checking

These are ordinary objects chosen by the instantiation service. The language commitment is explicit object parameters, with a fixed lookup and invocation semantics across configurations. Abstract inputs still require an implemented abstract interpreter; changing a bundle alone does not supply one.

1.3 Structural Encapsulation and Governance Boundaries

Nesting grants access to enclosing scope. Two objects nested under the same capability-rich module do not become isolated merely because each stores a different service in one slot: both can still reach their common outer scope. Inheritance and mixin definition scopes are additional capability routes.

For independent authority boundaries, use separate top-level modules and pass restricted object graphs to each. For example, these are two separate module expressions instantiated by an external service:

```
class Avionics(flightBus) {
    flightComputer = flightBus;
}
```

```

class Entertainment(mediaStream) {
    mediaServer = mediaStream;
}

```

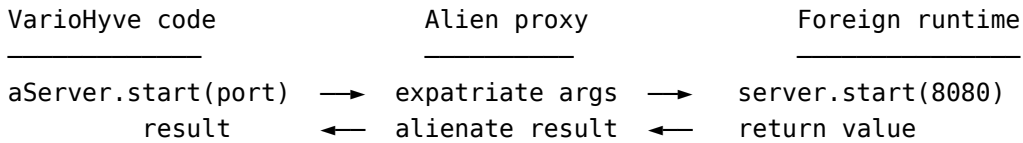
Neither module has an outer path to the instantiation service or to the other module. Isolation still depends on the supplied objects: a `mediaStream` that returns the flight bus would intentionally grant that access. Lexical nesting is useful where authority is meant to be shared; it is not a substitute for restricting the reachable capability graph.

1.4 The Alien/Expat Federation Protocol

The most operationally significant SoS feature in VarioHyve is the **Alien/Expat boundary model** — the formal protocol for interaction between VarioHyve and foreign runtimes.

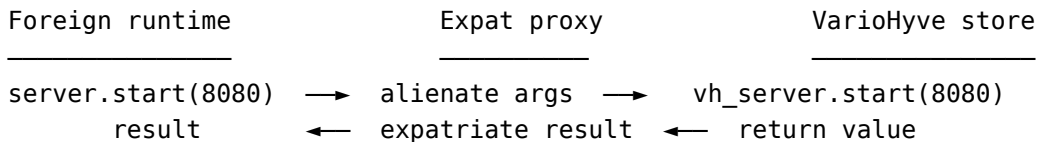
VarioHyve federates with external systems without absorbing them. Legacy subsystems in C++, controllers in Simulink, business logic in Java, and formal models in TLA+ can all remain in their original form. What VarioHyve provides is a behavioral bridge with precisely defined semantics.

Aliens are references to foreign semantic values or entities. An object-oriented adapter may expose message forwarding, for example:



An explicit Alien send crosses to its adapter: arguments are expatriated, the selected foreign operation runs, and its result is alienated. The foreign domain defines the operation. An opaque handle may support no direct sends; explicit capabilities can operate on it instead. Aliens do not acquire native classes, mixins, lexical outers or VarioHyve lookup. Native wrappers supply those abstractions explicitly.

Expats are the symmetric inverse — foreign proxies for VarioHyve objects:



When Python or Java code calls a method on an Expat, the Expat alienates the foreign arguments into VarioHyve values, dispatches the message to the wrapped VarioHyve object, and expatriates the result back.

The protocol preserves compatible reference round trips: exporting an Alien to its originating domain recovers the foreign entity, and importing a runtime's own Expat recovers the native value. Identity-bearing values preserve aliases; marshalled primitives instead follow their defined value conversions.

This matters for SoS because:

- **No forced monoculture.** Legacy subsystems stay in their original language. VarioHyve provides the behavioral coordination layer, not a migration target.
- **Explicit interaction contracts.** VarioHyve uses sends to request foreign operations; each adapter specifies how those requests map to its domain. The domains need not share an object model.
- **A principled seam for interface evolution.** The Alien and Expat surfaces are explicit. When a foreign subsystem changes its API, the change is localized to the proxy layer and the constructor-argument wiring — not scattered across the model.
- **Instantiation supplies explicit wiring.** The service passes Alien-backed or native objects as constructor arguments, independently or in optional bundles. Language-visible VM services use the same Alien/Expat boundary.

1.5 Behavioral Lingua Franca: One Interaction Model

Behavioral interactions in VarioHyve use (name, arity) message sends. Operators, slot access and function application use that common interaction form. Native receiver origins, lexical binding and construction still have specified language semantics; they are not foreign protocol requirements.

This is not minimalism for its own sake. In a federated system, adapters must bridge foreign systems to VarioHyve. A message-centric core means every adapter bridges exactly one concept:

```
incoming event    → message send to a VarioHyve handler
RPC call          → message send to an Alien proxy
simulation tick   → message send to a simulation coordinator
hardware callback → message send to a driver object
```

This common request form gives adapters a small language-facing transport surface. It does not remove the foreign domain's own semantic obligations.

The native lookup algorithm terminates at the Alien boundary. The adapter may expose method forwarding, a function call, a memory capability or another domain operation. This common transport does not make every participant an object or make adapter semantics trivial. Signatures, failure, identity, lifetime and effects remain explicit adapter obligations.

1.6 Composable Structure for Heterogeneous Participants

Not every SoS participant is “an instance of a class.” VarioHyve provides a spectrum of participant styles with a unified interaction model:

Participant style	VarioHyve construct	Example
Unique service	Singleton object literal	Radar = { ... }
Family of instances	Class	class Sensor(id) { ... }
Namespace/registry	Singleton with nested structure	Protocol = { v1 = ..., v2 = ... }

Participant style	VarioHyve construct	Example
Configurable factory	Class with <code>factory apply</code> variants	Multiple construction paths
Delegation wrapper	Object with <code>apply</code> method	Callable proxy
Dynamic coalition	Objects assembled at runtime	First-class object composition

A coordinator can use `sensor.detect(target)` with a native object or an Alien adapter when both support the required behavioral contract. Native receivers use fixed VarioHyve lookup; Alien receivers cross to their foreign domain. Common invocation syntax alone does not make arbitrary participants interchangeable: signatures, results, effects and failure behavior must agree.

The `factory apply` pattern is particularly relevant for federated deployment. A subsystem class can define multiple construction signatures for different operational contexts:

```
class Sensor(id, config) {
  factory apply(id, config) { new(id, config) }           // normal deployment
  factory apply(id) { new(id, SimConfig()) }               // simulation mode
  factory apply() { new(#mock, MockConfig()) }             // test/verification
}
```

This is a nested-class fragment: `SimConfig` and `MockConfig` must be available in its definition scope. The instantiating code selects a factory and supplies its arguments; there is no implicit kernel argument.

1.7 Mixins for Cross-Cutting Federation Concerns

SoS systems share cross-cutting concerns across subsystem boundaries: logging, tracing, fault tolerance, redundancy voting, time synchronization, cybersecurity guards, safety wrappers. These are not part of any one subsystem's functional behavior — they are federation-level concerns that must be applied uniformly.

In the following nested-class fragment, the enclosing scope supplies the superclass and mixins. Mixins in VarioHyve are the mechanism for this. A mixin is a regular object whose class holds methods. It becomes a behavioral layer applied to a class without forcing that class into a particular inheritance hierarchy:

```
class Drone(id) <: Logging, SecureComm, FaultTolerant, TimeStamped < Vehicle
  ↪ {
    sId = id
    fly(trajectory) { ... }
  }
```

Each mixin contributes behavior, member declarations and, if parameterized, initialization to the composed class. Physical slots may live on one instance; that representation

must not erase declaration or application origins. The initial policy selects contributions in declared order with overriding. Deep family composition with state in members is an intended later increment, not an implemented bootstrap feature.

For SoS this matters because:

- **Concerns can be added or removed without changing the subsystem model.** Adding a mixin to a class requires no change to its internal logic.
- **Mixin composition retains its structure.** Reified definitions and their origins remain available internally. A public mirror API is deferred.
- **Order and overrides are explicit.** Own contributions precede composed entries in declaration order and inherited behavior. Determinism does not remove state collisions or initialization-effect dependencies; those require explicit composition contracts and, later, state/adaptation operators.

1.8 Lexical Nesting Mirrors System Hierarchy

Lexical nesting can represent organizational hierarchy. Member classes bind to enclosing instances, while qualified outer `ClassName` selects a lexical receiver without exposing internal activations or a universal parent accessor:

```
class Mission(consoleService) {           // mission-level scope
  console = consoleService;
  class Fleet(id) {                       // fleet scope
    class Platform(pid) {                 // platform scope
      class Subsystem(sid) {              // subsystem scope
        log(msg) {
          outer Mission.console.print(msg) // explicit lexical
          ↪ mission receiver
        }
      }
    }
  }
}
```

Each level of nesting shares authority with its enclosing scope through lexical lookup and qualified outer. Constructor parameters are initializer-only inputs: the mission explicitly stores its console service in a slot for member behavior. A member declaration cannot recover an unstored constructor argument. Ordinary lambdas and class expressions created in an initializer can deliberately capture its parameters, with names retained only for named class expressions. Nested subsystems can also receive capabilities through inheritance, mixins and their own constructor parameters. The top-level mission has only the empty top activation beneath it, with no enclosing application scope; it receives its console as an explicit argument.

This structural alignment between language semantics and governance structure means the model itself expresses the authority hierarchy — it is not an external documentation artifact.

1.9 Late Binding for Evolving Systems

All names in VarioHyve are late-bound. A bare name is an implicit message send resolved at runtime through the lexical and inheritance chains. No name is resolved at compile time to a specific object address.

In SoS, this matters because:

- **Supplier replacement does not break callers.** A subsystem that calls `kernel.radar.detect(target)` does not bind to a specific Radar implementation. The instantiation service can supply a different bundle with the same behavioral interface without modifying the caller.
- **Interface versioning is handled by the message protocol.** A new version of a subsystem that adds methods does not break existing callers — unused methods are simply not called. A removed method surfaces as `doesNotUnderstand` at the call site, which can be caught and handled.
- **Dynamic coalition formation is natural.** Objects can be assembled and rewired at runtime. A coordination object can be handed a different set of participants without requiring structural changes to the coordination logic.

The apply protocol extends this: any object that implements `apply` is callable. A mock object, a proxy, a recorded trace, and a live hardware driver can all sit in the same slot and receive the same messages — the calling code has no way to distinguish them and no reason to care.

1.10 Execution and Verification from the Same Core

Many SoS efforts maintain separate artifacts for execution and verification: a simulation model, a formal model, a code implementation, a documentation model. These quickly diverge. VarioHyve unifies them by design.

The VarioHyve interpreter is an **AAM-style CESK machine**. The machine has four components (Control, Environment, Store, Kontinuation) and a single parameterization point: the `Alloc` function that determines how store addresses are allocated:

- **Concrete Alloc** (fresh integer per allocation) → the v1 deterministic interpreter. Runs the model.
- **Abstract Alloc** (k-CFA, bounded by call site and environment prefix) → a sound abstract interpreter. Explores all reachable abstract states. Computes program properties.

The step function — the semantic core — is the same in both cases. This is the key commitment: the verification artifact and the execution artifact share one semantic definition. There is no translation layer between them, no semantic gap to manage, no risk of the verification model drifting from the implementation.

For SoS this enables:

Activity	Mechanism
Simulation	Concrete CESK with <code>SimKernel</code>
Unit and integration testing	Concrete CESK with <code>TestKernel</code>
Type inference / interval analysis	Abstract CESK with abstract kernel values

Activity	Mechanism
State space exploration	Abstract CESK with nondeterministic abstract transitions
Model checking foundations	Abstract CESK with state-logging ModelCheckKernel
Trace replay and runtime monitoring	Concrete CESK with a trace-recording store

This is what SoS engineering desperately lacks: behavioral models that are simultaneously executable and formally analyzable, with no semantic mismatch between the two.

1.11 Descriptors: Multi-Stakeholder Semantic Layers

A federated system has multiple stakeholders, each with different concerns about the same behavioral model: safety engineers, cybersecurity analysts, simulation teams, certification bodies, code generators. These stakeholders need to attach their own metadata to model elements without altering the execution semantics.

Descriptors are VarioHyve’s mechanism for this. They are ordinary expressions that annotate syntactic elements — slots, methods, classes, lambdas, parameters — and are completely ignored by the core interpreter. A program with all descriptors stripped executes identically to the annotated version.

```
class Sensor(id, config) {
  reading = 0 : immutable : serializedAs("json") : safetyLevel(4)
  calibrate(ref) {
    ...
  } : cached : realTime : certified(D0178C)
} : module : traceable(reqId= "SYS-042")
```

Since descriptors are expressions, they can be message sends that return descriptor objects:

```
{
  altitude = 0 : constrained(0, 50000) : units("feet") : safetyLevel(4)
  velocity = 0 : constrained(0, 3000) : units("knots")
} : module : certifiedBy("avionics-safety-team")
```

The same model element carries annotations for:

- **Safety engineers:** safetyLevel(4), certified(D0178C), constrained(0, 50000)
- **Simulation teams:** realTime, cached
- **Serialization/integration:** serializedAs("json"), units("feet")
- **Traceability/audit:** traceable(reqId= "SYS-042"), certifiedBy(...)
- **Documentation tooling:** anything else

Each tool layer reads the descriptors it understands and ignores the rest. The core interpreter runs through all of them unchanged. This is federated modeling in its direct form: one behavioral model, multiple stakeholder interpretations, cleanly separated.

1.12 In Summary

VarioHyve is uniquely equipped for heterogeneous SoS design because its architecture directly encodes the structural realities of SoS:

SoS requirement	VarioHyve mechanism
Implementation diversity	Alien/Expat protocol — foreign runtimes stay foreign
Governance partitioning	No globals; restricted constructor arguments and reachable object graphs
Uniform interaction across boundaries	Everything is a message send — one adapter model
Participant diversity	Objects, classes, singletons, callables — same dispatch
Cross-cutting concerns	Mixins — behavioral layers without forced hierarchy
Organizational hierarchy	Member-class nesting and qualified lexical receivers
Evolving interfaces	Late binding — names resolve at runtime
Executable + analyzable	AAM-parameterized CESK — same core, swap Alloc
Multi-stakeholder views	Descriptors — optional, execution-transparent annotations
Context-specific deployment	Constructor arguments, optionally bundled — same model, different contexts

Most languages tolerate heterogeneity from the outside. VarioHyve is built for it from the inside.

2 Program Structure

VarioHyve is a language designed for **complex heterogeneous system-of-systems** — it serves as a *lingua franca* for behavioral-rich model federation. In this setting, multiple subsystems (potentially implemented in different languages, running on different runtimes, governed by different concurrency models) must compose into a coherent whole. VarioHyve provides the behavioral core: a deterministic, analyzable object model that can federate with foreign runtimes through a well-defined boundary (see [Foreign Object Interaction](#)) and scale to concurrent compositions through a layered semantic interface (see [Future Directions — Concurrency](#)).

2.1 A Program Is a Single Expression

The grammar entry point is:

```
top: expression? EOF;
```

A VarioHyve program is an optional single expression. A top-level class literal, named or anonymous, serves as a module. The service that instantiates it supplies the existing object instances required by its constructor; the language does not prescribe their number or names.

```
class Application(consoleService, clockService, settings) {  
  console = consoleService;  
  clock = clockService;  
  config = settings;  
  
  run { console.print(clock.now) }  
}
```

`class(consoleService, clockService, settings) { ... }` has the same dependency rules. Naming the class does not create an ambient binding or change how it receives capabilities. The top-level expression is optional; empty input is represented as `EmptyExpression` and is a no-op for expression evaluation. The module-instantiation entry specifically requires a class literal; it rejects empty input and other expression forms without evaluating them.

2.2 Four Sources of Class Capabilities

A class literal, whether named or anonymous, obtains its required capabilities through four mechanisms:

1. **Superclass inheritance:** inherited behavior and the capabilities available through that behavior.
2. **Mixin composition:** behavior contributed by composed mixins, preserving their definition scopes and the capabilities available through that behavior.
3. **Lexical nesting:** bindings available from the enclosing scope in which the class is defined.
4. **Constructor parameters:** existing objects supplied when an instance is constructed.

These describe where capabilities come from, not a new lookup priority. The language's [lookup rules](#) still distinguish lexical scope, composition and inheritance. Calling a class from another scope does not give it that caller's lexical bindings. Constructor arguments supply object references, not an implicit caller environment.

Constructor parameters are initializer-only bindings. Persistent behavior uses values explicitly bound to instance slots, or deliberate captures of expressions created during initialization. Member-class declarations access the enclosing instance and declaration origin; they do not capture its constructor parameters. Named and anonymous class expressions retain their actual lexical scopes.

Capabilities include ordinary VarioHyve objects, callable class objects, and [Aliens](#) representing foreign services. Sending messages to an available object may return further objects; such access derives from that object's authority and is not an additional ambient acquisition mechanism.

2.3 Top-Level Classes and Instantiation

Normal module evaluation starts in an **empty top activation** at the base of the lexical chain. This is an activation object, not a separate environment or an implicit platform object. It contributes no capability bindings and has no capability-bearing parent or receiver. Keeping this execution frame is compatible with saying that a top-level class has **no enclosing application scope**: the empty root supplies no interesting capabilities.

Under this normal entry contract, the top-level class inherits only from the language root **Object** and **composes no mixins**. The empty top activation supplies no other superclass or mixin to resolve. The root Object relationship is part of the object model; it does not supply an ambient platform namespace or host services. Internal lexical links may retain the empty activation without making it an enclosing application object exposed through outer.

This is VarioHyve's Newspeak-inspired module discipline: dependencies are supplied as objects when a module is instantiated, rather than imported from global scope.

Consequently, **constructor parameters are the only way to supply external capabilities to a top-level module**. There can be zero, one, or several parameters, with the normal declared arity. No name such as `kernel` or `platform` is reserved or required. A zero-parameter top-level module receives no external capabilities through construction.

The instantiation service acquires or already holds the existing objects needed by the module, then supplies them as arguments. Those objects may come from previously constructed modules, explicit platform assembly, or foreign systems through the Alien/Expat boundary. How that service acquires them is external to the module's lexical scope. The module cannot discover the service's other objects, its registry, or its caller scope. Objects intentionally passed as arguments retain their own behavior and

reachable object graphs.

Constructor parameters do not make a top-level superclass or mixin clause legal: class structure is established before that class's constructor receives arguments. A module can instead use supplied class/mixin objects when defining **nested** classes inside the constructed module:

```
class Module(baseClass, loggingMixin, output) {
  Base = baseClass;
  Logging = loggingMixin;
  console = output;

  class Worker(identifier) <: Logging < Base {
    id = identifier;
    report { console.print(id) }
  }
}
```

Module has only root Object inheritance and no composition. Its nested Worker uses all four routes: Base, Logging, the enclosing console, and its own identifier argument. The same rules hold for anonymous class literals.

2.4 The Kernel Parameter

A kernel is an **optional ordinary capability bundle**, not the unique parameter or a privileged entry mechanism. An application may choose a bundle, several independent service objects, or a mixture:

```
class(kernel, consoleService, repository) {
  null = kernel.null;
  true = kernel.true;
  false = kernel.false;
  console = consoleService;
  records = repository;

  run { console.print(records.summary) }
}
```

Here kernel is one ordinary argument. A `class(kernel) { ... }` example is simply a one-argument convention. The instantiation service must honor the module's declared constructor interface rather than always inserting one kernel. There is no global environment or implicit platform lookup.

2.4.1 Consequences

- **Explicit dependencies.** Superclass inheritance, mixin composition, lexical nesting and constructor parameters are the four class capability routes. The top-level restrictions leave parameters as the external entry route.

- **Testability.** Supply alternatives for individual arguments or for a bundle without changing the class's lookup or invocation semantics.
- **Analysis boundary.** An analysis must account for all supplied arguments and their reachable behavior. Supplying abstract input objects alone does not implement abstract interpretation; domains, effects and machine semantics also need an appropriate analysis model.
- **Isolation depends on reachability.** A nested class can access its enclosing scope, and inherited/composed behavior retains its own capabilities. Passing a narrow argument does not remove those other routes. Separate top-level modules with appropriately restricted argument graphs provide a clearer boundary.

2.4.2 Optional Bootstrap Bundle

A platform may bundle canonical null/Boolean values, library objects, collections and I/O capabilities for convenience. Such a bundle's inventory is a platform contract, not a required module signature. Services can equally be supplied as separate arguments. The current bootstrap exports `null`, `true` and `false`; `nil` may be a platform alias for the same null object, not a second null value.

The [implemented native Boolean protocol](#) uses `eager` and/or `and` and `thunk-taking` `andThen/orElse`, with `ifTrue`, `ifFalse` and `not`. Numeric, string and collection protocols remain library milestones. The inventory of one bootstrap bundle must not be mistaken for the language's only possible set of constructor parameters.

2.5 Bootstrap Conformance Requirements

These requirements describe normal module entry with an empty top activation. The explicitly supplied tooling contexts below have a different set of available objects, while using the same language lookup and invocation rules.

The visitor bootstrap implements this entry through `BootstrapRuntime.instantiateModule(source, maxSteps, arguments...)`. It accepts named and anonymous class literals with zero, one or multiple constructor parameters, preserves argument identity/sharing through the ordinary boundary, and rejects arity mismatches. It creates an empty top activation with no platform/service bindings, parent or receiver, uses only structural root `Object` inheritance, and rejects every explicit top-level superclass or mixin clause. Constructor arguments do not make such clauses legal.

The result is the constructed module instance as an `Expat`. The host explicitly chooses any subsequent entry message; the service does not guess `main` or `run`. Definition, construction and synchronous callbacks share one transition budget. Supplied objects retain their reachable capabilities and lexical captures. See the [entry contract](#).

`BootstrapRuntime.evaluate` remains a distinct tooling expression context with the assembled kernel as its lexical parent and receiver. It does not enrich the empty module root. Nested classes can use explicitly available superclass and direct mixin definitions, with initializer arguments routed from their own constructor bindings. See the [implemented construction contract](#) for composition limits and the [bootstrap roadmap](#).

2.6 Development and Debugging Contexts

Tools may explicitly supply capabilities for development, debugging and design-time expression evaluation. For example, an evaluator can provide a console, test objects or a mirror service through its evaluation activation or an enclosing tooling object. A debugger can evaluate in an explicitly selected activation, using that activation's existing lexical bindings. These contexts need not be empty.

This uses the same object-based lexical scope, lookup, invocation and Alien/Expat protocol as ordinary execution. It is not an ambient global registry or a fifth capability mechanism: the tool explicitly supplies objects to the evaluation context. It must distinguish this context from normal module entry and must not silently enrich the empty root used by ordinary modules.

Objects, closures and classes created in a tooling context retain their actual definition scopes. A class evaluated with tool-supplied lexical bindings may resolve superclass or mixin objects from that scope under the usual rules; it is not an isolated module defined over an empty top activation. Moving such a class to normal module entry requires making its dependencies explicit, rather than silently dropping captured references or preserving hidden tool authority. See [Tooling Architecture](#).

2.7 Error Handling

Status: v1 design is intentionally minimal.

The bootstrap reports fatal runtime errors, including:

- **Missing selector** — an ordinary send finds no exact selector, or invocation exhausts exact operations and eligible generated callable bindings at the requested arity. The bootstrap reports a diagnostic; it does not invoke a language-level `doesNotUnderstand:` handler.
- **Dead context** — a `^` return targets a method or initializer home that has retired. See [Literals — Early Return](#).
- **Premature initializer completion** — a return targets an initializer whose slot phase has not completed, including its final slot expression. See [Initializer completion](#).

These are **fatal language failures** with diagnostic messages, not recoverable language exception values in the current bootstrap. Public failures do not expose raw activation addresses or host exceptions. Boundary failures preserve their defined kinds; see [Foreign Objects](#). See [Future Directions](#) for DNU plans.

3 Literals

The grammar defines 6 literal forms. Two are primitive value literals (number, string); one is an incomplete-expression literal (hole); and three are structural literals (lambda, class, object).

```
expression:
  numberLiteral      // 42, -3.14
| stringLiteral      // "hello", 'world'
| holeLiteral        // ?
| lambdaLiteral      // fun(x) { ... }
| classLiteral       // class Point(x, y) { ... }
| objectLiteral      // { ... }
| ...               // message sends, parens
;
```

3.1 Number Literals

```
numberLiteral : '-'? NUMBER ;
NUMBER       : NATURAL ('.' NATURAL)? ;
fragment NATURAL : [0-9]+ ;
```

Numbers are signed decimal values. An integer has no decimal point; a float has exactly one.

```
42
-7
3.14159
-0.5
```

Numbers are value objects. All arithmetic is done via message sends:

```
x.plus(1)
x.times(y)
x.greaterThan(0)
```

3.2 String Literals

```
STRING : ''' (ESC | ~["\\])* '''
       | '\"' (ESC | ~['\\])* '\"'
```

```

;
fragment ESC : '\\\ ' (["\\\/bfnrt"] | 'u' HEX HEX HEX HEX) ;

```

Strings can use either double or single quotes. Both forms support the same escape sequences:

Escape	Meaning
\\	backslash
\"	double quote
\'	single quote
\/	forward slash
\b	backspace
\f	form feed
\n	newline
\r	carriage return
\t	tab
\uXXXX	Unicode code point

```

"hello, world"
'it\'s fine'
"line one\nline two"
"\u0041" // "A"

```

Single and double quoted forms are identical at runtime — choose whichever avoids escaping the more common quote character in the content.

3.2.1 Multi-Line Strings

The grammar allows raw newlines inside string literals — no special syntax is needed:

```

"line one
line two
line three"

```

This is equivalent to "line one\nline two\nline three". Use whichever form is more readable. Raw multi-line strings preserve the exact whitespace between the quotes, including leading indentation.

3.3 Hole Literals

```

holeLiteral : HOLE ;
HOLE : '?' ;

```

A hole is written as `?`. It is an explicit marker for an intentionally incomplete expression:

```
threshold = ?;

compute(x) {
  x.plus(?)
}

controller = (? : expected(Controller) : reason("provided by integration
  ↪ team"));
```

A hole may appear anywhere an expression is expected. It is not a parse error and it is not an omitted expression. It is part of the program's syntax and is represented explicitly in the AST.

This distinction is important:

```
x = ?;    // initialized by an explicit hole
x = ;     // syntax error or parser recovery: missing expression
x;        // uninitialized slot, defaults to nil/null according to the active
  ↪ kernel
```

A hole is also distinct from `null`. It does not denote absence. It denotes an unresolved value or expression that must be supplied, interpreted, synthesized, or otherwise handled by the active execution or tooling context.

3.3.1 Descriptors on Holes

Like other expressions, a hole may carry descriptors through parenthesized expression syntax:

```
speed = (? : range(0, 300) | units("km/h"));

selectController() {
  (? : expected(PIDController) : satisfies(StabilityContract))
}
```

Descriptors do not change the base syntax of the hole. They provide additional information for tools, abstract interpreters, model checkers, synthesis engines, documentation generators, or interactive environments. For example, a concrete interpreter may use a descriptor to display a better prompt, while an abstract interpreter may use it to choose an abstract value domain or constraint.

3.3.2 Runtime Semantics

Evaluating a hole does not produce a normal VarioHyve value by itself. Instead, it delegates to the active hole-resolution policy of the current runtime or kernel.

Different runtimes may choose different policies:

Runtime / tool context	Possible interpretation
Concrete interactive interpreter	Suspend execution and ask the user/tool to provide a value or replacement expression
Batch interpreter	Raise an <code>UnresolvedHole</code> runtime error
Test kernel	Fail the test with a diagnostic pointing to the hole
Abstract interpreter	Treat the hole as an unknown abstract value, possibly constrained by descriptors
Model-checking kernel	Branch over a bounded or descriptor-constrained set of possible values
Synthesis tool	Attempt to synthesize an expression satisfying the surrounding context and descriptors
LSP / IDE	Report an unresolved-hole diagnostic and offer code actions to fill it

The exact policy is not fixed by the surface language. The language only specifies that `?` is an explicit hole expression and that reaching it during execution must be handled by the active runtime policy.

3.3.3 Hole Identity

Each syntactic occurrence of `?` denotes a distinct hole. Tools and interpreters should assign a stable internal hole identity to each occurrence, typically derived from the AST node and source location.

For example:

```
x = ?;  
y = ?;
```

contains two different holes, even though they have the same spelling.

Stable hole identity is useful for diagnostics, LSP code actions, external annotations, abstract interpretation, trace replay, and interactive execution. A tool may refer to a hole through a descriptor-supplied name:

```
gain = (? : controllerGain);
```

or through a qualified path derived from the surrounding syntactic context.

3.3.4 Holes and Partial Programs

Holes support partial programs and partial models. They allow a VarioHyve artifact to remain syntactically well-formed even when some design decisions are still open.

This is especially useful in system-of-systems and federated modeling contexts, where different stakeholders may progressively supply behavior, configuration values, interface

adapters, or analysis assumptions. A hole marks an explicit design opening: something is known to be missing, and the program records that incompleteness directly.

3.4 Lambda Literals

```
lambdaLiteral : LAMBDA arguments LB statementList RB (COLON descriptorList)*
               ↪ ;
LAMBDA        : 'fun' | 'λ' ;
arguments     : (LP formallist? RP)? ;
statementList : statement (SEMICOLON statement)* SEMICOLON? ;
```

A lambda is an anonymous callable. Its body is a `statementList` — a sequence of statements separated by `;`. Parameters are optional.

```
fun { 42 }                // 0-arg lambda, implicit return of 42
fun(x) { x.plus(1) }      // 1-arg lambda, implicit return
fun(x, y) { x.plus(y) }   // 2-arg lambda, implicit return
λ(n) { n.times(2) }       // Unicode keyword form
```

3.4.1 Implicit Return

Both methods and lambdas implicitly return the value of their **last expression** — no `^` required:

```
fun(x) { x.plus(1) }      // returns x+1
fun(x) { x.plus(1); x.times(2) } // returns x*2 (last expression)
```

```
getX() { x }              // method – implicit return of x
add(a, b) { a.plus(b) }   // method – implicit return of sum
```

This is one rule applied uniformly. There are no special cases.

3.4.2 Early Return with `^`

`^` returns to the lexically enclosing **method or initializer** home. The home is fixed by creation context, not by the dynamic caller; invoking a lambda does not create another return home.

- **In a method body:** `^` exits the method, returning the given value.
- **In a lambda inside a method:** `^` exits the enclosing method, not the lambda.
- **In an initializer or its lambda:** `^` targets that initializer invocation. It fails during slot initialization; in post-slot code it completes only that contribution, ignoring the returned value. See [Initializer completion](#).

```

find(items, pred) {
  items.forEach(fun(item) {
    pred(item).ifTrue(fun { ^ item })    // ^ exits find, not forEach or
↪ ifTrue
  })
  nil                                     // implicit return – not found
}

```

Another example — the ^ in each lambda returns directly from search:

```

search(items, idx) {
  idx.eq(items.size).ifTrue(fun { ^ notFound })    // exits search
  items.at(idx).matches.ifTrue(fun { ^ items.at(idx) }) // exits search
  search(items, idx.plus(1))                       // implicit return
}

```

When is ^ necessary? Only for early exit. If the value you want to return is naturally the last expression, ^ is not needed. If you need to exit a method from inside a lambda (or from a branch before the last line), use ^.

Dead context risk: A lambda with ^ captures its enclosing method or initializer home. If it executes a return after that home has retired, a runtime error occurs. Captured data remains usable; it does not preserve return authority. A live home is necessary but not sufficient: initializer slot-phase returns and returns crossing an active Alien callback barrier are also rejected.

Lambdas capture the lexical activation/object scope at their creation site. Internal scope links preserve those bindings; source outer is a declaration-relative enclosing-object operation and does not expose the captured activation.

Every initializer invocation establishes a fresh home. Class expressions retain definition-time bindings without retaining that method's return target for future initialization. An outside lambda passed to an initializer keeps its own home and may unwind construction to it; it is not rebound to the initializer.

Lambdas supply the exact apply/N capability for their parameter count. A generated binding containing a lambda can supply invocation at that arity; a wrong-arity lambda is a lookup miss, not a reason to invoke it and catch an error. For a binding `f = fun { 7 };`, bare `f` returns the lambda and `f()` returns 7. Direct `f.apply()` also invokes it once. See [Slots and Accessors](#).

Lambdas can carry optional descriptors after a ::

```

fun(x) { x.plus(1) } : MyAnnotation

```

3.5 Class Literals

namedClassLiteral

```

      : CLASS IDENT arguments mixinApplications superclassApplication
      LB classBodyElement* initializerBody? RB (COLON descriptorList)* ;
anonymousClassLiteral
      : CLASS arguments mixinApplications superclassApplication
      LB classBodyElement* initializerBody? RB (COLON descriptorList)* ;
classLiteral
      : namedClassLiteral | anonymousClassLiteral ;
arguments          : (LP formallList? RP)? ;
mixinApplications  : (WITH expressionList)? ; // expressionList – comma-
separated, allows multiple mixins
superclassApplication: (EXTENDS expression)? ;
classBodyElement   : namedClassLiteral SEMICOLON? | methodDecl SEMICOLON?
                  | slotDecl SEMICOLON? | factoryDecl SEMICOLON? ;
initializerBody    : PIPE statementList? ;

```

The class keyword is **required** for all class definitions. A literal without class is always a singleton object (see [Object Literals](#) below).

Form	Result
class { body }	anonymous class — zero parameters
class() { body }	anonymous class — zero parameters (equivalent to above)
class(params) { body }	anonymous class with parameters
class Name { body }	named class — zero parameters
class Name() { body }	named class — zero parameters (equivalent to above)
class Name(params) { body }	named class with parameters

Forms with and without empty parentheses are exactly equivalent. Both produce a zero-parameter class. This follows from the arguments grammar rule — (LP formallList? RP)? — which makes the parenthesized form optional. This allows homogeneous syntax when defining families of classes with varying parameter counts.

Named class expressions carry an identifier after class. Both named and anonymous expression forms capture the lexical activation/object scope in which they are evaluated. An expression's name is retained as declaration metadata; it does not introduce a binding in the surrounding body.

A named class in bodyElement position is a **member-class declaration**. Its synthesized zero-arity accessor binds the declaration to the enclosing instance and caches that class per receiver and declaration occurrence. It does not capture the enclosing instance's constructor parameters, and it is not sugar for a slot initializer. A backend may use hidden storage to cache the class, but must retain its declaration identity, definition scope and source origin.

An invocation candidate can inspect the member declaration's constructor arity and owned factory apply signatures before realizing the class. A rejected arity evaluates no superclass or mixin targets. A selected member is realized normally: its contributor receiver expressions may compute class values, with mixin targets evaluated left-to-right before the superclass target. The terminal application routes arguments for later instance

initialization; receiver subexpressions run as ordinary computations. See [constructor clauses](#).

```
class Container(argument) {
  saved = argument;
  class Member { read { saved } }
  Expression = class Named { read { argument } };
}
```

`Member.read` uses the enclosing instance's slot. `Expression` is initialized by a class expression that deliberately captures `argument`. Its behavior is the same if the name `Named` is omitted.

Member names and slot names share a namespace; duplicates in one body are semantic errors. A member has no generated setter. Member declarations remain available even before their textual position is reached by ordinary slot initialization. Named forms are allowed in declaration and expression positions; anonymous forms occur only in expression positions. See [Objects and Classes](#).

A body can contain member-class declarations, slot declarations, method declarations, and factory declarations, followed by an optional `final` | initialization section. Factory methods (`factory methodDecl`) define alternate constructors on the metaclass — see [Object Model — Metaclass Details](#).

Full examples, semantics, inheritance (`<`), mixins (`<:`), and the `class` keyword are covered in [Objects and Classes](#).

The runtime store structure for class literals is covered in [Object Model — Store Entries](#).

3.6 Object Literals

```
objectLiteral : LB objectBodyElement* initializerBody? RB (COLON
↪ descriptorList)* ;
objectBodyElement      : namedClassLiteral SEMICOLON? | methodDecl SEMICOLON?
                        | slotDecl SEMICOLON ;
```

An object literal `{ body }` creates a singleton — it is evaluated immediately and produces exactly one instance. It never has a parameter list and never carries the `class` keyword.

```
config = {
  debug = false;
  verbose = true;
}
```

An object literal creates three store entries: the instance, its anonymous class, and its anonymous metaclass. See [Object Model — Store Entries](#).

3.7 Initializer completion

A class or object body may end with `|` and an ordinary statement list. These statements execute once, after that contribution's ordered slot declarations:

```
class Config(input) {
  saved = input;
  value { saved }
  |
  ^ "done"
}
```

Successful `new` returns the allocated instance, not "done". An object literal uses the same form, for example `{ value = 42; | ^ 99 }`. An empty `|` section is allowed. No member/method/slot declarations follow the delimiter; ordinary statements, including local `var` and class expressions, are permitted. Attached descriptor lists retain `|` as their separator. End a descriptor-bearing method, member-class or factory declaration with the optional `;` to separate it from post-slot code, for example `value { saved } : Tag; | performWork`. Constructor parameters are visible in both initialization phases; ordinary methods still cannot resolve their own constructor parameters, even if called by either phase.

Each invocation of each initializer contribution has its own fresh private return home. During slot initialization a return targeting that home is an error, including a return inside the last slot expression. The declaration completes only after its value has been assigned. Post-slot code may complete normally or use `^ value`, directly or inside a captured lambda; both results are ignored. An early completion finishes only this contribution. The enclosing construction schedule continues with any later contributions and ultimately returns the receiver. Supplied outside lambdas retain their existing lexical homes.

Homes retire on normal completion, early completion, failure and unwinding. Escaped lambdas/classes may retain bindings without preserving the right to return to those homes. Failures do not replay initialization or roll back effects, and an already escaped receiver is not retracted. The Alien callback barrier is unchanged. See the [accepted policy and implementation sequence](#).

4 Objects and Classes

This section covers the semantics of object and class literals: nesting, inheritance, mixins, and naming conventions. For the syntax of all literal forms (including number, string, and lambda), see [Literals](#).

4.1 Capabilities and Definition Scope

Named and anonymous class literals receive capabilities through superclass inheritance, mixin composition, their enclosing lexical scope, and constructor parameters. These routes preserve the distinction between lexical and inherited lookup; they do not give a class access to its caller's scope.

A normal top-level class is a module whose lexical base is an empty top activation. It has no enclosing application scope, only root `Object` inheritance, and no mixin composition. Its external capabilities arrive solely as existing objects passed to its constructor by an instantiation service. Neither a single argument nor the name `kernel` is required. Constructor parameters can supply class/mixin objects for nested class definitions, but cannot supply the top-level class's own superclass or mixin clauses. See [Program Structure](#).

Tools may explicitly provide a richer context for development-time evaluation. Classes created in that context retain its lexical bindings, so they are not isolated module definitions over the empty root. They use the same class and lookup semantics; see [Evaluation Contexts](#).

Examples below that refer to enclosing bindings, custom superclasses or mixins are fragments within an appropriate enclosing scope, not standalone top-level modules. A zero-parameter nested class can still receive capabilities through the other three routes; a zero-parameter module over the empty top activation has no external inputs.

4.2 The `class` Keyword

The `class` keyword is **required** for all class definitions. It is the sole discriminator between classes and singleton objects:

- `class { ... }` or `class(params) { ... }` — a class (instantiable via `.new` or the `apply` protocol).
- `{ ... }` — a singleton object (evaluated immediately, one instance).

4.2.1 Zero-Parameter Classes vs Singletons

```
class Config {  
  debug = false
```

```

    verbose = true
}

```

`Config` is a class — `Config()` selects its advertised `apply/0`, realizes the class if needed, and invokes that factory. `Config.new` and `Config.new()` both invoke primary construction once. See [zero-arity invocation](#). By contrast:

```

config = {
  debug = false
  verbose = true
}

```

`config` is a slot holding a singleton object — there is only one instance, created at definition time.

4.2.2 Member Declarations and Class Expressions

A named class in body-declaration position declares a **member class**:

```

class Container(input) {
  value = input;
  class Member { read { value } }
  Captured = class { read { input } };
  NamedCapture = class Snapshot { read { input } };
}

```

`Member` binds to the enclosing `Container` instance and its declaration origin. It can access that instance's stored value, but cannot implicitly use the instance's constructor parameter `input`. Its zero-arity accessor caches the bound class per enclosing receiver and declaration occurrence. Repeated access returns the same class, and different enclosing instances obtain distinct class objects. Member declarations are available regardless of their textual position among ordinary slot initializers.

`Captured` and `NamedCapture` are ordinary slots initialized by class expressions. Both expressions capture the initializer scope, including `input`. Naming an expression does not change this rule or declare a member in the surrounding body. Class expressions may occur wherever expressions are allowed.

These forms share callable class behavior, but are **not** semantically equivalent. Lowering must preserve member declarations instead of turning them into slot assignments. Member names and slot names share the same namespace; a duplicate in the same body is an error. A member accessor has no setter. An ordinary mutable slot holding a class can be changed through its setter. Both support invocation at their available `apply/N` arities. A stored class is checked through its callable capability; a member declaration advertises its constructor arity and owned factory `apply` signatures before realization. A wrong-arity member candidate is skipped without executing its definition clauses.

Methods do not implicitly retain their own class's constructor parameters. Initialization must bind the values needed by later methods into slots. Ordinary lambdas and named/anonymous class expressions evaluated during initialization may deliberately retain their lexical captures. See [Name Resolution](#).

4.2.3 Initializer Bodies

An optional final `|` section runs statements after the contribution's ordered slots. Constructor parameters remain available there. For example:

```
class Config(input) {
  saved = input;
  read { saved }
  |
  ^ input
}
```

Construction returns the allocated `Config`, ignoring the initializer's return value. Each invocation has a fresh return home. Returns to it during slot initialization are errors; post-slot returns complete only that contribution, without skipping others. Objects use the same final-section syntax and home lifecycle. Methods and deliberately captured data keep the scope distinctions above. See [Initializer completion](#).

4.3 Nesting

Classes and objects nest lexically. Inner objects access outer slots via the lexical chain:

```
{
  config = { debug = false }
  class Server(port) {
    start() {
      // 'config' resolves via lexical chain to the enclosing object's
      ↪ config
      config.debug
    }
  }
}
```

4.4 Inheritance

Classes can extend a superclass:

```
class Animal() {
  speak() { "... " }
```

```

}
class Dog() < Animal {
  speak() { "woof" }
}

```

The < (or extends) clause specifies a native class as superclass. It inherits that class's complete chain: its owned contribution, applied mixins and ancestors. Lookup reaches that chain after the receiving class's own and applied contributions. Construction runs the superclass's complete initialization schedule on the same allocated receiver. This is instance-side inheritance: it does not import the superclass's factories. Each class's meta-class contains only its own class-side contribution and directly extends common Class; see [Bootstrap Metaclasses](#).

Note that class Animal() uses the class keyword, making it a class. Writing Animal = { speak() { "..."} } would create a singleton object instance — not a class — and < Animal would pass that instance where a class is expected. Always use the class keyword when defining types intended for inheritance.

The runtime does not auto-dereference singletons. Given a singleton instance, the runtime will *not* silently follow instance.class to obtain its anonymous class. This is a deliberate design choice:

- **Ambiguity:** both classes and instances are objects. The runtime cannot reliably distinguish “you passed a class” from “you passed an instance whose anonymous class you intended.” Auto-deref would guess wrong in the common case of a slot holding a class reference.
- **Mixin composability:** super dispatch depends on a well-defined class chain. Silently inheriting from an anonymous class that was never designed for subclassing produces fragile, hard-to-debug lookup chains.
- **Static analysis:** a clear “this must be a class” constraint at < sites is easy to verify abstractly. Auto-deref adds a branching point that complicates hierarchy tracing.

4.5 Mixins

The <: (or with) clause applies mixins:

```

class Serializable() {
  serialize() { ... }
}
class Loggable() {
  log() { ... }
}
class Dog() <: Serializable, Loggable < Animal {
  speak() { "woof" }
}

```

A <: A(args) application selects **only the owned reified mixin of A**: its directly declared methods, slots, member classes and own initializer. It imports neither A's applied mixins nor its superclass or factory methods. The routed arguments bind A's formals for

this invocation; A's original superclass/mixin argument routes are not evaluated. Its definition scope remains captured normally. This holds even when A itself is inherited or composed.

Thus `class A <: M1, M2 < S1`, `class B <: M3, M4 < S2` and `class C <: A < B` give the lookup order `M(C)`, `M(A)`, `M(B)`, `M(M3)`, `M(M4)`, then the full S2 chain. Here `M(X)` means X's owned mixin. Initialization runs in reverse contribution order, with a fresh home for each invocation. See [Mixin Application](#) for scope, occurrence identity and state rules.

4.6 Computed Contributor Targets

Superclass and mixin receiver expressions may perform ordinary computations to select their contributor classes. For example, `< registry.choose(kind).Base(x)` evaluates `registry.choose(kind)` and reads its `Base` class during class realization. Only the terminal `.Base(x)` application has construction semantics: `x` is routed when constructing an instance, rather than invoking a factory at class realization. Parentheses around the whole clause do not change that rule.

This applies to member classes as well as class expressions. Direct mixin targets run left-to-right, followed by the superclass target. A member accessor realizes and caches its class for the enclosing receiver and declaration occurrence; subsequent constructions reuse those contributor choices. Invocation first checks member factory signatures, so rejecting a wrong-arity candidate evaluates none of its target computations. A selected realization failure is propagated and cached. See [constructor-clause semantics](#) and [member-class invocation](#).

4.7 Descriptors

Object and class literals can carry optional descriptor chains for use by future tooling, compiler, or runtime extension layers. See [Descriptors](#) for details. Descriptors have no effect on raw execution.

5 Slots and Accessors

Slots are the storage units of objects. Every slot declaration synthesizes accessor methods that follow the uniform message send convention.

5.1 Slot Declaration

Slots are declared in object and class bodies using `slotDecl`:

```
{  
  x = 0  
  y = 0  
  name  
}
```

- `x = 0` — slot `x` initialized to 0
- `name` — uninitialized slot (defaults to `nil`)

The `=` in a slot declaration is **initialization**, not assignment. It appears only at definition time.

5.2 Contribution Slots and Storage

A slot declaration has a local position in its owned reified mixin. Each application of that mixin creates a distinct logical occurrence of the slot, including repeated applications to one class. A superclass retains its complete set of occurrences. The actual receiver's class owns the immutable mapping from these logical slots to its storage locations.

The current policy maps same-named slots from different occurrences to one shared location. Accessor precedence remains owned contribution, applied mixins left-to-right, then superclass. After lookup selects a generated accessor, that accessor uses its occurrence and local slot position to select the physical location. A flat backing array does not replace the source-level message send or prevent an explicit method from overriding an accessor.

Declaration initialization uses the initializing occurrence's mapping directly, without invoking an overridable setter. The existing superclass, right-to-left mixin, then owned initialization schedule remains unchanged; later writes to a shared location replace earlier values. Storage sharing does not merge initializer invocations or their effects. Explicit sharing specifications and contribution-private access rules remain deferred. See [the implementation decision](#).

5.3 Synthesized Accessors

Every slot `x` synthesizes two accessors with distinct selectors:

Syntax	Selector	Behavior
<code>x</code>	<code>x/0</code>	getter — returns the current value of <code>x</code>
<code>x!(v)</code>	<code>x!/1</code>	setter — updates <code>x</code> and returns <code>v</code>

Invocation syntax `x(args)` is not another synthesized accessor. It selects behavior at the requested arity. When the effective getter is generated, its result can supply that behavior through an exact `apply/N` capability. Bare `x` returns the value; `x()` invokes its `apply/0` if available. A stored non-callable value still supports bare reads but supplies no invocation candidate. See [Message Sends](#).

The setter uses a separate name, `x!`, so it does not conflict with invocation through `x` at arity one.

```
class Point(a, b) {
  x = a
  y = b
  move(dx, dy) {
    x!(x.plus(dx))      // setter — updates x, returns new value
    y!(y.plus(dy))      // setter — updates y
  }
  getX() { x }          // getter — reads x
}
```

5.4 The apply Protocol and Callable Objects

An object is **callable at arity N** when it provides an exact `apply/N` method. Classes and lambdas implement `apply` by default:

- **Class objects:** `(apply, N)` delegates to `(new, N)`, which allocates and initializes a new instance.
- **Lambdas:** `(apply, N)` invokes the lambda body.
- **User objects:** any object can implement `(apply, N)` to become callable.

5.4.1 Interaction with the Apply Protocol

At each lexical scope or hierarchy contribution, invocation `x(args)` prefers an exact operation `x/N`, then an eligible generated getter whose result supports `apply/N`. Continue searching on a genuine miss. A wrong-arity lambda or non-callable value supplies no operation at that arity; an uninitialized read or selected operation failure remains an error.

For implicit invocation, an eligible declaration first selects its lexical receiver. The original `x/N` invocation then dispatches uniformly on that receiver: an overriding method can replace the generated getter/apply route. The eligibility result is retained if dispatch uses

that route. An ineligible accessor continues lexical search, even when the same receiver has an unrelated `x/N` method. See [uniform virtual invocation](#).

This rule also applies to `x()`: a method `x { 7 }` executes once, whereas a slot containing `fun { 7 }` delegates to the lambda's `apply/0`. A generated getter alone does not satisfy invocation merely because its own arity is zero.

```
{
  Point = class(x, y) { };
  handler = fun(req) { req };

  test(request, OtherClass, newFn) {
    Point;                // returns the class object
    Point(2, 3);           // invokes the stored class's apply/2
    handler;               // returns the lambda
    handler(request);      // invokes its apply/1
    Point!(OtherClass);    // replaces the class value
    handler!(newFn)        // replaces the lambda value
  }
}
```

Replacing a slot's value can change the arities available under its name. A later invocation may consequently select a farther scope or contribution. The candidate's value and callable operation are retained once selected; invocation never retries another candidate after failure.

Generated member-class accessors use declaration-derived factory signatures instead of stored-value inspection. Their classes are realized only when the accessor/apply route wins virtual dispatch. See [member-class invocation](#) and the [accepted design](#).

5.5 Setter Return Value

The return value of synthesized setters is **the new value** — the RHS expression. `x!(5)` evaluates to 5.

Generated setters do not return `self`. Thus `p.x!(2).y!(3)` sends `y!(3)` to 2, and fails if that value has no applicable operation. An overridden setter returns whatever its method body returns; neither `x!(v)` nor `x = v` forces the result to equal `v`. Use a cascade to send each message to the same receiver:

```
p..x!(2)..y!(3)    // correct – both messages go to p
```

See [Message Sends — Cascade](#) for the full comparison. For the `=` grammar sugar form of setter calls, see [Syntactic Sugar — = Setter Sugar](#).

5.6 Shadowing Accessors

An explicit method definition with the same selector as a synthesized accessor replaces it:

```

{
  x = 0
  x() { computedX }           // custom getter – replaces synthesized
  ↪ getter
}

```

An overridden getter no longer participates as a generated accessor. Lookup neither executes the overriding method to discover a callable nor reads the hidden backing storage. This masking persists during inheritance search. The overriding method itself still matches its declared arity: `x()` executes a custom `x/0` once, while `x(1)` does not execute it to inspect its result. To retrieve and invoke such a result explicitly, write `self.x.apply(1)`.

This distinction preserves accessor overriding and predictable ordinary reads, but generated getters returning callables and computed getters are not interchangeable for invocation syntax.

Setter shadowing is discouraged. Once a setter is overridden, the synthesized slot-write operation is no longer reachable — calling `x!(v)` from inside the custom setter dispatches to itself, causing infinite recursion. There is no built-in mechanism for “raw” slot access.

Two workarounds exist:

1. The backing-slot pattern — use a separate slot for internal storage and expose the public name via custom accessors:

```

{
  _x = 0                       // backing slot – private by convention
  x() { _x }                   // custom getter
  x!(v) { validate(v); _x!(v) } // custom setter – writes to backing slot
}

```

2. Privileged access (future) — a reflection API or kernel-provided primitive that enables resend-semantics or direct slot writes, bypassing the method dispatch chain. This is not available in v1 but is planned as part of the [Mirror-Based Reflection API](#).

`var` Locals

Local variables declared with ``var`` in method/lambda bodies follow the same message convention. Their generated accessors are supplied by the current activation object; reading or updating a local does not use a separate value lookup mechanism. Activations remain backend-private:

```

```vh
test() {
 var count = 0
 count!(count.plus(1)) // setter – updates local, returns new value
 count // getter – reads local
}

```

## 5.7 Parameter Immutability

Parameters expose **read-only accessor messages** and their names follow the same noun rule — no ! suffix is permitted. No setter (name!) is synthesized for parameters:

```
move(dx, dy) {
 dx!(5) // ERROR – dx is a parameter, no setter exists
}
```

To get a mutable copy:

```
move(dx, dy) {
 var localDx = dx // mutable local, initialized from parameter
 localDx!(localDx.plus(1)) // OK – var locals have setters
}
```

## 5.8 The ! Convention

The ! suffix is a **selector-only** convention. It may appear only on method selectors (verbs) — it is not permitted in slot names, parameter names, or local variable names (nouns).

Syntactic role	! allowed?	Examples
Slot name	<b>no</b>	x, count, handler
Parameter name	<b>no</b>	dx, value, n
var local name	<b>no</b>	acc, result
Method selector	<b>yes</b>	sort!, reset!, x!
Message selector	<b>yes</b>	x!(v), sort!()

This noun/verb split makes ! a reliable signal: whenever you see a !-suffixed name you are reading an effectful operation, never a data declaration. It also keeps the namespace of callable objects clean — a slot named handler will never collide with a setter named handler!.

```
List {
 sort() { ... } // return new sorted list – pure, no !
 sort!() { ... } // sort in place – mutation signaled by !
}
```

Synthesized setters follow this convention: a slot x gets x! as its setter. Because slot names cannot end in !, setter names are always unambiguously derived — x! can only mean the setter for slot x, never a slot named x!. Users can define their own !-methods to signal mutation or side effects (similar to Ruby and Scheme).

### 5.8.1 What ! Signals — and What It Does Not

! marks **invocation semantics**: calling this selector causes mutation or a side effect. It says nothing about the nature of the *value stored* in a slot.

This means a slot holding an effectful callback cannot use ! in its name:

```
{
 onCommit = fun(v) { db.write(v) } // stored effectful callback
}
onCommit(v) // invokes the stored lambda at arity one, causing side
 ↪ effects
```

The slot name onCommit looks pure. There is no way for the slot name alone to announce that invoking the stored value has side effects — that information lives in the lambda itself, not in the slot declaration.

This is a deliberate constraint: ! marks an *operation that is itself defined as effectful*, not a slot that *happens to hold something effectful*. The tradeoff is clarity at the method boundary vs. expressiveness in slot naming. Users coming from Ruby, where `attr_accessor :commit!` is idiomatic, will need to adjust their mental model.

The practical mitigation is to make effectful callbacks **explicit methods** rather than stored slots:

```
{
 _onCommit = fun(v) { db.write(v) } // internal backing slot, plain name

 onCommit!() { _onCommit(lastValue) } // explicit effectful method with !
 setOnCommit!(fn) { _onCommit!(fn) } // setter for the stored callback
}
```

Or, when the effectful nature should be visible at every call site, use a thin delegation method:

```
class SortingAdapter {
 inner = ...

 sort!() { inner.sort! } // explicit delegation – ! visible at definition
}
```

This pattern also applies to future **tear-off** forwarding: you cannot store a torn-off ! method in a slot that mirrors the original name, but you can always delegate explicitly:

```
// Not possible – slot name cannot end in !
// sort! = collection.sort!$

// Idiomatic replacement:
sort!() { collection.sort! }
```

### 5.8.2 Reading VarioHyve Assignment Patterns

For programmers coming from imperative languages, the `x!(value)` setter convention may look unusual. Here is a quick mapping to the canonical form:

Other languages	VarioHyve canonical	Notes
<code>x = 5</code> (assignment)	<code>x!(5)</code>	returns 5
<code>x = x + 1</code>	<code>x!(x.plus(1))</code>	getter + op + setter
<code>obj.x = 5</code>	<code>obj.x!(5)</code>	explicit receiver
<code>x = f(a)</code>	<code>x!(f(a))</code>	setter with any expression

The `same =` token introduces slot/local initialization in declarations and setter invocation in expressions, including arguments and initializer bodies. Declaration initialization and setter invocation have different semantics; see [Syntactic Sugar — = Setter Sugar](#).

## 5.9 Descriptors on Slots

Slots can optionally carry descriptor chains for use by future tooling or runtime extensions. See [Descriptors](#) for details. Descriptors have no effect on raw execution.

## 6 Message Sends

All computation in VarioHyve is expressed through message sends. There are no operators, no field access, no function calls — only messages dispatched to receivers.

### 6.1 Implicit Message Send

A bare name is an implicit zero-argument send. Parentheses request invocation at the written arity; both forms use declaration-relative lexical lookup:

```
x // ordinary x/0 send: execute an accessor or method
x() // invoke zero-argument behavior named x
x!(5) // invoke x!/1; a synthesized setter stores and returns 5
foo(a, b) // invoke foo/2 through a method or eligible accessor result
```

`x` and `x()` remain different forms. For a method `x { 7 }`, either executes that method once and returns 7. For a slot containing `fun { 7 }`, bare `x` returns the lambda and `x()` invokes it. A method returning a callable does not implicitly invoke its result. This distinction lets `Factory` return a class object and `Factory()` invoke its zero-argument factory.

### 6.2 Self-Dispatch Message Send (Leading Dot)

A message prefixed with a bare `.` — no expression to the left — sends the message directly to `self`, bypassing the lexical chain entirely:

```
.x // send 'x' to self – inheritance chain only
.x!(5) // send 'x!(5)' to self – setter, returns 5
.move(1, 2) // send 'move(1,2)' to self
```

This is the `#SelfMessageSendExp` grammar rule. It is semantically distinct from an implicit message send:

Form	Grammar rule	Lookup
<code>x</code>	<code>#ImplicitMessageSendExp</code>	Locals → lexical chain (enclosing objects) → self's class hierarchy
<code>.x</code>	<code>#SelfMessageSendExp</code>	Self's class hierarchy <b>only</b> — lexical chain skipped
<code>self.x</code>	<code>#MessageSendExp</code>	Self's class hierarchy only — identical to <code>.x</code>

`.x` and `self.x` are fully equivalent. **Prefer `self.x` for clarity** — the explicit receiver makes intent obvious, especially in code with nested classes. Use the leading-dot shorthand only in tightly scoped contexts where `self` is unambiguously clear.

The distinction between `x` and `.x` is **not stylistic** — it matters inside nested classes where the lexical chain reaches into an enclosing object:

```
class Outer {
 x = 0
 class Inner {
 process() {
 x!(5) // lexical lookup → finds Outer's x! → mutates
 ↪ Outer's slot
 .x!(5) // self-dispatch only → Inner has no x →
 ↪ doesNotUnderstand
 self.x!(5) // identical to .x!(5), explicit form
 }
 }
}
```

### 6.3 Explicit Message Send (Dot)

The `.` operator sends a message to an explicit receiver. The same distinction between ordinary sends and invocation applies:

```
p.x // ordinary x/0 send: return the accessor/method result
p.x() // invoke zero-argument behavior named x on p
p.x!(5) // send 'x!(5)' to p – setter, returns 5
p.move(1, 2) // send 'move(1,2)' to p
```

Dot-chains evaluate left to right. Each message is sent to the result of the previous:

```
a.b.c(3)
// equivalent to:
// temp = a.b
// temp.c(3)
```

This means `a.b.c(3)` sends `b` to `a`, then sends `c(3)` to whatever `b` returned. If `b` returns a different object, `c(3)` goes to that object — not to `a`.

#### 6.3.1 No Operator Precedence

Because VarioHyve has no binary operators — only message sends — there is no operator precedence. Dot-chains evaluate strictly left to right:

```
3.plus(4).times(2) // (3 + 4) * 2 = 14, not 3 + (4 * 2)
a.greaterThan(b).and(c.lt(d)) // explicit grouping required
```

To control evaluation order, use parentheses to nest receiver expressions:

```
3.plus(4.times(2)) // 3 + (4 * 2) = 11
```

This is a direct consequence of the “everything is a message send” principle: arithmetic expressions are just dot-chains, and all dot-chains evaluate left to right. The planned [extensible parser](#) will introduce operator syntax with conventional precedence rules, desugaring back to message sends.

## 6.4 Dispatch Rules

Invocation selects an operation by name and arity. Native methods, stored lambdas, stored native callable objects and member-class factories can supply the same invocation selector. Their bodies retain their own receiver, lexical scope and return home; invocation equivalence does not change those contexts.

All these routes consist of message sends: either the direct name/N operation, or a generated name/0 accessor followed by apply/N on its result. Locals and parameters also have generated accessor messages on backend-private activation objects. The same message protocol applies throughout. The VM may implement a generated accessor’s known behavior with an internal slot primitive and may inspect member declaration metadata for eligibility. These are implementations of the message protocol, not additional source-level ways to access values.

### 6.4.1 Candidate Selection

At each searched lexical scope or hierarchy contribution for name/N:

1. Prefer a directly matching operation. At  $N = 0$ , a generated getter is considered under the accessor rule below rather than accepted as the operation.
2. Otherwise consider the effective generated name/0 accessor. A storage accessor qualifies only when its value supplies exactly apply/N. A member-class accessor qualifies through its declaration’s factory signatures.
3. On a miss, continue to the next scope or contribution. Exhausting lookup produces a missing-selector diagnostic.

Implicit invocation searches declaration-relative lexical scopes before a self invocation. Explicit native invocation searches contributions in dispatch order: owned contribution, direct mixins left-to-right, then the superclass chain. `super` begins after the executing contribution’s exact application occurrence. The candidate rule is identical at every level, so an own callable slot can beat an inherited exact method.

For implicit invocation, a declaration-relative scope first establishes eligibility: it declares the requested operation, or it declares name/0 whose effective accessor on that scope’s receiver is generated and qualifies at the requested arity. Then dispatch the original name/N

invocation normally on that receiver, regardless of which kind of declaration established eligibility. An overriding `name/N` method can replace an eligible stored callable or member factory; an eligible callable accessor can replace a method. The winning behavior keeps its own receiver, declaration origin and lexical context. A candidate that fails eligibility continues outward before any self invocation; an unrelated method on an inner receiver cannot capture an eligible outer dependency.

An overridden generated getter is not eligible: do not execute the overriding method and do not inspect its backing storage. Getter masking persists down the hierarchy, even if the overriding getter does not supply the requested invocation arity. An ordinary zero-argument method directly matches `name/0`, but is not executed to discover behavior at other arities. Explicitly retrieve its result when that is intended, for example `obj.name.apply(arg)`.

For native values, capability selection looks for an exact `apply/N` method through the value's hierarchy, or checks a lambda's declared arity. It does not recursively treat a stored `apply` value as an `apply/N` method. An Alien value returned by an eligible generated accessor can qualify through its adapter's effect-free `apply/N` selection; `ForeignCallable` remains usable. A send whose explicit receiver is Alien selects the requested foreign `name/arity` directly. It does not speculatively invoke a foreign `name/0` operation to discover a callable.

Receiver and arguments are evaluated once, left-to-right, before selection. Retain an eligible generated accessor's result and selected callable operation through virtual dispatch. Revisiting it uses this retained result, including any selected foreign operation, rather than repeating the read or resolution. A wrong-arity or non-callable value is a miss; an uninitialized read or a selected operation's failure is an error. Neither errors nor non-local control flow cause lookup to resume or effects to replay. Foreign selection failures are errors, not selector misses. Eligibility precedes virtual dispatch: its uninitialized read or foreign selection failure remains an error even when that receiver has an overriding method at the requested arity.

### 6.4.2 Invocation at Arity Zero

`name()` uses the same candidate rule as every other invocation:

- A zero-argument method executes once and returns its result.
- A generated storage getter supplies a candidate only if its stored value has `apply/0`; invoke that operation once.
- An eligible member-class accessor supplies its class's `apply/0`.

Bare `name` remains an ordinary `name/0` send and returns the accessor or method result without testing callability. Consequently replacing a stored callable with a computed getter preserves bare reads but may change invocation behavior. Accessor overrides are legal; ordinary methods do not acquire generated-accessor semantics.

```
{
 method { 7 }
 thunk = fun { 7 };
 returnsThunk { fun { 7 } }
}
```

For this object, `method` and `method()` return 7; `thunk` returns the lambda and `thunk()` returns 7; `returnsThunk()` returns its lambda without invoking it. Write `returnsThunk().apply()` to perform both operations explicitly.

### 6.4.3 Member-Class Invocation

The generated accessor for a member class advertises callable arities from its selected declaration: the constructor's arity and any owned factory `apply` methods. Flat metaclasses import no superclass or mixin factories, so neither contributor lookup nor class realization is needed to discover these arities. An overriding member declaration supplies its own signatures. An ordinary method overriding the accessor has no generated-accessor metadata.

```
class Widget(x, y) {
 factory apply(x) { new(x, x) }
}
```

`Widget` is an ordinary access returning the class object. `Widget(1)` and `Widget(1, 2)` match the advertised arities. `Widget()` is a miss at this declaration, without realizing it, and lookup may continue elsewhere.

For implicit invocation, metadata eligibility first selects the lexical receiver. If virtual dispatch selects a method instead of the member factory, the member remains unrealized. When final dispatch selects the member factory, commit to that candidate and realize its class through the normal cache keyed by enclosing receiver and declaration occurrence. Class definition may execute computations to obtain mixin and superclass receivers: direct mixin targets are evaluated left-to-right, followed by the superclass target. Their results identify the actual contributors and must be native class values, excluding metaclasses. Routed constructor arguments remain deferred until instance construction. Realization failures are cached and propagated, not converted into arity misses.

Thus no rejected member-class candidate executes definition clauses, but the selected member class can use ordinary capability-bearing computations in its definition. See the [accepted design](#).

### 6.4.4 Complete Dispatch Walkthrough

Given an object with this shape:

```
{
 slot1 = 32;
 slot2 = fun(x) { x.plus(slot1) };
 method1(a) { a.plus(slot1) }
 answer { 7 }
 class NoArg { }
 class WithArgs(x, y) { }
}
```

Expression	Selected behavior	Result
<code>a.slot1</code>	Ordinary generated getter	32
<code>a.slot1()</code>	No callable candidate	Missing-selector error
<code>a.slot2</code>	Ordinary generated getter	The lambda
<code>a.slot2(3)</code>	Stored lambda's apply/1	35
<code>a.slot2()</code>	No candidate at arity zero	Missing-selector error
<code>a.method1</code>	No method1/0	Missing-selector error
<code>a.method1(2)</code>	Exact method1/1	34
<code>a.answer()</code>	Exact answer/0, once	7
<code>a.NoArg</code>	Cached member-class accessor	Class object
<code>a.NoArg()</code>	Declared class apply/0	Instance
<code>a.WithArgs(1, 2)</code>	Declared class apply/2	Instance

### 6.4.5 Reading Convention

Parentheses request invocation; bare selectors request ordinary zero-argument sends:

Syntax form	Meaning	Example
<code>name</code>	ordinary send — lexical lookup, then run the accessor/method	<code>count</code> , <code>describe</code> , <code>Factory</code>
<code>name()</code>	invoke zero-argument behavior once	<code>Factory()</code> , <code>thunk()</code>
<code>name(args)</code>	invoke matching method or callable accessor result	<code>move(1, 2)</code> , <code>Factory(x, y)</code>
<code>name!(v)</code>	invoke setter; generated setters return <code>v</code> , custom methods return their own result	<code>x!(5)</code>
<code>.name</code>	self-dispatch — skip lexical chain, send to self's class hierarchy	<code>.count</code> , <code>.describe</code>
<code>.name(args)</code>	self-dispatch with args	<code>.move(1, 2)</code>
<code>self.name</code>	same as <code>.name</code> — explicit form, prefer for clarity	<code>self.count</code>
<code>outer C.name</code>	qualified outer-dispatch — select receiver for lexically enclosing declaration <code>C</code>	<code>outer Counter.count</code> , <code>outer Module.log(x)</code>

**Receiver origins and selector names are distinct.** `self` denotes the current native receiver, while `alien.self` is an ordinary selector sent through the Alien boundary. Native receivers have no synthesized or user-defined `self` selector. Using `self` as a value passed to another object remains supported.

**Native outer is a qualified lexical receiver.** `outer C` selects the ordinary receiver associated with the nearest lexically enclosing declaration named `C`. A subsequent `.message` uses normal dispatch. Bare `outer` and native `expr.outer` are invalid; no universal parent getter or activation mirror is provided. Explicit Alien selectors `outer`, `self` and `super` are

eligible for foreign dispatch and have only the meanings supplied by that adapter. See [Name Resolution](#).

A normal top-level class has an empty top activation as its lexical base, with no enclosing application scope. Its outer path must not expose the instantiation service or an implicitly injected platform. External capabilities enter that module through its declared constructor arguments; see [Program Structure](#). Explicit tooling contexts can supply other lexical objects for evaluation; ordinary scope and dispatch rules apply to those objects as well.

For the = grammar sugar forms of all setter variants, see [Syntactic Sugar — = Setter Sugar](#).

Classes implement `apply` for construction (the default `apply` delegates to `new`). Lambdas implement `apply` for invocation. Any user object can implement `apply` to become callable. See [Slots and Accessors](#).

## 6.5 Cascade (..)

The cascade operator `..` sends one or more messages to the **same receiver**, ignoring intermediate return values. The receiver is evaluated once. Arms execute left-to-right, with each arm's arguments evaluated immediately before that send. Each arm preserves ordinary bare-send versus parenthesized-invocation behavior, and can use canonical messages or assignment sugar:

```
p.x!(2)..y!(3)..move(1, 1)
```

Its evaluation is equivalent to saving the receiver and sequencing sends (the actual lowering uses a private accessor name that cannot collide with source names):

```
var receiver = p;
receiver.x!(2); // return value discarded
receiver.y!(3); // return value discarded
receiver.move(1, 1) // the cascade expression returns the result of the
↪ last send
```

The key difference from dot-chains:

Expression	Receiver of each message
<code>p.x!(2).y!(3)</code>	<code>p</code> receives <code>x!(2)</code> , then <b>the result of <code>x!(2)</code></b> receives <code>y!(3)</code>
<code>p..x!(2)..y!(3)</code>	<code>p</code> receives <code>x!(2)</code> , then <b><code>p</code></b> receives <code>y!(3)</code>

Cascade does not depend on intermediate results. Generated setters return the new value, so `p.x!(2).y!(3)` sends `y!(3)` to 2. Custom setters may return anything; cascades still send every arm to the saved receiver. A failing arm or non-local return stops the cascade before later arms or their arguments execute. Earlier effects are neither undone nor replayed.

## 6.6 Cascade Return Value

The cascade expression evaluates to the result of the **last** message in the chain:

```
result = (p..x(2)..y(3)..describe)
// result is whatever p.describe returns
```

Thus `p..a..b..c` returns the last send's result. To return the receiver, write it explicitly after the cascade, for example `p..a..b; p`.

## 6.7 Implicit-Receiver Cascade (Leading `..`)

A cascade prefixed with `..` — no expression to the left — sends all messages directly to `self`. This is the `#SelfCascadeExp` grammar rule:

```
..x!(2)..y!(3)..move(1, 1)
// equivalent to:
self..x!(2)..y!(3)..move(1, 1)
```

Each message is dispatched through the inheritance chain only — the lexical chain is not consulted. **Prefer the explicit `self..` form for clarity.**

```
class Point {
 x = 0;
 y = 0;
 reset() {
 ..x!(0)..y!(0) // sends x!(0) then y!(0) to self
 }
}
```

### 6.7.1 `m..n..p` vs `..m..n..p`

These two cascade forms are **semantically different**:

Expression	Grammar rule	Cascade receiver
<code>m..n..p</code>	<code>#CascadeExp</code> (receiver = <code>#ImplicitMessageSendExp</code> <code>m</code> )	The <b>result</b> of evaluating <code>m</code> via lexical lookup
<code>..m..n..p</code>	<code>#SelfCascadeExp</code>	<code>self</code> directly — no lexical chain consulted

In `m..n..p`, `m` is evaluated first through the full lexical chain; whatever it returns becomes the cascade receiver for `n` and `p`. In `..m..n..p` all three messages go to `self`:

```

handler..notify..cleanup // evaluate handler via lexical chain, cascade
↪ on its result
..notify..cleanup // both messages go to self directly

```

## 6.8 Combining Dot and Cascade

Dot and cascade can be mixed. The cascade receiver is the expression to the left of the first ..:

```

a.b..c(1)..d(2)
// cascade receiver is (a.b)
// sends c(1) to (a.b), discards result
// sends d(2) to (a.b), returns result

```

The grammar does **not** accept an unparenthesized dot after a cascade:

```

a..b..c.d // parse error
(a..b..c).d // send d to the last cascade result
a.b..c..d // save a.b, then send c and d to that receiver

```

This follows from expression: a cascade has a `nonCascadeExpr` receiver and message arms, while a dot send's receiver must itself be a `nonCascadeExpr`. Parentheses turn a full expression into an eligible receiver. Arguments can contain nested cascades without extra grouping, as in `a..take(b..x..y)..finish`. Assignment's RHS has its own [cascade fence](#).

There is no `super..a..b` form: `super` is not a receiver value. A completed `super` send can supply a cascade receiver, as in `super.make..a..b`; the arms are normal sends on the value returned by `super.make`, not additional `super` sends.

These forms execute on the bootstrap VM. Lowering saves an explicit receiver in a private activation slot and sequences ordinary core sends. Leading `..` uses self sends. The lowering preserves lexical captures and non-local return homes without exposing a new scope object to programs. See [vh.g4](#) and the [source regressions](#).

## 7 Message Send, Invocation, Apply, and Tear-Off

Sections 1–5 specify accepted invocation behavior. Sections 6–9 describe a **deferred tear-off proposal**, not executable syntax. The detailed lookup rules are in [Message Sends](#) and the accepted [unified callable lookup decision](#).

### 7.1 1. General Principle

VarioHyve has no public direct slot access or separate function-call mechanism. Ordinary access sends a selector; invocation requests behavior by name and arity. Methods, generated accessors returning callables and member-class factories can supply the same invocation selector. `apply/N` is the callable capability, not an instruction to execute arbitrary getters until some result can be called.

Slots, locals and parameters have generated storage accessors. A named nested class is a distinct **member-class declaration**, exposed through a generated accessor that binds and caches the class by enclosing receiver and declaration occurrence. It is not equivalent to a slot initialized by a class expression: that expression captures its evaluation scope normally, whereas a member class does not capture its enclosing instance’s constructor activation.

### 7.2 2. Zero-Arity Messages

A bare selector is an ordinary zero-argument send:

```
obj.size
obj.name
obj.reset!
```

It returns the selected accessor or method’s result. Parentheses request invocation, including at arity zero:

Definition	<code>obj.f</code>	<code>obj.f()</code>
<code>f { 7 }</code>	7	7
<code>f = fun { 7 };</code>	The lambda	7
<code>f { fun { 7 } }</code>	The lambda	The lambda
<code>f = 7;</code>	7	No candidate here; continue lookup

Invoke a returned lambda explicitly with `obj.f().apply()`. Invocation never automatically executes the result of a directly selected zero-argument method.

### 7.3 3. Invocation Syntax

For `obj.m(e1, ..., en)`, evaluate the receiver and arguments once, left-to-right. Then search the receiver's contributions in dispatch order. At each contribution:

1. Prefer an exact `m/n` operation. At zero arity, generated getters are candidates for an accessor/apply route under the next step, not directly callable behavior.
2. Otherwise consider the effective generated `m/0` accessor. A storage accessor qualifies when its stored value provides exactly `apply/n`. A member-class accessor qualifies when its selected declaration advertises that factory arity.
3. Continue on a miss; report a missing selector if no candidate exists.

An implicit `m(args)` searches declaration-relative lexical scopes under this same eligibility rule, then invokes on `self` if none matches. An eligible scope selects its receiver; `m/n` then dispatches uniformly on that receiver, whether eligibility came from a method or generated accessor. An overriding method may replace the accessor/apply route. Eligibility is checked first and its result is retained if dispatch uses that route. `super` searches after its exact contribution occurrence. An unrelated method on a different receiver cannot capture an eligible lexical dependency.

Do not run computed getters while testing candidates or read behind an overridden getter. Getter masking persists through inheritance search. A wrong-arity or non-callable stored value is a miss. An uninitialized read, foreign resolution failure, selected class realization failure, or invocation failure is an error; none causes lookup to resume. Retain the chosen callable value and operation so selection is not repeated after effects.

For an explicit `Alien` receiver, select the requested foreign name/arity through its boundary adapter. Foreign getters are not generated storage accessors and are not executed to discover other operations.

### 7.4 4. Callable Objects and Member Classes

Native values qualify through exact `apply/n` methods (including inherited methods), or a lambda's declared arity. A stored `Alien` can advertise an exact `apply/n` through the boundary's effect-free selection contract, including `ForeignCallable`. Candidate testing never invokes the operation itself.

A generated member-class accessor knows its callable arities from the selected class declaration: constructor arity plus owned `factory apply` signatures. Flat metaclasses do not import contributor factories. Thus a wrong-arity lookup can reject a member class without executing any class-definition clauses.

When its accessor/apply route wins dispatch, the cached accessor realizes the class. Mixin and superclass targets may be computations that select the actual contributor classes. Evaluate direct mixin targets left-to-right and then the superclass target; routed constructor arguments wait for instance construction. Bare class access also realizes the class. Both forms use the same cache and propagate realization failures. See [member-class invocation](#).

For a declared class `Point(x, y) { }`, `Point` returns its class object and `Point(1, 2)` calls the class's `apply/2`. `Point()` is a miss at that declaration, without realizing it, unless the declaration also supplies `factory apply/0`.

## 7.5 5. No Implicit Tear-Off

`obj.m` sends `m/0`; it does not return a bound method object. `obj.m(a, b)` selects and invokes behavior at arity two. Method tear-off remains a separate, explicit proposal below and is not part of the executable language.

---

## 7.6 6. Postfix Tear-Off Syntax

The following design is a proposal. Its syntax is not implemented and remains subject to revision.

A method can be reified as a value using postfix arity syntax:

```
receiver.selector#arity
```

For example:

```
obj.move#2
obj.size#0
obj.x!#1
```

The expression:

```
obj.move#2
```

means:

```
evaluate obj;
lookup the actual method (move, 2) on obj;
return a bound method object for that receiver and method.
```

The result is a bound method value. Applying it later invokes the selected method with the original receiver as `self`.

Proposed tear-off lookup selects an actual method. It does not select callable generated getter/apply routes or member-class factory candidates.

Therefore:

```
handler(event)
```

may succeed through a generated getter returning an `apply/1` callable, while:

```
handler#1
```

requires an actual method handler/1 to exist.

---

## 7.7 7. Tear-Off as a Special Getter-Like Form

A tear-off form behaves like a special zero-arity getter.

```
obj.m#3
```

selects and returns the bound method object for m/3.

This means that immediate application of a tear-off does not require a special syntax category.

```
obj.m#3(e1, e2, e3)
```

is interpreted as:

```
select obj.m#3;
then apply the resulting bound method object with arguments e1, e2, e3
```

That is, it follows the same general invocation principle as other callable values.

The bound method object implements apply/3. Applying it with any other arity is an arity error.

Therefore:

```
obj.m#3(e1, e2)
```

is invalid because the selected bound method expects arity 3.

No dedicated TearOffInvoke construct is required.

---

## 7.8 8. Proposed Tear-Off Examples

```
p.move#2
```

Would return a bound method object for the actual `move/2` method of `p`.

```
p.move#2(1, 1)
```

Would select that bound method and invoke it with two arguments. These examples are proposal syntax, unlike currently supported generated accessors and `apply`.

## 7.9 9. Representation Consequences

The core must preserve the distinction between ordinary sends and invocation syntax, as well as implicit, explicit and `super` receiver origins. Invocation cannot be lowered unconditionally into a getter followed by `apply`: a direct method must execute only once, and an ineligible accessor must leave lookup free to continue. `super` retains the exact selected contribution occurrence, rather than merely the superclass of the receiver.

A future tear-off expression could reify the selected method and receiver as a bound callable. No implemented `CoreTearOff` or public method-reification API is implied by this proposal. Its detailed syntax and representation remain deferred.

## 8 Syntactic Sugar

VarioHyve is a minimal language. Its behavioral interactions use message sends, with distinct native receiver forms: `self`, `outer ClassName`, and `super` dispatch. Bare `self` and qualified `outer` produce receiver values; they are not ordinary getter sends. `outer` is contextual syntax, not a universal native accessor. An explicit Alien receiver may nevertheless support a foreign selector named `outer`, `self` or `super`. The grammar provides a small number of syntactic sugar forms, each with a precise desugaring to the canonical selector-based model.

This section documents each sugar, its desugaring, its scope of validity, and any constraints that arise from the grammar.

---

### 8.1 = Setter Sugar

The `=` token has declaration and expression uses. In slot declarations it introduces initialization (`x = 0;`); in var declarations it introduces local initialization (`var x = 0;`). In expression position, including arguments, returns and initializer bodies, it is grammar-level sugar for a setter call. It does not create a binding. Lookup follows the canonical `name!(v)` form. The authoritative syntax is the `message`, `explicitMessage`, `expression` and `nonCascadeExpr` rules in [vh.g4](#).

#### 8.1.1 Desugaring

`name = expr` in expression position desugars to `name!(expr)`, requesting the `name!/1` invocation selector. The receiver, when explicit, is evaluated first, then the RHS once, then the selected operation executes. Assignment has no privileged storage-write path; custom setters and foreign setters use ordinary dispatch and return their own results.

```
x = 5 // desugars to: x!(5)
p.x = 5 // desugars to: p.x!(5)
```

#### 8.1.2 Valid Contexts

`=` setter sugar works in every context where a message send is valid — implicit, self-dispatch, explicit receiver, `super`, and cascade:

Sugar form	Canonical form	Lookup
<code>x = v</code>	<code>x!(v)</code>	Lexical chain → self's class hierarchy
<code>.x = v</code>	<code>.x!(v)</code>	Self's class hierarchy only
<code>self.x = v</code>	<code>self.x!(v)</code>	Self's class hierarchy only
<code>p.x = v</code>	<code>p.x!(v)</code>	p's class hierarchy
<code>super.x = v</code>	<code>super.x!(v)</code>	Super-dispatch
<code>outer C.x = v</code>	<code>outer C.x!(v)</code>	Qualified lexical receiver for declaration C
<code>p..x=v..y=w</code>	<code>p.x!(v); p.y!(w)</code>	Cascade — both messages go to p
<code>..x=v..y=w</code>	<code>self..x!(v)..y!(w)</code>	Self cascade — skips lexical receiver lookup

The explicit cascade evaluates `p` once; the table's sequential spelling assumes an already saved receiver. Native `super` is a redirector, so `super..x=v` is not grammar syntax. `super.x = v` is supported. An explicit Alien receiver can use selectors such as `alien.outer = v`, which requests foreign `outer!/1`.

Assignment sugar accepts an unsuffixed selector before `=`. For a selector with additional bangs, write a canonical send explicitly; `x! = v` is not sugar.

```
consume(x = 7) // assignment can be an argument
var result = x = 7; // the declaration initializes result with the setter
↳ result
```

### 8.1.3 Return Value

`=` returns exactly the result of `name!(v)`. A synthesized setter stores and returns the RHS value:

```
x = 5 // evaluates to 5
a = b = 3 // b = 3 evaluates to 3; a = 3 evaluates to 3; result is 3
```

Chaining is right-associative: `a = b = 3` means `a!(b!(3))`. A custom `b!/1` returning 7 passes 7 to `a!/1`, even though the innermost RHS is 3. Custom setters need not mutate storage, return the supplied value, or return `self`. Missing setters fail normally; assignment cannot mutate a read-only parameter without resolving some other applicable setter under the ordinary lookup rules.

### 8.1.4 The Cascade Fence

The RHS of `=` is a `nonCascadeExpr` — it cannot contain an unparenthesised cascade. This is what allows `..` to act as the natural terminator in a cascade chain and makes `r..a=3..b=5` parse correctly as `r.a!(3)` then `r.b!(5)`:

```

r..a=3..b=5 // correct – desugars to r.a!(3); r.b!(5)
x = p..a..b // parses as (x = p)..a..b; cascades on the setter result
x = (p..a..b) // OK – parens admit a full expression

```

This is a grouping boundary, not a prohibition of cascades after assignment. With a generated setter, the ungrouped form stores `p` in `x`; the grouped form stores the last cascade result. Declaration initializers use full expression, so `var x = p..a..b`; and a slot declaration `x = p..a..b`; initialize `x` with the last result directly.

Canonical invocation arguments also accept full expressions:

```

p..x!(someValue)..y!(otherValue)
x!(p..a..b) // the argument is a full cascade expression

```

### 8.1.5 Setter Sugar vs ! — When to Use Which

Both forms dispatch the same selector. The choice is one of readability style:

- **name!(v)** — use in method definitions, custom setters, shadowing accessors, symbolic dispatch, and any context where the selector name needs to be fully visible.
- **name = v** — use in method bodies and lambdas for straightforward slot updates where the setter nature of the operation is already clear from context.

There is no semantic difference. Both appear interchangeably in cascade chains:

```

p..x!(2)..y!(3) // canonical in cascade
p..x=2..y=3 // sugar in cascade – same dispatch

```

### 8.1.6 Language Mapping

For programmers coming from imperative languages, assuming generated setters:

Other languages	Canonical	Sugar	Notes
<code>x = 5</code> (assignment)	<code>x!(5)</code>	<code>x = 5</code>	returns 5, not <code>self</code>
<code>x = x + 1</code>	<code>x!(x.plus(1))</code>	<code>x = x.plus(1)</code>	getter + op + setter
<code>obj.x = 5</code>	<code>obj.x!(5)</code>	<code>obj.x = 5</code>	explicit receiver
<code>x = f(a)</code>	<code>x!(f(a))</code>	<code>x = f(a)</code>	setter with any expression
<code>a = b = 3</code> (chain)	—	<code>a = b = 3</code>	right-associative; same notation as most languages

Declaration initialization reserves and initializes a binding. Assignment sugar instead invokes behavior through a setter selector. The production bootstrap supports both assignment sugar and cascades through surface-to-core lowering; see the executable [source regressions](#).

## 9 Name Resolution

Messages in VarioHyve are late-bound. A bare name `x` is an implicit send: lexical declaration context first selects a receiver; dispatch then selects behavior. If no lexical declaration supplies the requested selector, lookup falls back to the current receiver's inheritance chain. Lexical receiver selection is not a search of arbitrary members added by a subclass.

### 9.1 Lookup Order

1. **Local scope** — parameters and var locals of the current method/lambda
2. **Lexical chain** — slots and methods of enclosing objects, from innermost to outermost
3. **Mixin chain** - mixins methods, in declaration order - left to right
4. **Inheritance chain** — superclass methods, from most specific to most general

The first match wins. There is no global environment.

#### 9.1.1 Implicit invocation and callable accessor results

Locals, parameters and slots are reached through generated accessor messages. For locals and parameters, the receiver is the backend-private activation object. There is no separate lookup namespace or value-resolution mechanism. An invocation is implemented by a direct `f/N` message or by the generated `f/0` accessor followed by `apply/N` on its result.

A message selector is a name/arity pair. For `f(args)` at arity `N`, including zero, examine each declaration-relative lexical scope from nearest to farthest:

1. Prefer an exact operation `f/N` declared by that scope. At arity zero, a generated getter is an accessor candidate under the next rule, not an operation that satisfies the invocation by returning its value.
2. Otherwise, if the scope declares `f/0`, consider its effective generated accessor. A storage accessor supplies an invocation candidate only if its current value supports exactly `apply/N`. A member-class accessor supplies one only if its selected declaration's constructor or owned factory `apply` signatures include arity `N`.
3. Continue outward when the scope supplies neither applicable candidate. If all lexical scopes miss, perform a self invocation through the receiver's composition and superclass chain, with the same per-contribution rule.

Once a scope qualifies, dispatch the original `f/N` invocation uniformly on that scope's receiver. Eligibility selects the receiver; it does not fix the implementation to the method or accessor that established eligibility. Dynamic dispatch uses the same per-contribution candidate rule, so an overriding method can replace a callable accessor route and an overriding callable accessor can replace a method. Lookup never resumes at another lexical scope after choosing a receiver.

An activation supplies its parameter/local accessors. A lexical object scope supplies declarations from its originating contribution, not arbitrary members added by a subclass. If its declaration supplies only  $f/0$ , first select the effective  $f/0$  accessor on that scope's receiver and establish eligibility there. A subclass  $f/N$  alone cannot make an ineligible accessor qualify. Once eligible, that receiver's  $f/N$  overrides participate normally. A method added on a different, inner receiver cannot capture a dependency whose eligible declaration belongs to an enclosing receiver.

At arity zero, an ordinary method overriding the declared getter also qualifies as an exact  $f/0$  operation. Its body executes once; its result is not implicitly applied.

Only the **effective generated getter** can supply an accessor candidate. An ordinary method overriding that getter is not executed to discover a callable, and lookup never inspects the storage hidden behind it. The same masking rule applies while traversing inheritance; an overridden accessor cannot reappear as a hidden accessor candidate farther down the chain. Exact methods at the requested arity can still match there.

```
{
 f(x) { 42 }
 run {
 var f = fun(x) { 7 };
 f(1)
 }
}.run // 7: the local accessor's result supplies apply/1
```

```
{
 f(x) { 42 }
 inner = {
 f = fun(x, y) { 7 };
 run { f(1) }
 };
}.inner.run // 42: the inner accessor's result supplies apply/2, not apply/1
```

```
{
 class Base {
 f = fun(x) { 7 };
 run { f(1) }
 }
 class Child < Base { f(x) { 42 } }
}.Child().run // 42: Base establishes eligibility; Child overrides f/1
```

The last example previously returned 7 by preserving the selected accessor route. Programs relying on that behavior must retrieve and invoke their chosen callable explicitly, for example `f.apply(1)` when the intended getter remains effective. Ordinary overriding still applies to that getter access.

Within one scope, an exact method wins over a same-named callable accessor route. Wrong-arity and non-callable accessor results are misses; an uninitialized slot is an error.

Receiver and arguments are evaluated once, left-to-right, before selection. Retain the eligible accessor result and selected callable operation through eligibility and dispatch; revisiting that candidate reuses the result without rereading storage or resolving a foreign operation again. An uninitialized read or foreign selection failure while establishing eligibility remains an error even if that receiver has an overriding `f/N` method. Failure while accessing an uninitialized slot, resolving a foreign operation, realizing a selected member class, or executing selected behavior never resumes lookup.

Native callability is checked through exact `apply/N` methods, including inherited methods, or a lambda's declared arity. Foreign values may advertise `apply/N` through the boundary's effect-free operation-selection contract; `ForeignCallable` remains supported. No candidate is tested by executing it. Changing a stored value can change which invocation selectors the scope supplies, so a later call may select a farther lexical receiver.

Bare `f` remains the ordinary `f/0` send. `f()` invokes zero-argument behavior exactly once: a method `f { 7 }` returns 7, while a generated accessor returning `fun { 7 }` makes `f()` invoke the lambda. A method that returns a lambda returns that lambda; it is not implicitly invoked again. See [dispatch rules](#) and the [accepted design](#).

Member-class candidate selection reads declaration metadata and does not realize the class. Realization occurs only if final dispatch selects that member's factory (or on an ordinary bare access), through the accessor's receiver-and-declaration cache. It may compute mixin and superclass target receivers to identify the actual contributors; these computations are not speculative lookup effects. See [member-class invocation](#).

### 9.1.2 Capabilities and declaration scopes

The four [class capability sources](#) are inheritance, mixin composition, lexical nesting, and constructor parameters; that list is not a replacement lookup order. Constructor parameters have accessors on activation objects, like other parameters. Naming a class does not alter these rules. Normal top-level classes have only an empty top activation at the lexical base, with no enclosing application scope. They inherit only from root `Object` and compose no mixins. Their external inputs are the existing objects supplied as constructor arguments, with no special kernel name or single-argument rule. The instantiating caller's scope never becomes a class's lexical scope. Tools may explicitly choose a richer [evaluation context](#); its accessors participate in ordinary lexical lookup, with no change to precedence.

Constructor parameters have immutable accessors scoped to initialization. In `class(value) { value = value; }`, the initializer reads the argument on the right-hand side and stores it in the instance slot. Instance methods cannot resolve their own constructor parameters, even when called during initialization. They use their own parameters/locals, declaration-relative receiver members and the selected definition scope. A method may independently resolve an enclosing accessor with the same spelling; constructor names are not globally reserved. A contributed mixin method retains its own definition scope, never the receiving class's initialization arguments.

A member-class declaration binds to the enclosing instance and its declaration origin. It does not capture the instance's constructor activation. Lambdas and class expressions, both named and anonymous, capture their actual evaluation scope normally. If deliberately created in an initializer, an escaped expression may therefore retain constructor parameter accessors. No parameter alone synthesizes an instance accessor, and no ordi-

nary instance retains a constructor scope merely by having been constructed. Constructor lookup never visits the caller's scope. See [the accepted scope decision](#).

Lexical receiver selection is declaration-relative and precedes dynamic dispatch. Adding an unrelated member to a subclass must not capture an inherited method's existing reference to an enclosing accessor. The ordered lookup summary above must be read with this restriction; probing arbitrary dynamic receiver members first is not an equivalent implementation. Full composition support must retain the selected declaration and application origin.

## 9.2 self — Current Receiver

`self` retains identifier syntax, but denotes the current native receiver with fixed semantic meaning. It is not a lookup of an ordinary property named `self`. Frontend and core representations must preserve this distinction.

```
class Point(x, y) {
 describe() {
 self // returns the current Point instance
 }
}
```

Because `self` is an identifier: - It parses with identifier syntax, then lowers to `CoreSelfExpression`; its value comes from the current activation rather than ordinary selector lookup - The semantic layer **rejects** definitions that shadow it — `self` cannot be used as a slot name, parameter name, or method name - Native explicit receivers do not provide a `self` selector. For an `Alien`, `alien.self` is an ordinary foreign selector, eligible for boundary dispatch; it does not necessarily return that `Alien`. Passing the native `self` value to another object remains supported. - It participates uniformly in message send chains as a receiver expression: `self.x`, `self.move(1, 2)`

### 9.2.1 Leading-Dot Shorthand for Self

Sending a message to `self` without naming it explicitly is possible with a leading `.` (the `#SelfMessageSendExp` grammar rule):

```
.x // identical to self.x
.move(1, 2) // identical to self.move(1, 2)
```

A leading `..` (the `#SelfCascadeExp` rule) cascades all messages to `self`:

```
..x!(0)..y!(0) // identical to self..x!(0)..y!(0)
```

Both shorthands skip the lexical chain and dispatch through the inheritance chain only — exactly as `self.m` does. **Prefer the explicit `self`, and `self..` forms.** Reserve the leading-dot shorthand for unambiguously scoped contexts where there are no nested classes or shadowed names.

### 9.3 outer — Qualified Lexical Receiver

An outer receiver has the form `outer ClassName`, normally used in a send such as `outer ClassName.message(args)`. The qualifier names a class declaration that lexically encloses the expression, including its immediately containing class. It selects the ordinary receiver corresponding to that declaration; any subsequent message dispatches normally on that receiver. This follows Newspeak’s declaration-relative receiver selection with VarioHyve’s expression/send syntax. This is not a claim of full Newspeak compatibility: ordinary subsequent dispatch, runtime qualifier validation and exact captured application scopes have explicit differences. See the [outer comparison audit](#), including the unresolved deeper repeated-parent case.

```
class Counter(start) {
 n = start;
 class Stepper {
 step {
 outer Counter.n!(outer Counter.n.plus(1))
 }
 }
}
```

Inside `Stepper.step`, `outer Counter.n` sends `n` to the enclosing `Counter` instance. `outer Stepper` selects the current `Stepper` receiver. The qualifier is lexical declaration metadata, not an evaluated variable, dynamic class name or request to inspect the receiver’s class. Repeated class names select the nearest matching enclosing declaration.

A lambda preserves the enclosing declaration context. Named class expressions can provide lexical qualifiers for code within their bodies; anonymous class expressions and object literals add no invented qualifier. Their enclosing named declarations remain eligible. Captured method/initializer parameters and locals are internal lexical dependencies, not activation objects exposed by `outer`.

Contributed methods use their definition and application origin to select the appropriate lexical receiver. Applying a mixin to a class declared elsewhere must not substitute the receiving class’s outer scope for the mixin’s lexical dependencies. Member classes similarly retain their declaration origin as well as their bound enclosing receiver.

Bare `outer` and unqualified `outer.message` are invalid. A native explicit receiver does not provide an outer selector. An Alien may support a foreign selector named `outer`; `alien.outer` then uses ordinary boundary dispatch. There is no universal native lexical-parent getter or public activation reification. Repeated foreign `.outer` sends have only their adapter-defined meaning. Holding another object alone does not reveal its lexical graph. Source code can deliberately return `outer ClassName` as an ordinary object value without exposing an activation.

An unknown/non-enclosing qualifier is a semantic error. The bootstrap reports this through a language diagnostic during evaluation; static validation is a later frontend improvement. A normal top-level module still has no enclosing application scope: qualifying its own declaration can select its instance, but cannot select an instantiation service, platform bundle or empty top activation. Explicit tooling scopes remain separate evaluation contexts and do not change these rules.

## 9.4 super — Lookup Redirector

`super` is a **keyword**. As a receiver origin, it is not a value: it redirects the following send in `super.message(args)` (the `SuperMessageSendExp` rule). Standalone `super` or `x = super` are parse errors. In the selector position `alien.super`, however, the spelling denotes an ordinary foreign operation, not a native lookup redirector.

```
class Animal() {
 speak() { "..."}
}
class Dog() < Animal {
 speak() { super.speak } // starts lookup from Animal, not Dog
}
```

`super` is a keyword because: - It is not a value — it cannot appear as an expression on its own - It resumes lookup after the executing contribution's exact application position, continuing through subsequent mixins and then the superclass while retaining `self` - It cannot be intercepted or overridden — that would break the inheritance contract

## 9.5 Lexical, Self, and Outer Dispatch — Compared

VarioHyve distinguishes five receiver origins, each controlling where lookup starts:

Form	Name	Lookup path
<code>m(args)</code>	Implicit / lexical	Locals → lexical chain (enclosing objects) → self's class hierarchy
<code>.m(args)</code> / <code>self.m(args)</code>	Self-dispatch	Self's class hierarchy <b>only</b>
outer <code>C.m(args)</code>	Qualified outer-dispatch	Receiver for lexically enclosing declaration <code>C</code> , then its class hierarchy
<code>super.m(args)</code>	Super-dispatch	Same receiver, starting <b>after</b> the executing contribution's exact application occurrence, then its superclass chain
<code>receiver.m(args)</code>	Explicit receiver	Evaluate receiver once; fixed native dispatch or foreign boundary dispatch according to the resulting value

These are semantic lookup origins, not a universal protocol implemented by every visible value. In particular, Aliens have no synthesized VarioHyve receiver, outer, class or mixins. See [Foreign Semantic Values](#).

In flat, non-nested code the first two forms are usually equivalent — there is no enclosing object that could shadow `self`'s methods. The difference becomes significant in nested classes:

```
class Outer {
 count = 0
 log(msg) { ... }

 class Inner {
 count = 99

 run() {
 count // lexical → finds Inner's own count (99)
 self.count // self-dispatch → Inner's count (99) same
 ⇨ result here
 outer Outer.count // outer-dispatch → Outer's count (0)
 outer Outer.log("hi") // outer-dispatch → Outer's log method
 }
 }
}
```

When the lexical chain is clear (no shadowing, no nesting), prefer the implicit form `m(args)` for brevity. Use `self.m` to be explicit about self-only dispatch. Use `outer C.m` to select a particular lexically enclosing class receiver. Use `super.m` to continue a method chain up the inheritance hierarchy.

## 9.6 Lexical Receiver Syntax

`self` retains identifier syntax with fixed semantic meaning. `outer` introduces qualified receiver syntax, like `super` introduces its lookup redirector. Neither form is a public structural accessor. Activation inspection, `thisContext` and related reflection remain outside the bootstrap.

## 9.7 Reserved Words

VarioHyve has two categories of reserved words:

### 9.7.1 Grammar Keywords

These tokens are reserved by the grammar and cannot be used as declaration identifiers. They are lexed as distinct token types. The spellings `outer` and `super` are also allowed in explicit selector position so foreign APIs do not lose those names; this does not allow native declarations to bind them.

Keyword	Purpose
<code>super</code>	Lookup redirector — <code>super.message(args)</code>
<code>outer</code>	Qualified lexical receiver — <code>outer ClassName</code>
<code>class</code>	Required keyword for class literal definitions
<code>var</code>	Local variable declaration
<code>factory</code>	Factory method declaration (targets metaclass)
<code>fun, λ</code>	Lambda literal
<code>&lt;, extends</code>	Superclass declaration
<code>&lt;:, with</code>	Mixin declaration

### 9.7.2 Semantically Reserved Identifiers

These are regular identifiers in the grammar (they parse as `IDENT` and participate in normal message send syntax), but the semantic layer **rejects** definitions that shadow them:

Identifier	Scope of reservation	Rationale
<code>self</code>	Cannot be used as a native slot, parameter or method name, or sent as a selector to a native receiver	Denotes the current native receiver. An explicit <code>Alien</code> 's selector <code>self</code> belongs to its foreign protocol.
<code>new</code>	Cannot be used as a factory method name (i.e., <code>factory new(...)</code> is rejected)	Synthesized non-overridable primary factory method on source-class metaclasses and <code>Object</code> class; metamodel objects have no public construction factories. Regular instance methods named <code>new</code> are permitted — only factory <code>new</code> conflicts. See <a href="#">The Primary Factory</a> .

The distinction matters: grammar keywords affect parsing (they cannot appear where an `IDENT` is expected), while semantically reserved identifiers parse normally but are checked during semantic analysis. This keeps the grammar minimal and allows future pseudo-variables to be added without grammar changes.

## 10 Recursion

VarioHyve fully supports recursion. Because all names are late-bound and resolved at call time via implicit message sends, recursive references work naturally without any special syntax or declarations.

### 10.1 Named Method Recursion

A method can call itself by name. The name resolves via implicit `self` dispatch:

```
{
 factorial(n, acc) {
 n.eq(0).ifTrue(fun { acc }, fun { factorial(n.minus(1), n.mul(acc)) })
 }
}
```

### 10.2 Slot-Bound Lambda Recursion

A lambda assigned to a slot can reference itself via the slot name. The name resolves via the lexical chain at call time — not at definition time:

```
{
 fact = fun(n) {
 n.eq(0).ifTrue(fun { 1 }, fun { n.mul(fact(n.minus(1))) })
 }
}
```

This works because everything is late-bound. When `fact(n.minus(1))` executes, the name `fact` is looked up through the lexical chain, finding the enclosing object's slot — which already holds the lambda.

These lambdas return their final expression normally. `^` is not a lambda-local return: here it would target the initializer that created `fact`, whose home has retired by the time the constructed object's slot is invoked. The lambda keeps its lexical bindings, but not permission to return to that completed initializer. See [initializer completion](#).

### 10.3 Local Lambda Recursion

Local variables follow the same principle:

```

compute() {
 var fib = fun(n) {
 n.lt(2).ifTrue(fun { n }, fun {
↪ fib(n.minus(1)).plus(fib(n.minus(2))) })
 }
 fib(10)
 }
}

```

No special `let rec` form is needed. The `var fib` binding exists in the local scope, and by the time the lambda body executes, `fib` resolves to itself through the lookup chain.

Using `^` in its branches would instead exit `compute` at the first base case, skipping the pending additions. Ordinary expression results return through each recursive lambda invocation.

## 10.4 Mutual Recursion

Methods within the same object can call each other freely:

```

{
 isEven(n) {
 n.eq(0).ifTrue(fun { true }, fun { isOdd(n.minus(1)) })
 }
 isOdd(n) {
 n.eq(0).ifTrue(fun { false }, fun { isEven(n.minus(1)) })
 }
}

```

Both names resolve via implicit `self` dispatch. No forward declaration is required — all methods of the object are visible to each other regardless of definition order.

Mutual recursion also works with `var`-local lambdas:

```

test() {
 var isEven = fun(n) {
 n.eq(0).ifTrue(fun { true }, fun { isOdd(n.minus(1)) })
 }
 var isOdd = fun(n) {
 n.eq(0).ifTrue(fun { false }, fun { isEven(n.minus(1)) })
 }
 isEven(10)
}

```

This works even though `isEven` references `isOdd`, which is declared *after* it. A method activation behaves as an object whose slots (all `var` locals) exist for the entire activation — not only after their textual declaration. Since names are resolved at call time, by the time `isEven(10)` executes, `isOdd` is already bound. There is no temporal dead zone.

## 10.5 Passing Recursive Lambdas as Arguments

A recursive lambda can be passed as an argument to another method and continues to recurse correctly. The lambda's lexical scope is determined at definition time — when called from a different scope, it still resolves names through its own lexical chain:

```
{
 run(f) { f(5) }

 fact = fun(n) {
 n.eq(0).ifTrue(fun { 1 }, fun { n.mul(fact(n.minus(1))) })
 }

 test() { run(fact) } // fact recurses correctly inside run
}
```

When `run` calls `f(5)`, the lambda body executes. The name `fact` is resolved via the lambda's own lexical chain — back to the enclosing object's slot — not via `run`'s scope. The call site does not matter; the definition site does.

The same works for `var`-local lambdas:

```
{
 run(f) { f(5) }

 test() {
 var fact = fun(n) {
 n.eq(0).ifTrue(fun { 1 }, fun { n.mul(fact(n.minus(1))) })
 }
 run(fact) // fact resolves via test()'s activation - works
 }
}
```

This requires that the runtime properly captures closure environments: a method activation must survive as long as any lambda references its locals.

A lambda can also be returned from a method and called later — the closure keeps the enclosing activation's data reachable. The method's return home has retired, so normal expression results still work but `^` to that home would fail:

```
{
 makeCounter(start) {
 var n = start
 fun() { var old = n; n!(n.plus(1)); old }
 }
 test() {
 var c = makeCounter(0)
 c() // 0
 }
}
```

```

 c() // 1
 }
}

```

The one case that does **not** work is an anonymous lambda that wants to recurse — it has no name to reference itself:

```

{
 run(f) { f(5) }
 test() {
 run(fun(n) {
 // cannot recurse – this lambda has no name
 // 'f' is a parameter of 'run', not in this lambda's lexical scope
 })
 }
}

```

To recurse, the lambda must be bound to a name first (slot or var local).

## 10.6 Tail-Call Optimization

v1 does not implement tail-call optimization, but the CESK-style abstract machine allocates the stack frames in the heap (either a stack in K or if store allocated continuations in S) - so no risk for stack overflow. See [Future Directions](#) for the planned generalized TCO design.

## 11 Descriptors

Descriptors are VarioHyve’s mechanism for attaching meta-information to any syntactic element. They are expressions that describe the nature, constraints, or behavior of the element they annotate.

### 11.1 Design Constraint

Descriptors are **strictly optional**. The core execution model — object creation, class instantiation, slot access, message sends, inheritance, lexical scoping, lambdas, and return — is fully expressed through the base grammar without any descriptors.

A program with all descriptors stripped must execute identically to the original. Descriptors never alter the raw execution semantics; they are hooks for future layers:

- **Compiler** — optimization hints (: cached, : inline)
- **Tooling** — documentation, visibility, IDE support (: public, : deprecated)
- **Runtime extensions** — contracts, serialization, immutability (: immutable, : serializedAs("json"))

The interpreter ignores descriptors it does not understand. This means:

```
// These two are semantically identical in the raw interpreter:
x = 3 : immutable : tracked
x = 3
```

The class keyword and the extends/with clauses are part of the **core grammar**, not the descriptor layer. They affect execution directly:

Mechanism	Layer	Needed for execution?
class(params) { body }	core grammar	yes — creates a class
< SuperExpr	core grammar	yes — sets up inheritance
<: MixinExpr	core grammar	yes — applies mixins
fun(args) { body }	core grammar	yes — creates a lambda
: module	descriptor	no — it’s just an object in the raw interpreter
: immutable	descriptor	no — mutability is the default
: cached	descriptor	no — optimization hint

### 11.2 Syntax

Descriptors follow the element they describe, separated by ::

```

element : descriptor
element : descriptor1 | descriptor2
element : descriptor1 : descriptor2

```

- | separates descriptors within a single group
- Chaining with : applies multiple descriptor groups

### 11.3 Where Descriptors Apply

Element	Example
Slot	<code>x = 0 : immutable</code>
Method	<code>move(dx, dy) { ... } : cached</code>
Object/Class literal	<code>class(a, b) { ... } : typed</code>
Lambda	<code>fun(x) { ... } : async</code>
Formal parameter	<code>(x : nonNeg, y : nonNeg)</code>
var local	<code>var count = 0 : tracked</code>
Expression (parenthesized)	<code>(3 : immutable : tracked)</code>

### 11.4 Expression-Level Descriptors

Inside expressions, descriptors require explicit parentheses to avoid ambiguity with slot-level descriptors:

```

x = (3 : immutable) // the VALUE 3 is described as immutable
x = 3 : immutable // the SLOT x is described as immutable

```

The main use case for expression-level descriptors is naming subparts of long expressions. Expression-level descriptors support chaining, just like all other descriptor attachment points:

```

x = (3 : immutable : tracked) // two descriptor groups on the value

```

### 11.5 Chaining

Multiple descriptor groups chain with ::

```

x = 3 : immutable : serializable
move(dx, dy) { ... } : cached : public

```

Within a group, | separates alternatives or co-descriptors:

```
peak() { ... } : cached | logged
```

## 11.6 Descriptors Are Expressions

Descriptors are not special syntax — they are ordinary expressions. This means they can be message sends:

```
{
 x = 0 : constrained(0, 100)
 data : serializedAs("json")
} : module
```

The descriptor `constrained(0, 100)` is a regular message send that returns a descriptor object.

## 11.7 Semantics

The grammar defines where descriptors attach syntactically. Their interpretation is entirely up to future runtime extensions, the compiler, or tooling layers — never the core interpreter.

The same literal `{...}` can have its behavior specialized by descriptors, but only when a layer that understands those descriptors is active:

```
{...} : module // singleton namespace (object in raw interpreter)
{...} : closure // captures lexical scope (all objects do this already)
{...} : prototype(parent) // delegation-based (redundant with < parent)
```

This separation of syntax and semantics is the core design principle: the grammar provides uniform literal forms (`class ... { ... }` for classes, `{ ... }` for singletons), descriptors specialize meaning for higher layers, and the raw interpreter runs without them.

## 12 Object Model

This section defines the semantic relationships and the current backend model of objects, classes, mixins, metaclasses and activations. The field diagrams are backend sketches, not a public reflection contract. In particular, activation objects and structural outer links are not publicly reified; source outer `ClassName` uses [qualified lexical receiver selection](#).

Named and anonymous class literals have the same four capability sources: superclass inheritance, mixin composition, lexical nesting, and constructor parameters. A normal top-level class has an empty top activation as its lexical base, no enclosing application scope, only root `Object` inheritance, and no mixins; an instantiation service supplies its external dependencies as existing object arguments of the declared arity. `kernel` is only an optional bundle convention. See [Program Structure](#). Examples with custom superclass/mixin names assume an enclosing definition scope.

### 12.1 Core Structures

The backend stores runtime values in the explicit [Store](#). The following structures describe **native VarioHyve objects**, related by an additive hierarchy, and their reified contributions. They do not define the structure of every visible value: an [Alien](#) has no VarioHyve class, mixins or lexical outer merely because it is represented in the same store.

```
Object = { slots: Map<Name, Addr>, class: Addr, outer: Addr }
Mixin = { definitionScope, methods, slots, members, formals, initializer }
Class = Object + { ownedMixin, applications, super, initializationEdges, layout, initializer }
Metaclass = Class // owns its class's class-side mixin
```

A class **is** an object; a metaclass **is** a class. The `+` notation means “all fields of the parent, plus these new ones.” A metaclass owns only its class’s class-side contribution and has direct superclass `Class`; it does not import factories from instance-side superclass or mixin contributions.

For an instance of `C`, the structural class field reaches a two-object cycle:

```
instance → C → C class → Metaclass → Metaclass class → Metaclass → ...
```

`C class` denotes `C`’s distinct metaclass, not source syntax. These arrows do not introduce a public class selector. `Metaclass` and `Metaclass class` are distinct shared bootstrap identities; an ordinary metaclass is not self-classed.

Each reified instance mixin owns its constructor formals and anonymous slot/post-slot initializer. The realized class owns its initialization edges and cached complete storage layout. A mixin edge refers directly to the selected owned mixin; a superclass edge refers to the complete superclass. Their ordering and routes define the class’s generated private initialization coordinator.

Primary `new` is an ordinary generated method in the owned class-side mixin, specialized for the realized class. Contribution initializers and the class coordinator are private functions evaluated through the same expression and activation machinery. They are bound by identity and never entered through selector lookup. The metaclass has no separate primary arity, initializer or routing metadata. See [the construction decision](#).

**Arrow conventions in this document:**  $\rightarrow$  denotes a structural pointer (class, super, outer fields; initialization order);  $\Rightarrow$  denotes method lookup traversal.

### 12.1.1 Objects

An object holds:

- **Slots** — storage for the logical slots supplied by contribution occurrences. The class's complete layout maps those logical slots to physical storage locations; under the current policy, equal names share a location. Slots hold data values. Slot names are plain identifiers and must not end in `!`. Every slot automatically gets a synthesized getter (`x`, 0-arg) and setter (`x!`, 1-arg). The `!` suffix keeps setter names separate from getter names, leaving the getter name available for [callable invocation](#). See [Slots and Accessors](#).
- **Class** — an address pointing to the object's class. The class holds the object's methods.
- **Definition scope** — internal links used for lexical lookup. Class/object expressions may capture activation objects; member classes bind to their enclosing receiver and declaration origin. Diagrams abbreviate these relationships as `outer`, but no universal `obj.outer` getter exists.

Objects have **no methods**. All method lookup goes through the class.

### 12.1.2 Classes

A class **is** an object with additional fields. It has all the Object fields (`slots`, `class`, `outer`) plus:

- **Owned mixin** — the directly declared contribution, with its method map, slot declarations, member declarations, constructor formals, anonymous initializer and definition scope. A member accessor lazily binds and caches its class per enclosing receiver and declaration occurrence.
- **Super** — a reference to the declared superclass, defaulting to root `Object` when no `<` clause is specified.
- **Mixin applications** — an ordered sequence of references to selected owned mixins, populated from `<:`. Definition scopes and application positions survive composition. The backend lookup list places the owned mixin at position zero.
- **Initialization edges** — the resolved superclass or owned-mixin targets and delayed argument expressions, retaining their application positions.
- **Layout** — the immutable complete instance storage declarations and mapping from contribution occurrence/local slot number to physical location, computed at class realization. Allocation reserves fresh uninitialized locations from this cached layout. Same-named declarations currently share one location, while retaining their distinct logical identities.

- **Initializer** — a private generated function coordinating superclass and applied-mixin initialization, then executing the owned contribution's body in the coordinator's existing initializer activation and return home.

For a normal top-level class, `outer` records the absence of an enclosing application object, `super` denotes root `Object`, and `applications` is empty. Internal definition-scope links may retain the empty top activation. That activation supplies no capability bindings or capability-bearing parent/receiver; the class and its metaclass must not capture the instantiation service or its platform as an outer object. Root inheritance supplies no ambient host capabilities. Constructor arguments are bound when an instance is constructed; they cannot change this class's already established superclass or mixin structure.

The bootstrap defines capability-free `Object`, `Class` and `Metaclass` classes, and one distinct metaclass for each, per runtime. `Object` has no superclass; `Class` extends `Object`; `Metaclass` extends `Class`. Every metaclass has direct superclass `Class` and is an instance of `Metaclass`, including `Metaclass` class. These roots have no lexical parent. `Object` has an empty zero-argument initializer and retains `new/0` and default `apply/0`. `Class`, `Metaclass` and metaclass objects have no public construction factories. Implicit root inheritance is structural: it does not look up the source name `Object`. A platform may supply that class under an ordinary `Object` binding for expression tools; no such binding is added to the empty top activation used by normal module entry. General root protocols such as equality and hashing remain library work beyond this increment.

A class records the instance-side composition: its owned mixin, ordered mixin applications and declared superclass. Its metaclass holds the class-side contribution. The link from a class to its metaclass is the standard object→class relationship: the class *object* has `class = <its metaclass>`, just like any other object. Neither relationship manufactures intermediate classes for composition.

By convention, classes do not have value slots — only methods. Methods and slots are distinct concepts: a method is defined by the `methodDecl` grammar rule and lives in the class; a slot is defined by `slotDecl` and lives in the object. A slot may hold a lambda as its value, but it remains a slot — it can be reassigned to any value later. A method cannot be reassigned (at least without reflection support, but that is out of scope in v1 for now).

### 12.1.3 Store Entries

A **class expression** creates a class/metaclass pair when evaluated. A member declaration creates its pair on first accessor realization, cached for its enclosing receiver and declaration occurrence:

1. The **class** — a `Class` object retaining its reified instance contributions, superclass, initialization edges, layout and private coordinator. Its `class` field points to its metaclass.
2. Its **metaclass** — a class containing its owned class-side contribution, with direct superclass `Class` and `class` field pointing to shared `Metaclass`.

Each **instantiation** creates an `Object` whose `class` points to the class. These counts describe the object/class/metaclass relationships, not total backend allocations: activations, reified mixins and continuations need their own representation. Instances do not acquire a lexical link to their own constructor activation.

A **singleton object literal** (`{ ... }` without the `class` keyword) creates **three** entries: the

instance, its anonymous class, and its anonymous metaclass.

```
class Counter(start) <: Logging < Base {
 count = start // slot – lives in instances
 increment() { // method – lives in the class
 count!(count.plus(1))
 }
}
```

Produces this class/metaclass pair and reified contributions at definition time ( $M(X)$  denotes the owned instance mixin of  $X$ ):

```
CounterOwned = {
 definitionScope = enclosing, formals = [start],
 slots = [count], members = [], methods = {(increment, 0): ...},
 initializer = { initializeSlot(localSlot = 0, start) }
}
Counter = {
 slots = {}, class = CounterMeta, outer = enclosing,
 ownedMixin = CounterOwned, applications = [M(Logging)], super = Base,
 initializationEdges = [Base(), M(Logging())],
 layout = sharedNameLayout(Counter, Base.layout, [CounterOwned, M(Logging)]),
 initializer = coordinate(Base, M(Logging), CounterOwned)
}
CounterMeta = {
 slots = {}, class = Metaclass, outer = enclosing,
 ownedMixin = {(new, 1): ..., (apply, 1): ...},
 super = Class, applications = []
}
```

Primary new is an installed generated method. `initializeSlot`, `coordinate` and `layout` calculation in these sketches describe backend operations, not source sends. Accessors are derived from the selected contribution's slot declarations. The active initializer supplies its occurrence when resolving a local slot through the actual receiver's layout. Selected generated accessors use the same mapping. Private initializer functions have no message selector. See [occurrence-aware slot layouts](#).

Then `Counter(5)` adds one more entry — the instance:

```
instance = { slots = {count: addr1}, class = Counter, outer = enclosingObj }
```

Class chain: `instance`  $\rightarrow$  `Counter`  $\rightarrow$  `CounterMeta`  $\rightarrow$  `Metaclass`  $\rightarrow$  `Metaclass class`  $\rightarrow$  `Metaclass`  $\rightarrow$  ....

#### 12.1.4 Detailed Example: Point

Given:

```

class Point(ax, ay) {
 x = ax
 y = ay
 move(dx, dy) {
 x!(x.plus(dx))
 y!(y.plus(dy))
 }
}
aPoint = Point(2, 3)

```

The class definition creates a class/metaclass pair and its reified contributions; instantiation adds the receiver and its fresh slot storage:

```

PointOwned = {
 definitionScope = enclosing,
 formals = [ax, ay],
 slots = [x, y], members = [],
 methods = {
 (move, 2): fun(dx, dy) { x!(x.plus(dx)); y!(y.plus(dy)) }
 },
 initializer = {
 initializeSlot(x, ax); // internal writes, not setter sends
 initializeSlot(y, ay)
 }
}
Point = {
 slots = {}, class = PointMeta, outer = enclosing,
 ownedMixin = PointOwned, applications = [], super = Object,
 initializationEdges = [Object()], layout = [x, y],
 initializer = coordinate(Object, PointOwned)
}
PointMeta = {
 slots = {}, class = Metaclass, outer = enclosing,
 ownedMixin = {
 (new, 2): primaryFactory(Point), // specialized for Point's layout/plan
 (apply, 2): fun(ax, ay) { self.new(ax, ay) }
 },
 super = Class, applications = []
}
aPoint = {
 slots = { x = 2, y = 3 }, class = Point, outer = enclosing
}

```

Point(2, 3) obtains the class through the Point/0 getter, then invokes apply/2, which sends new/2. Primary construction allocates from Point.layout, initializes the empty root contribution and then PointOwned, and returns that receiver. Getter and mutable-setter lookup uses PointOwned.slots; initialization writes bypass those ordinary accessors.

Class chain: aPoint → Point → PointMeta → Metaclass → Metaclass class → Metaclass → ....

## 12.2 Mixins

A mixin is a **reified contribution**, retaining its definition identity and lexical scope. Each class owns the contribution declared directly in its body: methods, slot declarations, member-class declarations, constructor formals and its own slot/post-slot initializer. Denote this contribution by  $M(\text{Class})$ . A class object and its owned mixin are distinct; the class additionally records applied mixins, a superclass and metaclass behavior. Backend representation classes are not thereby public language classes.

A class body defines its owned mixin's methods normally:

```
class Logging {
 log(msg) { console.print(msg) }
 logError(msg) { log("ERROR: ".concat(msg)) }
}
```

Logging is a class literal with zero constructor parameters. Its owned mixin contains log and logError. Referencing Logging in a `<:` clause selects that owned mixin for application.

### 12.2.1 Mixin Application

When a class declares mixins:

```
class Server(port) <: Logging, Caching < Base { ... }
```

The class retains its own contribution and the applied owned mixins in order, with a separate superclass link. In schematic notation:

$\text{Server} = \{ \text{own}: M(\text{Server}), \text{super}: \text{Base}, \text{applied}: [M(\text{Logging}), M(\text{Caching})], \dots \}$

A mixin application imports only the target class's owned contribution, whether or not the target itself inherits or composes other classes. It does not import the target's superclass, applied mixins, or factories. A superclass application, in contrast, retains the entire existing chain of its target.

For class  $A <: M1, M2 < S1$ , class  $B <: M3, M4 < S2$  and class  $C <: A < B$ , the linear lookup structure is:

$M(C) \rightarrow M(A) \rightarrow M(B) \rightarrow M(M3) \rightarrow M(M4) \rightarrow \text{full } S2 \text{ chain}$

$M1, M2$  and  $S1$  are absent from this structure. They contribute no storage, accessors or initialization through the application of  $A$ . The owned  $A$  initializer receives its own formals from  $C$ 's `<: A(args)` clause and runs its own slots and post-slot body only.  $A$ 's original dependency argument expressions are not evaluated by this application. They still run when constructing  $A$  normally or initializing it as a superclass.

The selected definition keeps its original lexical scope, so structural exclusion does not erase captured references to capabilities or classes. Class realization still resolves  $A$ 's own

clause targets normally before it can be used as a target. `super` in `M(A)` resumes at the next contribution in its new application, here `M(B)`, rather than returning to `A`'s original chain.

No intermediate anonymous classes are created. Applications share the same owned reified definition, while each applying class/list position remains a distinct occurrence. Repeated applications initialize separately and retain their own return homes. Root `Object` supplies an empty owned mixin.

### 12.2.2 Mixin Identity

Reified definitions preserve identity and sharing independently of their application occurrences. `Source <:` selects an owned mixin from a native class. The bootstrap does not yet expose a public mixin extraction or reflection API; future mirrors can expose this structure without changing composition semantics.

## 12.3 Object Construction

When a class with parameters is called — e.g., `Server('localhost', 80, #info)` — the runtime constructs a single object by running a chain of initializers.

### 12.3.1 Construction Example

```
class Base(port) { sPort = port }
class Logging(level) {
 sLoggingLevel = level
 log(msg, lvl) { self.sLoggingLevel.gte(lvl).ifTrue(fun {
 ↪ console.print(msg) }) }
}
class Server(address, port, level) <: Logging(level) < Base(port) {
 sAddress = address
}
```

`Server('localhost', 80, #info)` produces:

```
obj = { slots: {sPort: 80, sLoggingLevel: #info, sAddress: 'localhost'},
 class: Server, outer: enclosing }
```

All slots live flat on the single instance. Methods are dispatched through the class chain.

### 12.3.2 Initializer Scope and Persistent State

Constructor parameters are immutable bindings local to initialization. An initializer can store selected values in slots, invoke behavior, or deliberately create lambdas and class expressions that capture its lexical scope. Ordinary instance methods cannot resolve their

own constructor parameters, including when called during initialization. A member-class declaration uses the enclosing instance and its origin, not its constructor activation. Named and anonymous class expressions share ordinary lexical capture behavior.

Each invoked superclass/mixin initializer has its own argument bindings. Do not install all arguments in a shared persistent scope or rebind a contributed method's definition dependencies. Captured expressions may retain internal activations, but no activation accessor/mirror is implied. See [the accepted decisions and modification sequence](#).

### 12.3.3 Construction Order

Construction creates **one** object and follows the class's instance-side composition in this order:

1. Allocate a new object with class C and fresh uninitialized storage for every location in C.layout. The cached layout accounts for the complete composition and current shared-name slot policy.
2. Run **super initialization**: Base's complete chain initializes with `self = obj` and the arguments from the `< Base(port)` expression, following its own routed dependency schedule.
3. Run **owned mixin initializers, right to left** (rightmost in the `<:` list first): each selected owned initializer runs with `self = obj` and its routed arguments. Its source class's superclass/mixin routes do not run.
4. Run **own initializer**: the class's owned contribution runs with `self = obj`.

The argument expressions in `< Base(port)` and `<: Logging(level)` are evaluated in the declaring contribution's initializer activation, where its constructor parameters are bound. For example, in `Server('localhost', 80, #info)`:

```
class Server(address, port, level) <: Logging(level) < Base(port) { ... }
// ^^^^^ ^^^^^
// `level` and `port` refer to the class parameters
// passed to `new` - i.e. level=#info, port=80
```

This means the super and mixin constructor arguments can be arbitrary expressions involving the class parameters:

```
class Grid(rows, cols) < Matrix(rows.times(cols)) {
 // Matrix receives rows*cols as its argument
}
```

In pseudocode for `Server('localhost', 80, #info)`:

```
Server.new('localhost', 80, #info):
 obj = allocate(Server, Server.layout)
 Server.initialize(obj, 'localhost', 80, #info)
 obj // return the fresh receiver
```

```

Server.initialize(obj, address, port, level):
 Base.initialize(obj, port) // complete superclass schedule
 M(Logging).initialize(obj, level) // only the selected owned mixin
 execute M(Server).initializer.body // same activation and home

```

The `.initialize` notation denotes private initialization entry points, not ordinary message selectors. Generated function bodies sequence the private calls using ordinary core expressions and explicit machine continuations. Executing the owned contribution body directly preserves the logical home shared with its route expressions; a superclass or applied-mixin invocation establishes a fresh home. Rightmost mixins initialize first; leftmost mixins initialize last (among mixins). The class's own initializer always runs last.

### 12.3.4 Initializer Return Homes and Completion

Each invocation of each initializer contribution has a fresh private return home. A contribution executes its ordered slots and then its optional `final |` statement section. Each contribution finishes both phases before the next contribution runs. This applies to member classes, named/anonymous class expressions and object literals, including superclass and direct mixin execution. Captured definition-time bindings do not supply an old return home, and the instantiating caller does not lend its home to construction.

A `^` targeting the initializer during its slot phase fails construction. This includes a return from a lambda in the final slot expression, before assignment completes. Methods invoked by initialization have their own homes and can return normally. After all of this contribution's slot declarations have completed, `^` value may complete the contribution early. Its expression is evaluated normally, its resulting value is ignored, and the next contribution executes. The ordinary value of the post-slot body is likewise ignored. Primary `new` always returns the allocated receiver on successful completion; custom factory results follow their separate factory contract.

Initializer homes retire on normal/early completion, failure and unwind. Captured data may survive; returning to a retired home is an error. An outside lambda keeps its original home and can unwind construction to that home if live. The active Alien-call barrier still applies. Failure does not roll back effects or retract an escaped `self`. See [Literals — Initializer completion](#) for source syntax and [the accepted policy](#) for implementation scope and the deliberate differences from Newspeak.

### 12.3.5 Ordered Overrides and Their Limits

The initial policy is ordered overriding: own contributions take precedence, then composed contributions in declaration order (leftmost first), then the superclass. Duplicate names within a single declaration body remain errors; overriding a contribution from composition/inheritance is a separate operation. For member classes, overriding selects the complete declaration and its origin, not an automatic deep merge or mutation of an existing class value.

The existing initialization schedule is superclass, mixins right-to-left, then own initializer. Under the initial shared-name slot policy, later initializer writes overwrite earlier values. This is deterministic, but does **not** make stateful mixins independent. If two surviving methods use the same slot name for unrelated state, both still observe one shared location. Matching method precedence does not remove the collision or its order dependence.

Keep declaration and application origin information in the semantic model. Deep family composition with state in members is a planned later increment; explicit state mapping, renaming, hiding, aliasing and more general ordering need separate rules before they can be implemented safely. A physically flat slot array does not require shared logical state identities. Effectful initialization also requires an explicit schedule; structural composition does not make foreign effects commute or roll back failed construction.

The bootstrap implements superclass chains and owned-mixin application using these ordered rules. An inherited or composed class supplies only its owned contribution when used as a mixin. This does not implement the future composition algebra; see [the construction contract](#).

### 12.3.6 What `<: Mixin(args)` and `< Base(args)` Mean

The `<:` and `<` expressions serve dual roles:

- **At class creation time:** they establish the mixin and super chain (method lookup structure).
- **At construction time:** their arguments are passed to the mixin/super initializer, which runs on the new object.

`Base(port)` in `< Base(port)` does not create a separate `Base` instance — it invokes `Base`'s initialization logic on the object being constructed.

**Important — special constructor semantics:** Although `superclassExpr` and `mixinExpr` are syntactically ordinary expressions in the grammar, they have **special semantics** at construction time. When `< Base(port)` or `<: Logging(level)` appears in a class definition, the runtime does **not** invoke their terminal application as an ordinary factory call that creates a separate instance. Instead:

1. At class realization, resolve the **target definition** (e.g., `Base`, `Logging`) once, evaluating mixin targets left-to-right and then the superclass target. The terminal parenthesized application is split into a pure zero-argument target `send` and delayed argument expressions. It does not call a factory.
2. On each construction, evaluate each **argument list** left-to-right in the declaring initializer's activation immediately before its contribution runs: superclass first, direct mixins right-to-left. Constructor bindings are local to that declaring initializer; supplied initializers receive their own formals.
3. Initialize on the same allocated receiver. A superclass target runs its complete dependency schedule, slots and post-slot section. A mixin target runs only its owned slots and post-slot section, without following its source class's dependency routes. Then resume the pending contribution. Routing occurs before slot completion and cannot return early to the declaring initializer's home.

The core AST represents this distinction as `CoreConstructorApplication(target, arguments)`. Receiver subexpressions within the target still evaluate normally. Targets are fixed for the resulting class; routed argument expressions execute again for each construction. Bare targets route no arguments. Native `super` cannot be a clause target. In the bootstrap, both superclass and direct mixin targets must be native classes; Aliens require explicit adaptation.

Computed target receivers are explicitly allowed, including for member classes. For example, in an enclosing declaration:

```

class Host(provider) {
 families = provider;
 class Member(x)
 <: families.choose("mixins").Mixin(x)
 < families.choose("base").Base(x) { }
}

```

Realizing `Member` evaluates `families.choose("mixins")`, reads its `Mixin` class, then evaluates `families.choose("base")` and reads its `Base` class. These ordinary receiver computations may invoke methods or foreign operations; their results determine the contributors. Only the **terminal** `.Mixin(x)` and `.Base(x)` applications route `x` to initialization. The calls to `choose` are ordinary invocation, not constructor routes. Parenthesizing a terminal application does not turn it into an ordinary factory call.

A member accessor caches the realized class or realization failure for its bound enclosing receiver and declaration occurrence. Bare member access realizes it; invocation first checks the declaration's factory signatures and realizes only a selected matching candidate. Rejected arities execute none of these computations. Once realized, the contributor choices remain fixed for that class, while routed arguments are reevaluated for each new instance. A selected realization failure is an error, never a reason to try a farther declaration. See [member-class invocation](#).

The syntactic order `<: Mixins < Super` matches the lookup priority: the class itself is checked first, then mixins left to right, then the superclass. Reading left to right in the declaration mirrors left to right in the lookup chain.

### 12.3.7 Synthesized Getters and Setters

Each class that declares a slot also holds the synthesized getter and setter for that slot in its methods:

- **Getter:** `(name, 0)` — reads `self.slots[name]`
- **Setter:** `(name!, 1)` — writes `self.slots[name]`, returns the new value

The getter and setter operate on `self.slots[name]`, so they work correctly regardless of which object `self` is — the slot lives on the instance.

When `obj.sLoggingLevel` is called, lookup walks the class chain and finds the getter in `Logging`'s class. The getter reads `self.slots["sLoggingLevel"]` — which is on `obj`. To mutate: `obj.sLoggingLevel!(newLevel)`. This is why methods and getters/setters work uniformly through dispatch.

### 12.3.8 The apply Protocol

A value is callable at arity `N` when it provides the exact `apply/N` capability:

- **Class objects** have owned class-side `apply/N` methods, delegating to `new/N` by default or implementing a declared custom factory.
- **Lambdas** provide invocation at their declared parameter count.
- **Native user objects** can define or inherit an `apply/N` method.

- **Foreign values** may advertise apply/N through effect-free boundary selection, including ForeignCallable.

Invocation name(args) tests candidates at each scope or hierarchy contribution: prefer an exact operation, then an eligible generated accessor supplying a callable of the required arity. Stored values are read without executing user getters. Member-class accessors advertise their selected declaration's constructor arity and owned factory apply arities without realizing the class. Continue searching on a genuine miss, never after a selected failure.

The rule includes arity zero. A method f { 7 } makes f() return 7 once; a slot containing fun { 7 } supplies the lambda's apply/0. Bare f remains an ordinary f/0 send. Generated getters are not themselves invocation matches at arity zero, and computed getters are not executed to discover callables at other arities. Overridden getters mask backing storage throughout lookup.

```
{
 Point = class(x, y) { };
 handler = fun(req) { req };
 test(request) {
 Point(2, 3); // stored class's apply/2 constructs an
 ↪ instance
 handler(request) // stored lambda's apply/1
 }
}
```

The separate setter name handler! leaves handler available for callable invocation. See [Message Sends](#) for the full selection order, failures, zero-arity behavior and Alien dispatch distinctions.

## 12.4 Method Lookup

For an explicit native invocation m(args), the runtime searches for applicable behavior at selector (m, arity) in this contribution order:

1. **The object's class** — its own methods.
2. **Applied owned mixins, left to right** — applications[0], then applications[1], etc. Lookup checks each selected owned contribution, without traversing its source class's original dependencies.
3. **The superclass** — recursively, which walks the super's own methods, then the super's mixins, then the super's super, etc.

At each contribution, an exact operation precedes an eligible callable binding. A nearer callable can therefore win over a farther exact method. The first applicable candidate wins; exhaustion produces a missing-selector diagnostic. Ordinary bare sends perform exact zero-arity lookup.

```
class Server(port) <: Logging, Caching < Base { ... }
```

Lookup order for a message sent to a Server instance:

Server's class methods

- ⇒ Logging's class methods
- ⇒ Caching's class methods
- ⇒ Base's class methods
- ⇒ Base's mixins (if any)
- ⇒ Base's super (recursively)

### 12.4.1 Super Dispatch

`super.m(args)` starts lookup at the **next position in the chain** relative to where the currently executing method was found. The receiver remains `self`.

- If the current method is in Server's class → super starts at Logging.
- If the current method is in Logging → super starts at Caching.
- If the current method is in Caching → super starts at Base's class.
- If the current method is in Base's class → super starts at Base's first mixin (if any), or Base's super.

To implement this, method activations track the **dispatch origin**: which class or mixin provided the currently executing method, and its position in the mixin list. This is the only additional bookkeeping beyond standard dispatch.

### 12.4.2 Super in Mixin Methods

A mixin method that calls `super` does not know at definition time what `super` will resolve to — it depends on the application site:

```
class Logging {
 process(data) {
 log("processing")
 super.process(data) // goes to whatever is next in the chain
 }
}

// Here, super.process() in Logging goes to Caching
class App <: Logging, Caching < Base { ... }

// Here, super.process() in Logging goes to Base directly
class Simple <: Logging < Base { ... }
```

This dynamic dispatch of `super` is essential for mixin compositionality. The mixin's implicit contract is: "I assume `super` responds to `process(data)`." If the application site does not satisfy the contract, a runtime error occurs. See [Future Directions](#) for planned static verification.

## 12.5 Metaclass Details

A metaclass is a class (which is an object), containing its class's owned class-side contribution. This section covers primary construction (`new`), the callable protocol (`apply`), named factory methods and the flat metaclass hierarchy. See [Core Structures](#) for the representation.

### 12.5.1 The Primary Factory (`new`)

Every source class and root `Object` gets an automatically synthesized, **non-overridable** new method on its metaclass. This is the **primary factory** — the standard construction path that always produces a fresh instance:

```
class Point(ax, ay) {
 x = ax
 y = ay
}
Point.new(2, 3) // always allocates + initializes → fresh Point
```

`new` follows the full [construction order](#): `allocate → run < Base(args) contribution → run <: Mixin(args) contributions (R→L) → run own contribution → return the new object`. Each contribution has a fresh return home, ordered slot initialization and optional post-slot code; permitted early completion ends only that contribution. Its generated method body uses private allocation and identity-bound initializer invocation operations; message lookup and method invocation have no special primary-construction target. Object literals use this same generated construction path after realizing their anonymous class.

`new` is never written explicitly and cannot be overridden. Its arity always matches the class parameter list. It is the one reliable way to create a guaranteed fresh instance.

Synthesized `new` and default `apply` belong to each class's own constructor signature. In particular, inheriting from root `Object` does not add its zero-argument construction signature to a class with parameters. User-defined factories also belong only to their declaring class: neither superclass inheritance nor mixin application imports them. Class, Metaclass and metaclass objects have no public `new` or default `apply` construction entry points.

Like `self`, `new` is **semantically reserved** — it is a regular identifier in the grammar (not a keyword), but the semantic layer rejects any factory `new(...)` declaration. This is the precise check: only factory `new` conflicts with the synthesized `new` (which lives on the metaclass). A regular instance method named `new` on a non-class object (e.g., a builder or pool) does not conflict and is permitted. See [Reserved Words](#) for the full list.

### 12.5.2 The Callable Protocol (`apply`)

By default, the metaclass synthesizes an `apply` method that simply delegates to `new`:

```
synthesized apply(ax, ay) { new(ax, ay) }
```

This supplies the callable capability for `Point(2, 3)`: select its class `apply/2`, which delegates to `new/2` and returns a fresh instance. A generated member-class accessor can advertise this arity before the class is realized.

A class can define **one or more** `factory apply` methods to **fully replace** the synthesized `apply` at a given arity. User-defined applies have **full control** — no automatic slot initialization runs through `apply`, no forced return value. The user may call `new` inside the body (to get standard construction) or do anything else:

```
class Point(ax, ay) {
 x = ax
 y = ay

 // Same-arity apply – wraps standard construction with validation:
 factory apply(ax, ay) {
 validate(ax, ay)
 var obj = new(ax, ay) // calls the standard construction chain
 obj.register
 obj // full return-value control
 }

 // Different-arity apply – zero-arg constructor:
 factory apply() {
 new(0, 0) // delegates to new with defaults
 }
}
```

With this definition:

```
Point(2, 3) // → (apply, 2)(2, 3) → validated, registered instance
Point.apply // → (apply, 0) → Point at origin
Point.new(2, 3) // → always raw construction, no validation, no
↪ registration
```

**Key principle:** `apply` is the callable protocol — **homogeneous across all callable objects**:

Callable	Default apply	Overridable?
Lambda	Invoke body	No (intrinsic)
Class	<code>new(args)</code>	Yes, via factory <code>apply(...)</code>
User object	Whatever user wrote	Yes (it's a regular method)

### 12.5.3 Multiple Constructors via Multiple `apply` Arities

A class can define `factory apply` at multiple arities, providing multiple construction signatures:

```

class Color(r, g, b) {
 red = r
 green = g
 blue = b

 factory apply(r, g, b) {
 validate(r, g, b)
 new(r, g, b)
 }

 factory apply(hex) {
 new(hex.extractRed, hex.extractGreen, hex.extractBlue)
 }

 factory apply() {
 new(0, 0, 0) // black
 }
}

```

```

Color(255, 0, 0) // 3-arg apply → validated red
Color(0xFF0000) // 1-arg apply → from hex
Color.apply // 0-arg apply → black
Color.new(255, 0, 0) // raw – always the 3-arg primary factory

```

The synthesized `apply` (the one that delegates to `new`) is only generated if the user does not define a `factory apply` at that arity. User-defined applies at other arities coexist without affecting the synthesized default.

#### 12.5.4 Caching, Pooling, and Abstract Factories

Because user-defined `apply` has full return-value control, patterns that return non-fresh objects work naturally:

```

class CachedPoint(ax, ay) {
 x = ax
 y = ay

 factory apply(ax, ay) {
 cache.getOrPut(ax, ay, fun { new(ax, ay) })
 }
}

```

```

CachedPoint(2, 3) // may return cached instance
CachedPoint.new(2, 3) // always fresh – bypasses cache

```

The guarantee: `ClassName.new(args)` = always fresh. `ClassName(args)` = goes through `apply`, which the class author controls. The primary factory provides fresh construction while the callable protocol remains customizable.

### 12.5.5 Named Factory Methods (factory)

The `factory` keyword also declares **named factory methods** on the metaclass — alternate construction entry points:

```

class Counter(start) < Base {
 count = start

 factory create() { new(0) }
 factory fromList(items) { new(items.size) }
}

```

Named factory methods are invoked on the class object:

```

Counter(5) // class apply/1 delegates to new/1
Counter.create // named factory → new(0)
Counter.fromList(myList) // named factory → new(myList.size)
Counter.new(5) // direct construction – always

```

Named factory methods are ordinary methods that live on the metaclass. They have full access to `new` and to other factory methods. Like user-defined `apply`, they control their own return value.

Produces:

```

Counter = {
 slots = {}, class = CounterMeta, outer = enclosing,
 ownedMixin = {formals: [start], slots: [count], initializer: {count = start}, ...},
 super = Object, applications = [], initializationEdges = [Object()],
 layout = [count], initializer = coordinate(Object, ownedMixin)
}
CounterMeta = {
 slots = {}, class = Metaclass, outer = enclosing,
 ownedMixin = {(new, 1): ..., (apply, 1): ..., (create, 0): ..., (fromList, 1): ...},
 super = Class, applications = []
}

```

### 12.5.6 Why `factory` Is a Separate Keyword

Without `factory`, a method `create()` defined inside the class body becomes an instance method — it lives on the class and is dispatched to instances. With `factory`, the same method lives on the metaclass and is dispatched to the class itself.

This is the only semantic effect of `factory`: **it changes which object holds the method** (metaclass instead of class). Everything else — method bodies, dispatch, `super` — works identically.

Declaration	Lives on	Dispatched via	Invoked as
<code>increment() { ... }</code>	Class	<code>instance.increment</code>	<code>counter.increment</code>
<code>factory create() { ... }</code>	Metaclass	<code>class.create</code>	<code>Counter.create</code>
<code>factory apply(start) { ... }</code> ( <i>synthesized</i> )	Metaclass	Class invocation at arity one	<code>Counter(5)</code>
<code>new(start)</code>	Metaclass	<code>class.new(5)</code>	<code>Counter.new(5)</code>

### 12.5.7 Factory Method Lexical Environment

Factory methods (including `factory apply`) live on the metaclass and are dispatched when a message is sent to the **class object**. Within a factory method:

- **`self`** resolves to the **class object** — the metaclass’s instance. This is the object the factory was dispatched on.
- **Qualified lexical receivers** use the factory’s declaration origin and its class definition scope. `outer X` selects the receiver of the lexically enclosing class `X`; no public metaclass parent accessor is involved.

This means factory methods have lexical access to the environment where the class was defined. For inner classes, this environment varies per instance of the enclosing class:

```
class X(n) {
 m = n

 class Y() {
 factory toto() {
 var z = outer X.m // X instance that contains this Y
 new
 }
 }
}

o1 = X(3)
o2 = X(4)
o1.Y.toto // outer X.m → 3 (o1's slot m)
o2.Y.toto // outer X.m → 4 (o2's slot m)
```

Each *X* instance gets its own cached *Y* member class bound to that receiver. When *toto* executes, *outer X* uses the member declaration's origin to select that enclosing *X* instance. Since *o1* and *o2* have different *m* values, the factory method sees different values. Custom factories and this lexical behavior are implemented in the current bootstrap.

An outer link captures an enclosing scope only when one exists at definition time. A normal top-level class and its metaclass retain at most the empty top activation, with no enclosing application scope; invoking a factory does not capture the caller or instantiation service. Nested factory methods follow their class's definition scope through the standard outer chain. Classes evaluated in an explicitly supplied tooling context retain that context under the same rule; see [Evaluation Contexts](#).

### 12.5.8 Bootstrap Metaclasses

VarioHyve uses a Newspeak-like flat metaclass hierarchy while retaining explicit reified mixin composition inside each class. Every ordinary class *C* has a distinct *C class* containing only *C*'s owned class-side contribution, specialized for that realized class. Its superclass is shared *Class*, independently of *C*'s instance-side superclass and applied mixins. The common instance protocol of *Class* is initially empty; no contributor factories enter through it.

The seeded relationships are:

Object	Structural class	Direct superclass
Object	Object class	none
Class	Class class	Object
Metaclass	Metaclass class	Class
Object class	Metaclass	Class
Class class	Metaclass	Class
Metaclass class	Metaclass	Class
Ordinary <i>C</i>	<i>C class</i>	its declared superclass, or Object
Ordinary <i>C class</i>	Metaclass	Class

If *Counter* < *Base*, instance lookup reaches *Base* normally. A factory declared by *Base* is available on *Base*, but is not thereby available on *Counter*. Factory super continues into *Class*, not *Base class*. An explicit lexical reference to *Base* may still invoke its factories normally.

Object is the only null-superclass root. The seeded classes and their metaclasses have no capability-bearing lexical parent. Seeding these identities creates no ambient source bindings, reflection selectors or public metaclass constructors. The existing explicit kernel Object export remains optional for expression tools. See [Flat metaclasses and class-owned construction](#) for migration and construction ownership.

## 12.6 Method Activations

When a method is called, the runtime creates an **activation object** in the store. It uses the internal object representation for scope, without exposing activation reflection or general

public activation access:

- **Slots** — one per var local declaration, with getter/setter access. Local names must not end in !.
- **Parameters** — read-only entries; no setter synthesized, and parameter names must not end in !.
- **Class** — points to a class holding the method body (for potential inner method calls).
- **Outer** — points to the receiver object (for lexical lookup from within the method body).

### 12.6.1 Lambda Activations

A lambda activation differs in one critical way: its **outer** points to the enclosing activation (the activation that created the lambda), not to an object. This is what makes closures work — a lambda captures the lexical environment of its definition site.

```
class Counter(start) {
 count = start
 makeIncrementer() {
 ↪ Counter instance
 var step = 1
 fun() {
 ↪ makeIncrementer's activation
 count!(count.plus(step))
 ↪ makeIncrementer's activation
 count
 ↪ → Counter instance
 }
 }
}
```

*// method activation: outer →*

*// lambda activation: outer →*

*// `step` resolved via outer →*

*// `count` resolved via outer chain*

In this example: - The method activation for `makeIncrementer` has `outer → Counter instance` (the receiver). - The lambda activation has `outer → makeIncrementer's activation`, which gives it access to `step`. - `count` is not found in the lambda's locals or in `makeIncrementer's` locals, so lookup continues through `makeIncrementer's` `outer` to the `Counter` instance.

This activation-as-object model is what makes closures work naturally: a lambda captures a reference to the enclosing activation's address, and the activation survives in the store as long as any lambda holds a reference to it. See [Recursion — Passing Recursive Lambdas as Arguments](#).

## 12.7 Dispatch Origin Tracking

Each activation records the **dispatch origin**: the applying class and exact contribution position that supplied the method. Definition scope remains on the selected reified mixin. This distinguishes two applications of the same definition and tells `super` where to resume lookup.

```
Activation = { slots, class, outer, dispatchOrigin: (applyingClass, contributionIndex) }
```

The backend contribution list starts with the owned mixin at index zero, followed by explicit applications at indices one through N. `super` resumes at the next contribution, then continues into the superclass after the last application. A direct initializer uses the same applying-class/index origin as its methods; it does not recover an origin through the mixin's source class.

## 12.8 Design Summary

Concern	Mechanism
Object structure	{ slots, class, outer } — base of the additive hierarchy
Class structure	Object + owned mixin, ordered applications, superclass, initialization edges, cached layout and private coordinator
Metaclass structure	Class with only its owned class-side contribution; superclass <code>Class</code> , class <code>Metaclass</code>
Object state	Slots flat on the instance, methods in the class chain
Construction	Generated ordinary new method: allocate → private class coordinator → return receiver; coordinator runs <code>super</code> → mixins ( $R \rightarrow L$ ) → own body
<code>apply</code> protocol	Callable protocol; default delegates to <code>new</code> ; overridable via <code>factory apply(...)</code>
<code>&lt; Super(args)</code>	Initialize the complete superclass chain on <code>self</code> with these args
<code>&lt;: Mixin(args)</code>	Initialize only the target class's owned contribution on <code>self</code> with these args
Method lookup	$\text{Class} \Rightarrow \text{mixins} (L \Rightarrow R) \Rightarrow \text{super}$ (recursive) — leftmost wins
Slot overlap (initial policy)	Shared names are overwritten in initialization order; deterministic but not independent state
Setter naming	<code>x!</code> (distinct from getter <code>x</code> ) — no arity clash
Callable invocation	At each scope/contribution prefer an exact operation, then an eligible generated binding with exact <code>apply/N</code> ; member classes advertise owned factory arities
Mixin representation	Owned methods, slots, members, formals and anonymous initializer; preserved definition scope and separate application positions

Concern	Mechanism
Mixin sharing	All classes referencing a mixin share the same mixin object
super dispatch	Next in chain, tracked via dispatch origin
Factory methods	Only the declaring class's class-side mixin; all metaclasses directly extend <code>Class</code>
Activations	Internal scope objects; expressions can capture them, ordinary instances do not retain their own constructor frame
Conflict resolution (v1)	First match wins (leftmost mixin) for both methods and slots
Composition (future)	Deep member families and explicit adaptation/state rules — see <a href="#">Future Directions</a>

## 13 Interpreter Design

The v1 interpreter is an **AAM-style CESK machine** — a small-step abstract machine with four components:

- **Control** — the current AST node being evaluated
- **Environment** — a mapping from names to store addresses
- **Store** — a mapping from addresses to values
- **Kontinuation** — a stack of frames representing what to do next

The interpreter runs by repeatedly applying a transition function  $\text{step} : \text{State} \rightarrow \text{State}$  until a final state is reached.

**Current implementation note:** The visitor machine represents lexical scope with activation objects, not a separate Environment class. Method and initializer invocations establish private return homes. `CallReturnFrame` implements method completion; `InitializerReturnFrame` completes one initializer contribution, discarding its value and preserving the construction receiver. `ReturnFrame` below is a design-level name from an earlier interpreter sketch, not a current Java class. Configurable allocation alone does not provide the abstract interpretation described as a goal in this chapter.

### 13.1 Module Instantiation Requirements

Named and anonymous class literals receive capabilities through inheritance, mixin composition, lexical nesting and constructor parameters. Normal module definition starts in an empty top activation: no capability bindings, no capability-bearing lexical parent or receiver, and no enclosing application scope. The activation frame still exists. Such a class has only root `Object` inheritance and no mixins. The instantiation service supplies existing object arguments at the declared constructor arity. A kernel is an optional bundle, not a mandatory single input.

The machine must bind constructor parameters using activation objects and must not expose those bindings as instance-method scope or retain an automatic edge from an instance to its own initialization activation. Ordinary lambdas and class expressions can deliberately capture that activation. Member-class declarations instead bind to the enclosing instance and retain their declaration/application origin. Activations remain backend-private, with no public reification. The machine must not capture the service's or caller's scope. Class creation and subsequent instance construction are distinct: constructor arguments cannot supply that top-level class's own superclass or mixins. Nested class definitions may use class/mixin objects available through their enclosing module. See [Program Structure](#).

Tools may explicitly supply a richer evaluation activation or select a debug activation. These contexts use the same object-based scope and invocation rules, and values created there retain their actual lexical captures. The current kernel-injected expression harness is an example of supplied evaluation context: its lack of local slots does not make its parent/receiver capability graph empty. The separate `BootstrapRun-`

`time.instantiateModule` path evaluates a class literal over an empty top activation and constructs it through an exact Expat new send. Definition, initialization and synchronous callbacks share one transition budget; host sends contribute an empty caller scope. The instance is returned as an Expat, and its entry method is invoked explicitly by the host. See [Evaluation Contexts](#).

## 13.2 Why CESK

VarioHyve’s uniformity makes CESK a natural fit:

VarioHyve property	CESK benefit
Everything is a message send	One dispatch rule to implement, not dozens of special forms
No operators or control structures	<code>ifTrue/ifFalse</code> are message sends — no special-case evaluation
No globals	No global store partition — only lexical environments + object slots
Few AST node types	Small step function
Uniform object model	Objects, classes, and activations share the same store representation

## 13.3 Machine Components

### 13.3.1 Addresses and Allocation

Every binding goes through the store via an explicit address. Values are never stored directly in the environment.

```
Env = Map<Name, Addr>
Store = Map<Addr, Value>
Alloc : Context → Addr
```

The `Alloc` function is the single parameterization point:

- **Concrete allocation** — returns a fresh integer on every call. This is the v1 interpreter.
- **Abstract allocation** — returns a bounded address derived from the call site and a prefix of the environment (k-CFA). This enables finite-state analysis.

The interpreter core is written once; swapping `Alloc` switches between concrete execution and abstract analysis.

### 13.3.2 Values

All runtime values live in the store. The full runtime representation — objects, classes, mixins, metaclasses, method activations, and dispatch origin tracking — is defined in the [Object Model](#).

The key points for the CESK machine:

- **Objects** hold slots and point to their class and internal definition scope. Public outer `ClassName` is declaration-relative, not a lexical-parent getter.
- **Classes** hold methods and point to their super, mixin list, and metaclass.
- **Mixins** are regular objects — no special runtime type.
- **Method activations** are objects in the store — their var locals are slots, their parameters are read-only entries. This uniformity is what makes activation-as-object semantics (and closure capture) fall out naturally.
- **Dispatch origin** is tracked per activation so super knows where to resume in the class/mixin chain.

### 13.3.3 Continuations

The continuation is an explicit linked list of frames. Each frame records what work remains after the current sub-evaluation completes:

- **MessageFrame** — receiver evaluated, waiting for arguments; then dispatch
- **ArgFrame** — evaluating argument list left to right
- **SeqFrame** — statement list; evaluate next statement
- **ReturnFrame** — conceptual method boundary; the current visitor uses `CallReturnFrame` and a private activation that tracks home liveness
- **InitializerReturnFrame** — ends one initializer contribution and resumes its continuation with the allocated receiver, ignoring the body's result
- **Private initializer-call frame** — evaluates the receiver and arguments of an identity-bound invocation before entering a fresh initializer activation
- **Private slot-write frame** — completes an initializer expression, writes its value through the receiver's complete storage layout, then resumes ordinary expression sequencing

$\wedge$  targets its lexically captured method or initializer home. Each initializer invocation has a fresh home even when its class captures an earlier activation. When  $\wedge$  is evaluated, the continuation chain is scanned for that exact home; it never chooses the nearest dynamic method or borrows the caller's home.

1. **Method or initializer entry** establishes a fresh live home and the corresponding completion frame. An initializer begins in the slot phase.
2. **Normal completion** retires the home. A method delivers its body result; an initializer ignores its result and resumes the contribution continuation with the receiver.
3.  **$\wedge$  evaluated** checks the captured home's liveness and matching frame. Missing/retired homes produce a dead-context error. A live initializer target still in its slot phase produces a premature-completion error, including a return before the final slot value is assigned. A post-slot initializer target completes only its contribution and ignores the returned value. A method target returns that value normally.
4. **Failure or unwinding** retires intervening homes. An outside lambda retains its own target and can unwind construction to it if live. An active Alien-call barrier forbids unwinding across that boundary.

The return expression is evaluated normally before delivery; failure there is not a successful early exit. Home liveness is separate from reachable captured data. See [Initializer completion](#) and [the accepted policy](#). Explicit continuations implement these transitions without using host exception unwinding as the language's return semantics.

### 13.3.4 State Transitions

Class realization finalizes its immutable complete instance layout and retains resolved initialization edges. Formals and the anonymous slot/post-slot initializer belong to each reified instance mixin. Direct mixin edges refer to the selected owned mixin, while superclass edges refer to the complete class. Allocations reuse the class's physical layout and preserve shared-name storage. Each generated accessor resolves its selected contribution occurrence and local slot number through the actual receiver class's immutable logical-to-physical mapping.

Class realization generates ordinary `CoreUserFunction` bodies for primary `new`, each owned contribution initializer and the class's coordinator. `ClassTargetFrame` resolves native class targets before publication; `MemberClassReturnFrame` completes the declaration/application cache.

Primary `new` is installed on the owned class-side mixin and follows ordinary lookup and method invocation. Its body allocates the bound class's receiver, invokes the private coordinator, and returns that receiver. The coordinator sequences superclass and applied-owned-mixin invocations with ordinary core expressions, then executes its own initializer body in the same activation/home. An applied mixin never follows its source class's original dependency routes. Object literals run this same generated construction path.

Backend-private `ConstructionCode` operations provide bound-class allocation, identity-bound initializer invocation, direct slot writes and an explicit slots-complete marker. A slot write resolves its contribution-local number using the active initializer's occurrence and actual receiver layout before evaluating the value expression, then retains the destination in its continuation. The marker separates mandatory slot initialization from the optional final body; a return before the final slot write remains invalid. Private call arguments and writes use explicit continuations, while ordinary expression sequencing replaces the former contribution/slot schedule frames. No source or Expat selector selects a private initializer.

Empty argument lists still pass through the machine scheduler, so deep initialization does not recurse on the host stack. Failure or unwind retires initializer homes and caches in-progress member realization as a defined failure. See [the construction operation contracts](#).

The step function pattern-matches on `Control × Kontinuation`:

Control	Action
Literal (number, string)	Allocate value in store, pass to continuation
Hole Expression (?)	Delegate to the active hole-resolution policy. The policy may suspend execution, raise <code>UnresolvedHole</code> , produce an abstract unknown value, branch nondeterministically, or invoke a synthesis / interactive completion mechanism
Class expression (named or anonymous)	Capture definition scope in owned contributions; create class with resolved edges/layout and metaclass with owned factories

Control	Action
Member-class accessor	Bind declaration to enclosing receiver and origin; cache class/metaclass per receiver and occurrence
outer <code>ClassName</code>	Select receiver by lexical declaration origin; never expose an activation
Object literal	Realize anonymous class + metaclass; use their generated construction path and deliver the receiver
Lambda literal	Capture current environment, allocate closure in store
Implicit message send — bare <code>(name)</code>	Look up name via lexical chain; dispatch <code>(name, 0) → return result (invoke only)</code>
Implicit message send — with parens <code>(name(args))</code>	Evaluate args once; at each lexical scope prefer exact name/N, then an eligible generated callable binding, before self invocation. Arity zero executes selected behavior once. See <a href="#">invocation rules</a> .
Explicit message send — bare <code>(e.name)</code>	Evaluate receiver <code>e</code> , dispatch <code>(name, 0)</code> on receiver <code>→ return result</code>
Explicit message send — with parens <code>(e.name(args))</code>	Evaluate receiver and args once; native lookup tests exact operations and eligible generated callable bindings per contribution. Alien lookup selects exact foreign name/arity. Arity zero executes selected behavior once.
Cascade <code>(e..m1..m2)</code>	Evaluate receiver once, dispatch each message to that receiver, pass last result to continuation
<code>super</code>	Resume lookup from dispatch origin's next position — see <a href="#">Object Model — Super Dispatch</a>
var declaration	Allocate address in store, bind in current environment
<code>^ return</code>	Find the captured lexical home; enforce initializer phase and foreign-call barriers, retire intervening homes and resume its completion frame

### 13.4 Explicit Store: The Key Commitment

The one structural decision that enables the AAM path: **the store and allocation are explicit from day one**. Objects live in the interpreter's own Store, not on the Java heap.

This costs almost nothing in v1:

- A `HashMap<Integer, VHObject>` with an incrementing counter is a concrete store.
- Slot reads and writes go through `store.get(addr) / store.put(addr, value)` instead of direct field access.

But it unlocks the full AAM abstraction path later: replace the concrete store with a store that joins on write, replace Alloc with a k-CFA allocator, and the interpreter becomes a sound abstract machine — without rewriting the core step function.

### 13.5 Connection to Analysis Tools

The CESK design connects directly to the [Tooling Architecture](#):

1. **Concrete CESK** → v1 interpreter. Deterministic, one successor state per step.
2. **Abstract CESK** → static analysis tool. Non-deterministic, returns a set of successor states per step. Explores all reachable abstract states to compute program properties.
3. **External annotations** (via descriptor names) tell the abstract instantiation what abstract values to assume for named elements — e.g., type constraints, intervals, or contract pre/post-conditions.

This follows the approach of Van Horn and Might (2010), *Abstracting Abstract Machines*: a systematic method for deriving sound abstract interpreters from concrete machines by parameterizing over address allocation.

### 13.6 TCO and Continuations

Because continuations are explicit, tail-call optimization is a natural extension: a tail call in tail position pops the current ReturnFrame before pushing the callee’s frame. TCO is introduced incrementally in layers — from no optimization (v1 bootstrap) through direct tail calls to non-local-return-as-tail-return — all as local changes to the step function’s frame-push logic. See [Future Directions](#) for the full layered design.

#### 13.6.1 Stack Safety

The CESK machine runs as a flat while (!isFinal(state)) { state = step(state); } loop. The continuation K is a heap-allocated linked list — the host language’s call stack is not involved in VarioHyve recursion. This means:

- **No stack overflow.** A VarioHyve program that recurses 10 million times deep will not crash with a StackOverflowError. Each recursive call allocates a continuation frame on the Java heap, not on the JVM thread stack.
- **Graceful degradation.** Without TCO, deep recursion consumes heap memory proportional to the recursion depth (one continuation frame per call). This is bounded by available heap (gigabytes), not by the JVM’s thread stack size (typically 512KB–1MB). The program may eventually run out of memory, but it degrades gracefully with GC pressure rather than crashing abruptly.
- **TCO as a space optimization.** In this context, TCO is not needed for correctness or crash prevention — it is a **space efficiency** optimization. A tail call in tail position can reuse the current continuation frame instead of allocating a new one, reducing heap allocation from O(n) to O(1) for tail-recursive loops. See Clinger (1998) for the formal treatment of proper tail recursion as a space efficiency property.

### 13.7 References

- William D. Clinger. 1998. *Proper tail recursion and space efficiency*. In Proceedings of the ACM SIGPLAN 1998 conference on Programming language design and implementation (PLDI '98), 174–185. <https://doi.org/10.1145/277650.277719>
- David Van Horn and Matthew Might. 2010. *Abstracting abstract machines*. SIGPLAN Not. 45, 9 (September 2010), 51–62. <https://doi.org/10.1145/1932681.1863553>
- David Darais, Nicholas Labich, Phúc C. Nguyen, and David Van Horn. 2017. *Abstracting definitional interpreters (functional pearl)*. Proc. ACM Program. Lang. 1, ICFP, Article 12 (September 2017), 25 pages. <https://doi.org/10.1145/3110256>

## 14 Foreign Semantic Values and the Alien/Expat Boundary

VarioHyve programs interact with values and entities governed by other semantic domains: Python, Java, C, simulation engines, databases, hardware interfaces, other VarioHyve runtimes, and the implementation VM itself. This section defines the boundary model. Foreign systems retain their own semantics and execution mechanisms; federation does not require a shared object model.

### 14.1 Design Principles

The boundary follows these constraints:

1. **Foreign semantic sovereignty.** Alienation makes a foreign value reachable; it does not supply a VarioHyve class, mixins, lexical receiver structure, or methods. The foreign adapter determines the operations it supports.
2. **Native semantic sovereignty.** VarioHyve lexical lookup, `self`, qualified outer `ClassName`, `super`, and native dispatch have fixed language semantics. Crossing to an Alien leaves that semantic domain; it does not customize lookup.
3. **Symmetric crossing.** Arguments, results, failures, identity and callbacks obey the Alien/Expat contract in both directions, including VM services.
4. **Explicit adaptation.** Ordinary VarioHyve wrappers supply VarioHyve behavior around foreign values. Alienation preserves foreign semantics; wrapping constructs a VarioHyve abstraction.

### 14.2 Aliens — Foreign Semantic Values Inside VarioHyve

An **Alien** is a VarioHyve-visible reference to a value or entity governed by a foreign semantic domain. Examples include a Python object, C pointer, function, struct value, opaque resource handle, simulator model or remote endpoint. An Alien need not support any direct operation. Its presence in an object slot does not make that containing object its lexical outer.

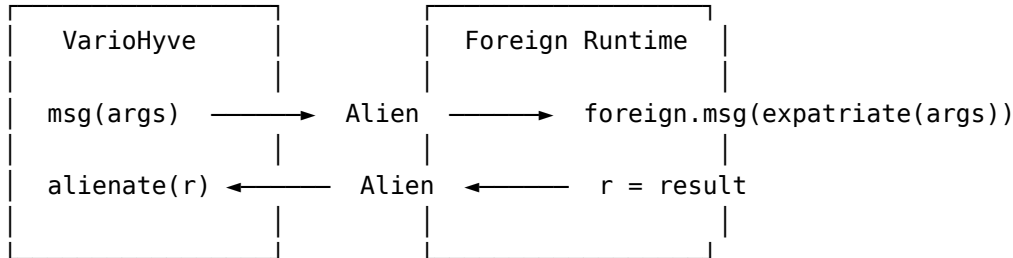
When VarioHyve code sends a message to an Alien:

```
aServer.start(port) // aServer is an Alien wrapping a Python object
```

The boundary performs the following protocol when its adapter supports that operation:

1. **Expatriate the arguments** — each VarioHyve argument is converted (or wrapped) into a foreign representation suitable for the target runtime. VarioHyve objects become Expats; primitive values are marshalled.
2. **Invoke foreign behavior** — the adapter interprets the (name, arity) request according to its foreign contract. OO forwarding is one policy; capability operations on otherwise opaque handles are another.

3. **Alienate the result** — the foreign return value is converted (or wrapped) back into a VarioHyve value. Foreign objects become Aliens; primitive values are unmarshalled.



### 14.2.1 Alien Dispatch

After evaluating an explicit receiver, the evaluator distinguishes native values from Aliens. Native values use fixed VarioHyve lookup. An Alien uses its boundary adapter; native class and mixin lookup does not continue through the Alien. Selector/arity transport does not imply that the foreign entity intrinsically has methods or a VarioHyve class. The adapter may return a genuine selector miss.

An explicit Alien invocation selects exactly its requested name/arity. A foreign getter is not a generated native accessor and is not executed to discover a callable; write `alien.f.apply(args)` to invoke its returned value explicitly. A stored Alien can supply callable behavior if its adapter advertises exact `apply/N`, including through `ForeignCallable`. Candidate selection retains the operation and does not resolve it a second time before execution.

The adapter must distinguish absence before operation entry from failure after selection. Resolution must not execute operations or call back into the language. A resolution or execution failure never resumes lookup or replays effects. Receiver and arguments are evaluated once. See [Message Sends](#).

Explicit foreign selectors named `self`, `outer` or `super` are eligible for ordinary boundary dispatch: `alien.outer` requests the foreign operation named `outer`. It does not navigate a VarioHyve scope. The receiver forms `self`, `outer` `ClassName` and `super.message` retain their separate native meanings. Native declarations do not acquire those reserved names through this exception.

### 14.2.2 Alien Identity

For identity-bearing values, a boundary defines foreign identity and preserves sharing across repeated crossings. The current in-process Java adapter canonicalizes by Java object identity. This is an adapter policy, not a universal identity law for all domains, and does not synthesize an `eq` method on Aliens.

For C, numeric address equality, allocation identity, pointer provenance and value equality are distinct concerns. Reused addresses must not automatically identify the same semantic entity. A future C adapter must define its lifetime and identity policy explicitly; no C FFI is implemented by this specification.

### 14.3 Expats — VarioHyve Objects Outside VarioHyve

An **Expat** exposes a native VarioHyve value’s behavior to a foreign domain. It may present that behavior in the foreign runtime’s idiom, but its public contract does not expose a store, address, activation or VM representation. Invocation reenters fixed VarioHyve semantics. Backend adapters may privately retain the information required to do so.

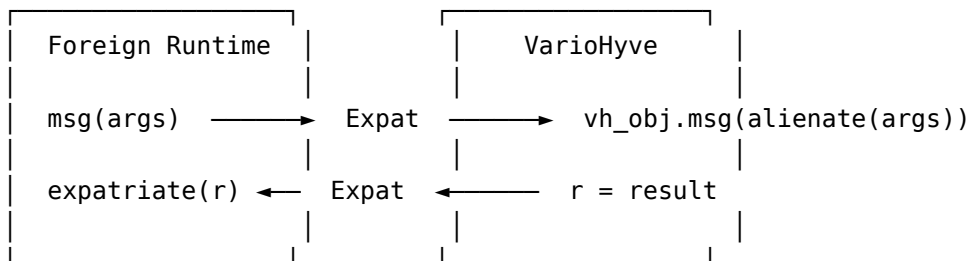
The bootstrap’s `VHExpat.send` API performs exact selector dispatch, including zero-argument access. It does not apply source invocation’s generated-binding selection: read a callable binding, then send `apply/N` to that result explicitly.

When foreign code sends a message to an Expat:

```
Python side – server is an Expat wrapping a VarioHyve object
server.start(8080)
```

The Expat performs the mirror-image protocol:

1. **Alienate the arguments** — each foreign argument is converted (or wrapped) into a VarioHyve value. Foreign objects become Aliens; primitive values are unmarshalled.
2. **Forward the message** — the foreign call is translated into a VarioHyve (name, arity) dispatch and sent to the wrapped VarioHyve object.
3. **Expatriate the result** — the VarioHyve return value is converted (or wrapped) back into a foreign representation. VarioHyve objects become Expats; primitive values are marshalled.



#### 14.3.1 Expat Surface

An Expat exposes the VarioHyve object’s **full behavioral interface** — every method the VarioHyve object can respond to is available through the Expat. In v1 (no access control), this means all methods. When access control is introduced (see [Future Directions — Access Control](#)), Expats will expose only the **export surface** — the same contract that any external VarioHyve caller sees.

#### 14.3.2 Expat Identity

Each VarioHyve object has at most one Expat proxy per foreign runtime. If the same VarioHyve object is expatriated through multiple paths, the runtime returns the same Expat. This preserves identity across the boundary.

### 14.4 The Symmetry

The Alien/Expat model is symmetric:

	Alien	Expat
<b>What it represents</b>	Foreign semantic value/entity	Native VarioHyve value
<b>Lives in</b>	VarioHyve store	Foreign runtime
<b>On message receive</b>	Expatriate args $\rightarrow$ forward $\rightarrow$ alienate result	Alienate args $\rightarrow$ forward $\rightarrow$ expatriate result
<b>Proxy for</b>	Foreign $\rightarrow$ VarioHyve	VarioHyve $\rightarrow$ Foreign

The two operations preserve compatible reference round trips:

- **Alienate:** foreign value  $\rightarrow$  VarioHyve value (wrap foreign objects as Aliens, unmarshall primitives)
- **Expatriate:** VarioHyve value  $\rightarrow$  foreign value (wrap VarioHyve objects as Expats, marshall primitives)

For a native reference  $v$  and an identity-bearing foreign value  $f$  within a compatible runtime pair:

```
alienate(expatriate(v)) = v
expatriate(alienate(f)) = f
```

An Expat owned by the importing runtime recovers the original native value; an Alien exported to its originating adapter recovers the foreign value. Cross-runtime references require an explicit bridge, not a guessed identity. By-value conversions instead follow their declared value semantics and may normalize host types; these equations do not promise host representation identity for marshalled primitives.

### 14.5 Explicit Wrappers

A native wrapper can combine several foreign capabilities and supply its own identity, methods, invariants and lifecycle policy. Constructor parameters remain initializer-only, so persistent dependencies are bound into slots:

```
class Device(deviceHandle, reader, writer, closer) {
 handle = deviceHandle;
 readFn = reader;
 writeFn = writer;
 closeFn = closer;

 read(buffer) { readFn(handle, buffer) }
 write(buffer) { writeFn(handle, buffer) }
 close { closeFn(handle) }
}
```

The function capabilities can support `apply/n`, while the handle itself remains opaque and supports no messages. This illustrates an adapter contract, not a built-in C calling convention. Multiple wrappers may share the same Alien while having distinct native identities, lexical scopes and behavior.

## 14.6 VM Services and the Trusted Seed

Language-visible VM services are foreign capabilities and use the same membrane. There is no separate privileged native-function invocation ontology. A finite trusted seed may privately allocate values, maintain stores, and implement lookup and continuations; those internal operations are not language services.

A VM service constructing a native object must return its Expat so import recovers the native value. Returning a Java `VMObject` representation as an ordinary foreign result is invalid. Representation-sensitive capabilities, if ever exposed, require explicit authority and backend-private Expat unwrapping; public Expats provide no raw-world or address access. See [The VarioHyve–VM Boundary](#).

## 14.7 Constructor Arguments and Optional Bundles

The instantiation service can supply Aliens directly as any of a top-level module’s [constructor arguments](#). It can also assemble an ordinary bundle conventionally called a `kernel`. The parameter name, number of arguments, and use of a bundle are application choices, not requirements of the foreign protocol:

```
class(consoleService, fileSystemService) {
 // Each argument may be an Alien or an ordinary VarioHyve object.
 console = consoleService;
 fs = fileSystemService;
}
```

An optional bundle may itself be an Alien or a VarioHyve object holding Aliens. Objects already belonging to VarioHyve retain their identity through the normal Expat round trip. The service passes existing objects without exposing its other capabilities as a lexical scope. Normal module evaluation has only an empty top activation at its base, with no enclosing application scope; the class inherits only from root `Object` and composes no mixins. Its constructor inputs are its only external capability entry route.

Tools may explicitly supply Aliens or native objects in development/debugging evaluation contexts. The same boundary contract applies there; those bindings must not silently become capabilities of normal modules’ empty top activations.

## 14.8 Primitive Marshalling

Certain values cross the boundary without proxying — they are **marshalled** (converted by value) rather than wrapped:

- **Numbers** — VarioHyve numbers  $\leftrightarrow$  host numeric types

- **Strings** — VarioHyve strings  $\leftrightarrow$  host strings
- **Booleans** — `true/false`  $\leftrightarrow$  host booleans
- **Null** — canonical `null`  $\leftrightarrow$  host `null/None/nil` (`nil` is only a possible platform alias)

Conversions must have explicit value semantics and reject unsupported values. The current Java adapter supports strings, Booleans, null and signed 64-bit integers; it normalizes Java integral representations to Long and rejects unsupported numeric forms. A foreign pointer must not be passed as an integer and expected to retain pointer identity or provenance. Use an explicit handle whose interpretation belongs to its adapter.

Canonical null and booleans are ordinary native objects inside VarioHyve, with language-defined methods and thunk-based control flow. Their Java representation at the boundary is `null`, `Boolean.TRUE` or `Boolean.FALSE`, never an Expat. Incoming Java values map back to the corresponding canonical object in that runtime. This applies equally to results, arguments and callbacks.

Conversion recognizes the canonical identities; it does not call `toString`, `isNull`, not or a truth-testing protocol to classify an arbitrary object. Another native object with similar behavior retains its own identity and uses the ordinary Expat route. Core null and empty expressions select the native canonical null directly, without a trip through Java conversion.

## 14.9 Design Considerations

### 14.9.1 Arity Mismatch

Foreign runtimes may not have a native concept of (name, arity) dispatch. The Alien/Expat layer must map between VarioHyve's arity-based dispatch and the foreign runtime's calling convention:

- **Python/Ruby** — keyword arguments, `*args`, default parameters. The mapping is name-based; arity mismatch may surface as a `TypeError` on the foreign side.
- **Java** — selector and arity may participate in an explicit adaptation policy; Java overload resolution can also depend on types. Automatic reflection is not required and is not supplied by the bootstrap adapter.
- **C** — a pointer or struct need not receive messages. Explicit capabilities can instead provide `call`, `read`, `write`, `field` or `dereference`, subject to a defined ABI, layout, ownership and lifetime contract.

### 14.9.2 Concurrency Boundary

A future Semantic Language Interface (SLI) may govern scheduling between a sequential VarioHyve VM and concurrent foreign domains. The current boundary implements synchronous same-thread callbacks, not an SLI scheduler. See [Future Directions — Concurrency and Parallelism](#).

### 14.9.3 Error Propagation

Selector absence and an operation's raised failure are different outcomes. Foreign failures become defined language failures; native failures crossing an Expat become defined for-

eign diagnostics. Concrete host exception classes must not define language control flow or leak as ordinary language values. The bootstrap preserves `FOREIGN_FAILURE`, `CONVERSION`, `OWNERSHIP` and `EXECUTION_POLICY` through machine execution and callback failure propagation. Native missing-selector and control failures remain `LANGUAGE_FAILURE`. These are defined diagnostic categories, not retained Java exception types or causes. The runtime does not implement resumable conditions or asynchronous callbacks.

#### **14.9.4 Lifecycle and Garbage Collection**

Ownership and lifetime belong to the boundary contract for the foreign domain. The current Java adapter retains strong proxy references for the runtime's lifetime. Future adapters must account for resources without a GC, callback lifetime, and cross-boundary cycles. No universal reclamation protocol is yet defined; exposing raw addresses is not a solution.

#### **14.9.5 Foreign Kinds and Capability Contracts**

An adapter may classify values as callable, pointer, struct, remote endpoint or semantic model. Such classifications are foreign contracts, not VarioHyve classes. Future analysis can consume explicit contracts for effects, callbacks, blocking, determinism or state transitions; unknown behavior remains opaque. The bootstrap has no general contract metadata system, provenance registry or foreign reflection API. Those additions must preserve this semantic boundary.

## 15 Tooling Architecture

VarioHyve has a highly dynamic base runtime — everything is an object, every name is a late-bound message send, and no static declarations are required. Yet the language is designed to support an arbitrarily large number of **optional static analysis tools**: type checkers, abstract interpreters, contract verifiers, documentation generators, linters, and others.

The key principle: **the source stays clean; tools annotate externally**.

The annotation tools described below do not supply runtime capabilities merely by attaching descriptors. Interactive evaluation is a separate tool operation with an explicitly selected context, as described next.

### 15.1 Evaluation Contexts

**Design requirement; a general tooling API is not yet implemented.** Normal module entry uses an empty top activation, providing no capability bindings and no path to a platform through its parent or receiver. The frame still exists as the base of execution. See [Program Structure](#).

Development tools may supply richer contexts explicitly:

Context	Available capabilities
Normal module definition	Empty top activation; external inputs subsequently arrive as constructor arguments
Design-time expression evaluation	Objects explicitly supplied by the tool, such as sample data or a console
Debugger evaluation	Bindings of the selected activation, plus any explicitly supplied debugging services

Use activation objects and ordinary lexical object graphs. Tool-supplied bindings follow the same name resolution as other bindings; they do not replace lookup, introduce a second Environment abstraction, or create a global service registry. Language-visible VM debugging and mirror services must use the normal Alien/Expat boundary. Access to a debug context does not imply public world/store access, and existing callback and non-local-return rules still apply. The bootstrap does not reify activations for programs. A future debugger may select an activation internally without making it a first-class public capability; any later mirror interface needs an explicit authority contract.

A tool should identify which context it evaluates in and which additional objects it supplies. It must not silently add them to the root of ordinary module evaluation. Results can capture the supplied context: closures and class definition scopes retain their lexical references after the tool call returns. A class using tool-provided superclass or mixin bindings

is defined in that context, rather than satisfying the isolated top-level module contract. Export or persistence must account for these references explicitly.

The current `BootstrapRuntime.evaluate` is a useful initial expression harness: its top activation has no local slots, but its parent and receiver are the assembled kernel. This is a supplied evaluation context. The separate `BootstrapRuntime.instantiateModule` operation defines and constructs modules over an empty root with explicit arguments. Neither operation implements a general selected-activation debugger API.

## 15.2 Descriptors as the Naming Bridge

The [descriptor layer](#) gives every syntactic element an attachment point for meta-information. One critical use of this attachment point is **naming**: giving an identity to an AST node so that external tools can reference it.

Most elements already have intrinsic names — slot names, method names, parameter names. For elements that lack a natural identity (anonymous sub-expressions, intermediate results), a descriptor can supply one:

```
{
 compute(x, y) {
 var result = (x.mul(y).plus(x) : intermediateSum)
 result.div(y)
 }
}
```

The descriptor `intermediateSum` does not alter execution — the core interpreter ignores it. But a tool can now attach information to that specific sub-expression by referencing its name.

## 15.3 External Annotation Files

Tools store their annotations in **external files**, separate from the source. Each tool defines its own annotation format and references elements by name:

```
hypothetical type-checker annotation file
compute.x : Int
compute.y : Int
compute.result : Int
compute.intermediateSum : Int

hypothetical abstract-interpreter annotation file
compute.x : interval(0, 100)
compute.y : interval(1, 50)
compute.intermediateSum : interval(0, 10100)
```

This separation has several consequences:

- **The source is never polluted.** A program reads the same whether zero or ten tools are in use. Adding a tool does not require modifying source files.
- **Multiple tools coexist without conflict.** Each tool has its own annotation files. A type checker’s annotations and an abstract interpreter’s annotations never interfere — they are independent views of the same program.
- **Tools are optional from the runtime’s perspective.** The base interpreter runs without consulting any annotation file. Tools layer on top.
- **Annotations can evolve independently.** Upgrading a tool, changing its analysis strategy, or switching abstract domains does not touch the source.

## 15.4 Qualified Naming

To disambiguate identically named elements in different scopes, tools use **qualified paths** rooted at the module or top-level object:

```
module.ClassName.methodName.localVar
module.ClassName.methodName.paramName
```

The exact path format is a tooling convention, not a language-level concern. The language provides the names (intrinsic or via descriptors); tools define how to compose them into unambiguous references.

## 15.5 Connection to Abstract Interpretation

The v1 interpreter is an **AAM-style CESK machine** — a small-step abstract machine whose store and allocation are explicit and parameterized. By swapping the concrete allocator for an abstract one (k-CFA), the same interpreter core becomes a sound abstract machine capable of type checking, interval analysis, or other static analyses. See [Interpreter Design](#) for the full machine specification.

This follows two complementary approaches:

- **AAM** (Van Horn and Might, 2010) — systematically derive abstract interpreters from concrete CESK machines by bounding address allocation.
- **ADI** (Darais, Labich, Nguyen, and Van Horn, 2017) — parameterize a definitional interpreter over monadic operations; swap concrete monads for abstract ones.

Both converge at the same insight: the store and allocation function are the parameterization points. The CESK machine is the v1 choice because it maps naturally to Java (explicit states, no higher-kinded types).

The descriptor-based naming layer provides the bridge between any abstract analysis and the programmer’s intent. External annotation files tell the abstract instantiation what abstract domains, type assumptions, or invariants to apply to each named element — without those assumptions being embedded in the source.

Because descriptor names are stable across interpreter instantiations (the AST is the same whether run concretely or abstractly), a single annotation file can serve any abstract instantiation that understands its format.

## 15.6 Design Summary

Concern	Mechanism
Element identity	Intrinsic names (slots, methods, params) or descriptor-supplied names
Annotation storage	External files, one per tool or analysis
Cross-referencing	Qualified paths (tooling convention)
Annotation runtime cost	Zero — interpreter ignores descriptors and never reads annotation files
Interactive evaluation	Explicit tool-supplied or selected activation context; ordinary lookup and Alien/Expat rules
Tool independence	Each tool has its own view; no shared annotation format required
Source cleanliness	Descriptors are optional; most elements need no explicit naming
Interpreter design	AAM-style CESK — parameterized store/allocator enables concrete and abstract execution

## 16 Future Directions

This section documents features intentionally omitted from v1 of VarioHyve, the rationale for their absence, and the planned approach for introducing them after v1 is bootstrapped.

Examples using kernel use an optional ordinary capability bundle. A normal top-level class can take any declared number of existing object arguments; its lexical base is an empty top activation with no enclosing application scope. It inherits only from root Object and composes no mixins. Tools may explicitly supply richer [evaluation contexts](#) for development and debugging, using the same lexical and boundary rules. Class fragments using other superclass/mixin names assume an enclosing scope supplying them. Future features must preserve all four [capability routes](#): inheritance, composition, nesting and constructor parameters. A proposed runtime default-handler service must be supplied explicitly to the execution service or through application arguments, not discovered via an ambient kernel binding.

### 16.1 Operators and Control Structures

**Status:** Not in v1.

v1 has no binary operators ( $a + b$ ,  $a > b$ ) and no built-in control structures (if/then/else, while loops). All computation is expressed through message sends:

```
x.plus(dx) // instead of x + dx
x.greaterThan(y) // instead of x > y
cond.ifTrue(fun { ... }).ifFalse(fun { ... }) // instead of if/else
```

**Why not in v1:** Operators and control structures are surface syntax — they don't introduce new semantics. Adding them as fixed grammar rules would make the language harder to extend and evolve. They are better expressed as syntactic sugar that desugars to message sends.

**Planned approach:** After v1 is bootstrapped, we plan to add a **monadic extensible parser** in the spirit of Lean 4. This will allow syntax extensions to be defined *within the language itself*:

- Binary operators (+, -, \*, >, ==) as sugar for message sends
- Control structures (if/else, while, for) as sugar for message sends with lambda arguments
- Pattern matching
- Specialized DSLs defined by library authors

The extensible parser will desugar all new syntax forms into the core v1 AST (message sends, object literals, lambdas). This ensures the interpreter and runtime remain simple while the surface language grows in expressiveness.

## 16.2 Named and Optional Arguments

**Status:** Not in v1.

v1 message sends support only positional arguments:

```
point.move(3, 4)
```

**Why not in v1:** Positional arguments are sufficient to bootstrap the language and its core libraries. Named arguments, default values, and varargs add complexity to the dispatch mechanism (arity-based lookup, synthesized setters) and would slow down the bootstrapping effort.

**Planned extensions:**

- **Named arguments** — allow callers to specify arguments by name for readability and to avoid positional errors. We believe named arguments will clearly improve the readability and ease the use of the language, especially for message sends with multiple arguments where positional ordering is not obvious:

```
point.move(dx= 3, dy= 4)
rect.resize(width= 100, height= 50)
```

- **Optional arguments with defaults** — parameters with predefined values that callers can omit:

```
connect(host, port= 8080, timeout= 30) { ... }
connect("localhost") // port and timeout use defaults
```

- **Varargs** — methods that accept a variable number of arguments:

```
log(messages...) { ... }
log("start", "processing", "done")
```

**Token consistency:** Named argument syntax (`dx= 3`) uses the same `=` token as [setter sugar](#) and slot initialization — `=` is VarioHyve’s unified binding operator. In call-argument position the grammar will introduce a dedicated argument rule that distinguishes `name = expr` (named argument) from a positional expression, keeping the two contexts syntactically unambiguous.

The exact syntax and dispatch semantics for these features will be designed after v1 is stable, potentially leveraging the extensible parser to define them as syntax extensions rather than grammar changes.

## 16.3 String Interpolation

**Status:** Not in v1.

v1 strings are plain literals. Building dynamic strings requires explicit concatenation:

```
"hello, ".concat(name).concat("!")
```

**Why not in v1:** Interpolation is surface syntax — it desugars to message sends. The core runtime doesn't need it.

**Planned approach:** An interpolation form (syntax TBD) that desugars to concat calls:

```
"hello, ${name}!" // desugars to "hello, ".concat(name).concat("!")
```

This is a natural candidate for the extensible parser. The interpolation syntax would be defined as a syntax extension, not a grammar change.

## 16.4 Collection Literals

**Status:** Not in v1.

v1 has no built-in syntax for arrays, lists, or maps. Collections must be built via message sends:

```
List.of(1, 2, 3)
Map.of(#a, 1, #b, 2)
```

**Why not in v1:** Collection literals are sugar — they create objects via the same construction protocol as everything else. Getting the core object model right first ensures collection types have a solid foundation.

**Planned approach:** Literal syntax for common collection types (syntax TBD):

```
[1, 2, 3] // array/list literal
{#a: 1, #b: 2} // map/dictionary literal – syntax must not conflict
↪ with object literals
```

The exact syntax requires careful design to avoid ambiguity with object literals (`{ ... }`) and will likely be introduced via the extensible parser.

## 16.5 Mixin Contract Verification

**Status:** Not in v1.

In v1, mixin methods that call `super` assume the application site satisfies their implicit contract — e.g., that `super` responds to a given message with a given arity. If the contract is not satisfied, a runtime error occurs.

**Why not in v1:** Contract verification requires static analysis of mixin method bodies to extract the set of super messages they send, then checking each application site against that set. This is analysis tooling, not core runtime.

**Planned approach:** An external verification tool that:

1. Inspects each mixin's methods and collects the set of super sends (name + arity).
2. For each class that uses the mixin, resolves what super would point to at that position in the mixin list.
3. Checks that the target class or mixin provides all required methods.
4. Reports violations as warnings or errors.

This fits naturally into the [Tooling Architecture](#): the tool reads the AST, follows the mixin lists and super chains defined in the [Object Model](#), and reports results externally — without modifying the source or the runtime. Descriptors could be used to annotate a mixin's expected super contract explicitly, enabling more precise verification.

## 16.6 Deep Family and Mixin Composition

**Accepted long-term direction; deferred beyond the initial ordered composition.** Vari-oHyve intends to support deep family composition with state in member classes. This requires more than selecting a same-named member from an ordered list: it must define how nested declarations, constructor signatures, state and sibling references compose. The bootstrap must retain declaration identities, definition scopes and composition origins without claiming that this richer operation already exists.

Initially, own contributions override composed contributions, composed entries are considered in declaration order (leftmost first), and inherited contributions follow. A member override selects a whole declaration. Duplicate names within one body remain errors. Ordinary slot initializers still execute in declaration order; member accessors bind and cache classes independently of that sequence. This simple policy leaves deep member merging for a later increment.

Future operators may include aliases, renaming, hiding, exclusion, explicit precedence and state mapping/sharing. These are different operations: an alias need not rename recursive self sends; hiding need not remove internal access; and selecting a method does not resolve unrelated state collisions. Repeated application of one mixin needs occurrence identity rather than reliance only on the shared definition object or mutable list index.

Effectful initializers require their own argument routing and execution plan. An algebra over declarations cannot assume registrations, callbacks or other foreign effects commute, nor infer rollback on construction failure. Preserve one-object construction and explicit Alien/Expat boundaries as this model grows. See [the accepted decisions and sequence](#) and [the earlier design-space analysis](#).

## 16.7 Trait-Style Conflict Resolution

**Status:** Not in v1.

In v1, when multiple mixins define a method with the same name and arity, the **first match wins** (leftmost mixin in the `<`: list). This is simple and deterministic, but conflicts are silent — the programmer may not realize a method was shadowed.

**Why not in v1:** Conflict resolution requires detection (scanning all mixins for overlapping method signatures) and a resolution mechanism (syntax for the programmer to explicitly choose which mixin's method to use). Both add complexity to the runtime and the grammar.

**One future option, not yet an accepted replacement for ordered overrides:** trait-style conflict handling in two phases:

1. **Detection** — at class creation time (or via static analysis), scan all mixins for methods with the same (name, arity). If a conflict is found and the class does not provide its own override, raise an error.
2. **Explicit resolution** — the class must define its own method that explicitly delegates:

```
class App <: Logging, Auditing < Base {
 // Both Logging and Auditing define log(msg)
 // The programmer must resolve:
 log(msg) { Logging.log(self, msg) }
}
```

This requires a mechanism for calling a specific mixin’s method directly (passing the receiver explicitly). The exact syntax is TBD.

Reified mixin definitions preserve information useful for conflict detection. The list alone is not the complete future model: declaration identity, definition scope, application occurrence and state identity must also survive composition. Do not assume the entire algebra can be added without changing backend metadata.

### 16.7.1 doesNotUnderstand as a Message

The Newspeak/Smalltalk approach — where `doesNotUnderstand` is itself a message sent to the receiver — is a natural fit for VarioHyve’s “everything is a message” philosophy. An object could intercept failed dispatches by implementing a `doesNotUnderstand(selector, args)` method, enabling proxies, dynamic delegation, and other meta-programming patterns.

However, this introduces design questions that require careful exploration:

- **Performance:** every failed dispatch becomes another dispatch, which can fail again (infinite recursion risk).
- **Debugging:** intercepted failures can mask bugs — a typo in a method name silently triggers `doesNotUnderstand` instead of a clear error.
- **Abstract interpretation:** `doesNotUnderstand` as a method is a dynamic dispatch with an open-ended set of selectors — it complicates static analysis significantly.
- **Base implementation:** does Object provide a default `doesNotUnderstand` that raises the fatal error? Or is it a primitive mechanism?

This is a post-v1 design exploration. See [Future Directions](#) for other deferred features.

## 16.8 Exception System

**Status:** A language exception/condition system is not in v1. Missing dispatch (`doesNotUnderstand`), dead return contexts and other invalid language operations terminate execution with diagnostics. Boundary failures also remain fatal, while preserving defined categories such as conversion, ownership and foreign failure; see [Foreign Error Propagation](#). These diagnostic categories do not provide language-level catch, resume or retry.

### 16.8.1 Design Requirements

A VarioHyve exception system must satisfy four requirements that arise directly from the SoS deployment model:

#	Requirement	SoS motivation
(a)	A component gets a chance to handle an exception <b>locally</b> , near the signaling site, before the problem escalates	A subsystem should be self-healing by default — local restarts keep system-level responsibilities contained
(b)	If unhandled locally, the exception <b>escalates up the call chain</b> — each caller gets a chance to handle it before the next	Organizational boundaries map to call boundaries; governance decisions propagate upward naturally
(c)	A handler can <b>resume execution at the signaling site</b> — optionally supplying a degraded value	Sensors, actuators, and communication channels fail transiently; the correct response is often “use the last known value and continue”, not “abort the mission”
(d)	If no handler fires, the exception reaches a <b>kernel-level default handler</b> whose behavior is configurable per deployment context	A simulation kernel, test kernel, and production kernel have different appropriate “last resort” behaviors — this should not be hardcoded in the language

### 16.8.2 Why Not Try/Catch

Java-style try/catch satisfies (a), (b), and (d) but **cannot satisfy (c)**. When a catch block executes, the call stack above the handler has already been discarded. There is no way to resume the signaling site from inside a catch block. For a language where sensor-read degradation, communication-timeout recovery, and graceful degradation under partial failure are first-class concerns, this is a disqualifying limitation.

Delimited continuations and algebraic effects can satisfy (c) but require capturing a slice of the continuation as a first-class value. In the CESK machine, this means the continuation  $K$  is no longer a flat linked list at signal time — it must be split, allocated, and passed to the handler. This has significant consequences for the AAM abstract interpretation path: captured continuation slices create potential state-space explosion in the abstract machine, undermining the static analysis story that is one of VarioHyve’s core architectural commitments.

**Implementation tradeoff — post-v1 study item.** The condition system below can be implemented either as a direct mechanism (two new  $K$  frame types,

no additional language semantics) or as a derived mechanism built on top of delimited continuations. The direct approach minimises runtime complexity and has predictable AAM cost. The derived approach is theoretically more general and may simplify the interpreter core if delimited continuations are introduced independently for other purposes. This tradeoff — direct vs. derived — will be evaluated after v1 is bootstrapped.

### 16.8.3 The Common Lisp Condition System

The **Common Lisp condition system** satisfies all four requirements while requiring only two new continuation frame types — no continuation capture at any point.

The design separates three orthogonal concerns:

Concept	Installed by	Lives where	Purpose
<b>Condition</b>	signaling code	raised at the error site	describes what went wrong
<b>Handler</b>	caller code	anywhere in the call chain	decides the recovery <i>policy</i>
<b>Restart</b>	component code	near the signaling site	named recovery <i>mechanism</i> — the actual fix

**The key insight:** the *policy decision* (which caller says “on SensorError, use the last measured value”) is separated from the *mechanism* (which component provides the code that actually retrieves and returns the last measured value). The handler is written by the caller without knowing the sensor’s internals; the restart is written by the sensor component as a named escape hatch. The handler invokes the restart by name. The call stack is never unwound — the signaling call simply returns the restart’s value.

**CESK cost:** Two new frame types pushed on K, no slice capture: - `signal(condition)` — walks K collecting matching `HandlerFrames`, calls each in order **with K intact**. The activation that called `signal` is still on the stack. If a handler calls `invokeRestart(name)`, the runtime scans K for the matching `RestartFrame` and executes it; the result becomes the return value of `signal`. Only if a handler explicitly aborts does any unwinding occur. - `withRestartFrame(name, fn)` — pushes a `RestartFrame(name, fn)` onto K - `withHandlerFrame(condClass, fn)` — pushes a `HandlerFrame(condClass, fn)` onto K

This keeps K as a flat, finite, finitely summarizable linked list in the abstract machine — the AAM path is unaffected.

### 16.8.4 Worked Example 1 — Component Handles and Resumes (local restart)

The sensor component installs a restart near the signaling site. It also installs a local handler that immediately invokes that restart — the component resolves the problem itself without involving the caller at all.

```
// Layer 1 – the raw hardware sensor: this is the signal site
class TemperatureSensor(hardwareId, kernel) {
```

```

hardwareId = hardwareId
hardware = kernel.hardware
exc = kernel.exceptions

read() {
 var raw = hardware.readSensor(hardwareId) // raw I/O call
 raw.isError.isTrue(fun {
 exc.signal(SensorError(hardwareId, raw.errorCode)) // << SIGNAL
 ↪ SITE
 })
 raw.value
}
}

// Layer 2 – the component controller: installs both the restart and a local
↪ handler
class SensorController(rawSensor, kernel) {
 rawSensor = rawSensor
 exc = kernel.exceptions
 lastValue = 0

 read() {
 // Restart installed here – the component owns the recovery mechanism
 exc.withRestart(#useLast, fun { lastValue }) {
 // Local handler also installed here – component also owns the
 ↪ policy
 // Fires before any handler in the call chain above
 exc.withHandler(SensorError, fun(e) {
 exc.invokeRestart(#useLast)
 }) {
 var v = rawSensor.read() // calls TemperatureSensor.read –
 ↪ may signal
 lastValue = v // update cache on success
 v
 }
 }
 }
}
}

```

When `rawSensor.read()` calls `exc.signal(SensorError(...))` inside `TemperatureSensor.read`: 1. `signal` walks K upward from the signal site. The `HandlerFrame(SensorError, ...)` installed by `withHandler` in `SensorController.read` is the first match. 2. The handler calls `invokeRestart(#useLast)`. The runtime finds the `RestartFrame(#useLast, ...)` also on K, installed just above by `withRestart`. 3. The restart executes `fun { lastValue }` — returns the cached value. 4. `signal` returns `lastValue` to the call site — the `exc.signal(...)` expression inside `TemperatureSensor.read` evaluates to `lastValue`. The `ifTrue` body completes, and `TemperatureSensor.read` returns `lastValue`. 5. `SensorController.read` receives `lastValue` as `v`, updates `lastValue`, and returns it normally.

The caller of `SensorController.read` sees a normal return value and is never aware an error occurred. The component fully encapsulates both the mechanism and the policy.

### 16.8.5 Worked Example 2 — Caller Handles and Resumes (remote restart)

The sensor component installs **only** the restart — it declares the recovery mechanism but makes no policy decision. The caller installs the handler and decides which restart to invoke. The stack is not unwound: `signal` finds the handler upstack, the handler calls `invokeRestart`, and the sensor's `read()` returns normally.

```
// Layer 1 – same raw hardware sensor as Example 1: this is the signal site
class TemperatureSensor(hardwareId, kernel) {
 hardwareId = hardwareId
 hardware = kernel.hardware
 exc = kernel.exceptions

 read() {
 var raw = hardware.readSensor(hardwareId)
 raw.isError.isTrue(fun {
 exc.signal(SensorError(hardwareId, raw.errorCode)) // << SIGNAL
 })
 raw.value
 }
}

// Layer 2 – the component controller: installs ONLY the restart, no local
// handler
// The component declares the recovery mechanism but makes no policy decision
class SensorController(rawSensor, kernel) {
 rawSensor = rawSensor
 exc = kernel.exceptions
 lastValue = 0

 read() {
 exc.withRestart(#useLast, fun { lastValue }) {
 var v = rawSensor.read() // may signal SensorError – will
 // escalate
 lastValue = v
 v
 }
 }
}

// Layer 3 – the mission coordinator: installs the handler, decides the
// policy
class MissionCoordinator(sensors, kernel) {
 sensors = sensors
 exc = kernel.exceptions
```

```

runMission() {
 exc.withHandler(SensorError, fun(e) {
 // Policy decision: when any sensor raises SensorError, use its
 ↪ last value
 exc.invokeRestart(#useLast)
 }) {
 // Both reads are covered by the single handler above
 var alt = sensors.altimeter.read()
 var gyro = sensors.gyroscope.read()
 computeManeuver(alt, gyro)
 }
}
}

```

When `rawSensor.read()` calls `exc.signal(SensorError(...))` inside `TemperatureSensor.read`: 1. `signal` walks K upward from the signal site. It passes through the `RestartFrame(#useLast, ...)` installed by `SensorController.read` — restart frames are noted but do not stop the search. 2. `signal` continues up K through `SensorController.read`'s activation, then through `sensors.altimeter.read`'s call frame in `runMission`, until it finds the `HandlerFrame(SensorError, ...)` installed by `withHandler` in `MissionCoordinator.runMission`. 3. The handler calls `invokeRestart(#useLast)`. The runtime scans K for the `RestartFrame` — finds the one installed in step 1, still present on K. 4. The restart executes `fun { lastValue }` — returns the cached value from `SensorController`. 5. `signal` returns `lastValue` to the signal site inside `TemperatureSensor.read`. The `exc.signal(...)` expression evaluates to `lastValue`. `TemperatureSensor.read` returns `lastValue` → `SensorController.read` returns it as `v` → `sensors.altimeter.read()` in `runMission` returns `lastValue`. Execution continues with `var alt = lastValue`.

`TemperatureSensor` has no knowledge of who will handle its error. `SensorController` has no knowledge of what policy will be applied to its restart. `MissionCoordinator` has no knowledge of how `SensorController` stores its last value — it only knows the restart's name. The three concerns are fully decoupled.

### 16.8.6 The `doesNotUnderstand` Resolution

`doesNotUnderstand` becomes a condition object, not a method on `Object`:

```

class MessageNotUnderstood(receiver, selector, args) < Condition { }

```

When dispatch fails, the runtime signals a `MessageNotUnderstood` condition. The kernel's default handler terminates with diagnostics. Objects that need proxy or delegation behavior install a `withHandler(MessageNotUnderstood, ...)` around relevant code — no DNU method required, no open-ended dispatch to an unknown method, no complication of the abstract interpretation graph.

This resolves all four concerns raised in the `doesNotUnderstand` discussion above.

### 16.8.7 Kernel Default Handlers

If signal reaches the bottom of K without a matching handler, the condition is delivered to `kernel.exceptions.defaultHandler`. Different kernels provide different behaviors:

Kernel	Default handler behavior
ConcreteKernel	Print diagnostic (receiver, selector, condition class), terminate
TestKernel	Record the unhandled signal as a test failure, optionally continue
SimKernel	Log to the simulation trace with timestamp, continue with a degraded state
ModelCheckKernel	Nondeterministically handle or not handle — explores both paths
AbstractKernel	Soundly approximate: assume any handler may or may not fire

The v1 behavior (fatal termination) becomes the `ConcreteKernel`'s default handler — a runtime object, not a hardcoded language rule.

### 16.8.8 SDK Sugar Layer

The condition system primitives are low-level by design. An SDK layer provides familiar sugar:

```
class Exceptions(kernel) {
 exc = kernel.exceptions

 // try/catch — for callers who want unwind-on-error semantics
 try(body) catch(condClass, handler) {
 exc.withHandler(condClass, fun(e) {
 exc.abort(e, handler) // explicit unwind + handler body
 }) { body() }
 }

 // ensure — always run cleanup, regardless of outcome
 ensure(body, cleanup) { ... }

 // guard — assertion-style; signal a condition if predicate fails
 guard(pred, conditionFn, body) {
 pred().ifFalse(fun { exc.signal(conditionFn()) })
 body()
 }
}
```

Users who want Java-style try/catch import from the SDK. Users who need restarts use the core mechanism directly.

### 16.8.9 AAM Compatibility

Mechanism	AAM cost
Try/Catch	Low — HandlerFrame is finite; unwind is K-scan
Condition + Restarts	Low — HandlerFrame + RestartFrame on K; signal is K-scan, no capture
Algebraic effects	High — continuation capture = reachable-state explosion

The condition system keeps K as a finitely summarizable linked list. The abstract machine can represent `HandlerFrame(condClass, fn)` and `RestartFrame(name, fn)` symbolically — condition class addresses and restart name symbols are both finite in any given program. The abstract `signal` operation joins the values returned by all reachable handlers, which is a standard widening step. No new AAM machinery is required.

### 16.8.10 Incremental Roadmap

Phase	What	Interpreter impact
<b>v1</b>	Fatal <code>doesNotUnderstand</code> and dead-context errors. No condition system.	None
<b>v2</b>	Condition base class. <code>signal/withHandler</code> primitives in <code>kernel.exceptions</code> . <code>MessageNotUnderstood</code> replaces hardcoded DNU. Kernel default handler receives unhandled conditions.	Two new K frame types: <code>HandlerFrame</code> , <code>DefaultHandlerFrame</code>
<b>v3</b>	<code>withRestart/invokeRestart</code> primitives. Full component-level recovery. abort for explicit unwind.	One new K frame type: <code>RestartFrame</code> . <code>signal</code> extended to scan for restarts
<b>v4</b>	SDK sugar layer: <code>try/catch/ensure/guard</code> . Dead-context error becomes a condition.	SDK library only — no interpreter changes
<b>Future</b>	Abstract interpretation of condition handlers. Verify that all signaled conditions have matching handlers. Mixin-contract violations as conditions.	Analysis tooling

## 16.9 Tail-Call Optimization

**Status:** Not in v1.

The v1 CESK interpreter is stack-safe — continuations are heap-allocated, so deep recursion cannot crash the host runtime (see [Interpreter Design — Stack Safety](#)). However, without TCO, each recursive call allocates a continuation frame on the heap, consuming  $O(n)$  memory for  $n$ -deep recursion. TCO is a **space efficiency** optimization: tail calls reuse the current frame instead of allocating a new one, reducing heap usage to  $O(1)$  for tail-recursive loops.

### 16.9.1 Implementation Layers

TCO is introduced incrementally. Each layer builds on the previous one; all layers are local changes to the step function’s frame-push logic.

#### 16.9.1.1 Layer 0: No TCO (v1 bootstrap)

The v1 interpreter allocates a continuation frame for every call. Programs run correctly — the CESK machine is stack-safe — but deep recursion consumes  $O(n)$  heap. This is the simplest correct implementation and the starting point.

**What works:** everything. **What’s expensive:** any loop expressed as recursion (which, without operators or control structures, is *all* loops).

#### 16.9.1.2 Layer 1: Direct tail calls

Detect **syntactic tail position** in method and lambda bodies: the last expression before the implicit return. When step is about to dispatch a message send in tail position, it pops the current ReturnFrame before pushing the callee’s frame. This is a one-pass AST check and ~20 lines in step.

**Covers:**

```
// the recursive call is the last expression – tail position
loop(n, acc) {
 ^ loop(n.minus(1), n.mul(acc))
}

// generalized – the tail call resolves to any method, not just self
process(data) {
 ^ transform(data).process(next(data))
}
```

**Does not cover:** ^ inside lambdas passed to control-flow methods (the common `ifTrue/ifFalse` pattern).

Note: Layer 1 already provides **generalized** TCO for direct tail calls — any message send in tail position is optimized, not just self-recursive ones. Because all names are late-bound, the runtime cannot distinguish self-recursion from mutual recursion or delegation at the

call site. The optimization is purely structural: “is this call the last thing the method does?” This generality comes for free — there is no implementation difference between optimizing a self-call and optimizing a call to an unknown method.

### 16.9.1.3 Layer 2: Non-local return as tail return

When  $\wedge$  fires inside a lambda, it unwinds the continuation to the enclosing method’s `ReturnFrame`. If that `ReturnFrame` is the topmost frame — i.e., the method had nothing left to do after the call that invoked the lambda — then  $\wedge$  is effectively a tail return. The runtime detects this dynamically: after unwinding, check whether the continuation was about to return anyway.

This requires no knowledge of `ifTrue` or `ifFalse` — it is a purely structural check on the continuation. It handles any control-flow pattern, including user-defined ones.

**Covers the common VarioHyve idiom:**

```
factorial(n, acc) {
 n.eq(0).ifTrue(fun { $\wedge$ acc }).ifFalse(fun { $\wedge$ factorial(n.minus(1),
 $\hookrightarrow$ n.mul(acc)) })
}
```

Here,  $\wedge$  `factorial(...)` inside the `ifFalse` lambda unwinds to `factorial`’s `ReturnFrame`. With Layer 2, the runtime recognizes that `factorial` had nothing left to do, and reuses the frame —  $O(1)$  space for the recursion.

**Key property:** This is a *dynamic* check, not a static one. Whether  $\wedge$  lands in tail position depends on the continuation shape at runtime. But for a CESK machine, the continuation is runtime data, so the check is natural and cheap — inspect the top of  $K$  after unwinding.

**Implementation:** When  $\wedge$  fires and unwinds to a `ReturnFrame`, before delivering the return value, check if the next frame in the continuation is another `ReturnFrame` or a frame that would immediately pass the value through. If so, collapse the frames. This is the  $\wedge$ -as-tail-return optimization.

### 16.9.1.4 Layer 3 and beyond (if needed)

Further sophistication is possible but that is left for future work:

- **Continuation marks** (à la Racket, following Clinger and Flatt) — annotate tail-position boundaries on the continuation itself, enabling  $O(1)$  tail-position detection without post-hoc unwinding checks. More principled than Layer 2 but adds continuation mark infrastructure.
- **Known-method inlining** — recognize `ifTrue`, `ifFalse`, and, or as known control-flow patterns and inline their dispatch, turning the lambda bodies into direct tail positions. Philosophically compromised (“no special forms” principle) but potentially faster. Only worth considering if profiling shows Layer 2’s dynamic check is a bottleneck.
- **Static tail-position analysis across lambda boundaries** — a compile-time analysis that determines when a  $\wedge$  in a lambda is guaranteed to land in tail position. This would let the runtime skip the dynamic check of Layer 2. Requires flow analysis

of which methods always invoke their lambda arguments in tail position, which is complex to do in general, we will see if the VarioHyve support for abstract interpretation makes this easier.

### 16.9.2 Does Generalized TCO Matter?

In most languages, generalized TCO (optimizing tail calls to *any* method, not just self) is a nice-to-have — most loops are self-recursive. In VarioHyve, generalized TCO is **essential**:

- All names are late-bound. The runtime cannot determine at the call site whether `process(data)` will resolve to `self.process` (self-recursion), `OtherClass.process` (delegation), or something else entirely. Restricting TCO to self-calls would require runtime self-call detection, which is more complex than simply optimizing all tail calls.
- State machine patterns — where each state dispatches to the next — are naturally expressed as mutual tail calls between methods. Without generalized TCO, these patterns leak memory.
- The Clinger formalization is inherently generalized — it defines tail position structurally, independent of the callee’s identity.

The good news: generalized TCO is not harder to implement than self-only TCO in a CESH machine. The optimization is “pop the current frame before pushing the new one” — the callee’s identity is irrelevant. Layers 1 and 2 are already fully generalized.

### 16.9.3 Design Summary

Layer	What it optimizes	Cost	When
0	Nothing — $O(n)$ heap per recursion depth	Zero	v1 bootstrap
1	Direct tail calls (last expression in method/lambda)	~1 day	Immediately after v1 runs
2	^ in lambdas that land in tail position (dynamic check)	~2-3 days	Shortly after Layer 1
3+	Continuation marks, inlining, static analysis	Significant	Only if profiling demands

Layers 1+2 together cover the vast majority of real recursive patterns in VarioHyve, including all `ifTrue/ifFalse` control flow. Layer 3+ is unlikely to be needed unless the language acquires deeply nested control-flow combinators.

**Implementation basis:** The layered approach follows Clinger (1998), *Proper tail recursion and space efficiency*, which formalizes proper tail recursion as an asymptotic space complexity property. Clinger defines tail position inductively over expression forms and proves

that an implementation is properly tail-recursive if and only if it evaluates tail expressions in bounded space regardless of recursion depth. For the CESK machine, this translates directly: a tail call pops the current `ReturnFrame` before pushing the callee’s frame, keeping the continuation at constant depth for tail-recursive loops.

**Debugging:** TCO eliminates continuation frames, which can make traces less informative. This is a **tooling concern**, not a language concern — the debugger and profiler should reconstruct the logical call chain when needed, without compromising the runtime guarantee of bounded heap usage.

**Reference:** William D. Clinger. 1998. *Proper tail recursion and space efficiency*. In Proceedings of the ACM SIGPLAN 1998 conference on Programming language design and implementation (PLDI ’98), 174–185. <https://doi.org/10.1145/277650.277719>

## 16.10 Mirror-Based Reflection API

**Status:** Not in v1.

v1 provides no runtime reflection — objects cannot inspect their own class, list methods, or introspect slot names programmatically.

**Why not in v1:** Reflection introduces a large API surface and raises questions about encapsulation, security, and optimization. Getting the core object model right first ensures the reflection layer has a solid foundation.

**Planned approach:** VarioHyve will adopt a **mirror-based reflection API** inspired by Bracha and Ungar’s mirrors:

- **Separation of concerns** — reflective operations are not methods on `Object`. Instead, a mirror is a separate object that provides an introspective view of a target object.
- **First-class mirrors** — mirrors are regular VarioHyve objects. They can be passed around, stored in slots, and composed.
- **Stratification** — base-level code (application logic) and meta-level code (reflection) are cleanly separated. An object does not need to “know” it is being reflected upon.

Example (syntax TBD):

```
mirror = reflect(aPoint) // obtain a mirror on aPoint
mirror.slotNames // → ["x", "y"]
mirror.className // → "Point"
mirror.send("move", [1, 2]) // reflective message send
```

**Key design note:** No new runtime structures are needed. Mirrors are ordinary objects whose methods read the existing store entries — slots, class, methods, super, mixins. The additive object model (`Object` → `Class` → `Metaclass`) already exposes all the information mirrors need; the mirror API simply provides a controlled interface to it.

## 16.11 Access Control

**Status:** Not in v1.

v1 has no access control — all methods and slots are publicly visible. Any object with a reference to another object can send any message to it.

**Why not in v1:** Access control interacts deeply with inheritance, mixins, lexical scoping, and construction. Designing it correctly requires a stable object model and practical experience with real VarioHyve programs. The v1 bootstrap prioritizes getting the core semantics right.

### 16.11.1 Design Principles

VarioHyve’s access control follows an **object-capability model**: an object’s externally visible behavior is defined entirely by its **exported methods**. Everything else is internal.

**1. Private by default, explicit export.** An object does not answer to any message by default. Only methods explicitly marked as exported are visible to external callers. The export surface defines the object’s **behavioral contract** — it is the set of messages the object promises to respond to.

```
class Point(ax, ay) {
 x = ax // internal slot
 y = ay // internal slot
 recompute() { ... } // internal helper
 export x() { x } // exported getter
 export y() { y } // exported getter
 export move(dx, dy) { x!(x.plus(dx)); y!(y.plus(dy)) } // exported
 ↪ method
}
```

External code can call `aPoint.x`, `aPoint.y`, and `aPoint.move(1, 2)`. It cannot call `aPoint.recompute` or `aPoint.x!(5)` — those are internal.

**2. Exports are heritable, internals are sealed.** A subclass can only `super.method(...)` on methods that the parent exports. Internal methods are invisible to subclasses — they do not appear in the super chain. A subclass is a *client* of its parent’s behavioral contract, not a privileged insider.

```
class ColorPoint(ax, ay, ac) < Point(ax, ay) {
 color = ac
 export color() { color }
 export describe() {
 super.x // OK – x() is exported by Point
 super.recompute // ERROR – recompute is internal to Point
 }
}
```

This forces class authors to design explicit extension points as part of the export surface. Subclasses extend behavior through the same contract that external callers use.

**3. Mixins are sealed — export surface becomes private in the applier.** A mixin contributes only its exported methods to the applying class. The mixin’s internal helpers are invisible — they are implementation details of the mixin’s exported behavior.

The mixin's exported methods become **internal** (private) in the applying class by default. The applier can choose to re-export them, making them part of its own behavioral contract, or keep them as internal implementation helpers:

```
class Logging() {
 formatMsg(msg) { "LOG: ".concat(msg) } // internal to
 ↪ Logging
 export log(msg) { console.print(formatMsg(msg)) } // exported by
 ↪ Logging
}

class Server(port) <: Logging < Base {
 // Logging.log is now available internally – it was exported by Logging
 // but it is private to Server unless re-exported
 export start() { log("starting on port ".concat(port.toString)) }
 // Server does NOT have access to Logging.formatMsg – it is internal to
 ↪ Logging
}
```

Here, `aServer.log("test")` fails — `log` is not exported by `Server`. But `aServer.start` works and internally uses `log`. If the server author wants to expose logging to external callers, they explicitly re-export:

```
class Server(port) <: Logging < Base {
 export log // re-export Logging's log as part of Server's contract
 export start() { log("starting") }
}
```

This is consistent with how superclasses are handled: the export surface is the interface, and each class in the chain decides what to expose.

**Implementation note:** This works cleanly because VarioHyve mixins are **not flattened**. Each mixin is a separate object in the class's mixins: `List<Addr>` — its methods live in its own class, not merged into the applier's method dictionary. Method lookup walks `class → mixins[0] → mixins[1] → super`, checking each mixin's own methods. The runtime already tracks **dispatch origin** (`classAddr, mixinIndex?`) in each **activation**, so it knows which class or mixin provided the currently executing method. Enforcing mixin internal visibility is then a comparison: when a self-send resolves to a method in mixin `M`, the runtime checks whether the dispatch origin is also `M` (internal access — always allowed) or something else (external access — requires export). No new provenance tags or per-method closures are needed; the existing dispatch machinery provides all the information.

**4. Lexical scope gives full access.** Code inside a lexical scope sees everything — all methods and slots of enclosing objects, not just their exports. This follows the Newspeak model: lexical nesting implies a trust relationship. If you wrote the inner class, you control it.

```

class Outer() {
 secret = 42 // internal slot
 export getSecret() { secret } // exported getter

 class Inner() {
 export test() {
 secret // OK – lexical scope gives full access
 // 'secret' is resolved via the outer chain, which is privileged
 }
 }
}

```

The qualified receiver `outer` `ClassName` uses lexical declaration context. Any future access-control distinction for such sends must be defined explicitly; it cannot be inferred merely from possessing an object reference.

**Accepted outer contract:** `outer` `ClassName` selects an ordinary receiver from lexical declaration origin. There is no `object.outer` accessor to export, and activation reification is deferred. A future access-control design may distinguish lexical sends from external sends, but must not reintroduce a universal parent getter. Programs can deliberately expose enclosing objects through ordinary methods under their own capability contract.

**5. Construction follows the same rules.** `new`, `apply`, and named factory methods on the metaclass are internal by default. A class that does not export any construction path is unconstructable from outside — only code with lexical access to the class can instantiate it.

```

class PublicPoint(ax, ay) {
 x = ax
 y = ay
 export x() { x }
 export y() { y }
 export new // explicitly export the constructor
}

class InternalHelper() {
 // new is not exported – only the enclosing module can instantiate this
 export process(data) { ... }
}

```

The exact syntax for exporting `new` (which is synthesized, not user-written) is TBD — it may use a class-level descriptor (`: exportNew`) or an explicit `export new` declaration in the class body. Object construction happens through the exported behavioral interface of the class object, consistent with the principle that the export surface *is* the contract.

### 16.11.2 Roadmap

Phase	What	Runtime enforcement
<b>v1</b>	Everything public. No access control.	None
<b>v2</b>	Add export as a descriptor ( <code>: export</code> ). A static analysis tool checks that external callers only use exported methods. The interpreter ignores the annotation.	None — tool-only
<b>v3</b>	Runtime enforcement. Private-by-default. Message sends to non-exported methods from external callers are rejected with an access error.	Yes — dispatch checks export set
<b>Future</b>	Research: can abstract interpretation derive access-control properties from export sets? Can it verify mixin contracts respect the export surface? Is a heritable dimension (visible to subclasses but not external callers) needed, or does the export-only model suffice?	Analysis-driven

### 16.11.3 Connection to Existing Design

- **Capability model alignment.** VarioHyve has no globals. Top-level modules receive external capabilities through their constructor parameters; classes generally also receive capabilities through inheritance, mixins and lexical nesting. A kernel is only an optional bundle. Private-by-default would restrict exported methods without changing these sources of authority or making nested scopes isolated.
- **! convention.** With private-by-default, slots are automatically encapsulated. The `_x` backing-slot pattern becomes unnecessary — just don't export the setter. Since slot names cannot end in `!`, setter names (`x!`) are always unambiguous and easy to control independently from getters.
- **Descriptor layer.** Export annotations fit naturally as descriptors in v2. The v1 interpreter ignores them; a separate tool checks them.
- **Non-flattened mixins.** The [runtime representation](#) of mixins (`mixins: List<Addr>`) and the [dispatch origin](#) tracking in activations provide the infrastructure needed for mixin access control. No new runtime structures are required — enforcing export visibility on mixin members is a check in the existing dispatch loop.
- **Abstract interpretation.** Export sets provide free precision for the abstract interpreter — message sends to non-exported methods are statically dead, pruning the

abstract state space.

#### 16.11.4 Prior Art

- **Newspeak** (Bracha) — private/public with lexical module boundaries; lexical scope gives full access
- **E language** (Miller) — pure capability model; encapsulation via lexical scope only, no access modifiers
- **Wyvern** (Aldrich et al.) — capability-safe language with effect systems
- **Joe-E** (Mettler et al.) — capability discipline as a subset of Java

The combination of private-by-default capability model, sealed inheritance, mixin export propagation, Newspeak-style lexical trust, and AAM-based abstract interpretation is — to our knowledge — unexplored in the literature. This intersection represents a potential research contribution.

### 16.12 Tear-Off: First-Class Method References

**Status:** Not in v1.

v1 provides no syntax for obtaining a method as a first-class callable without invoking it. A bare name *x* always invokes (runs the accessor or method); there is no way to “tear off” a method and pass it around as a value.

**Why not in v1:** [Unified invocation](#) lets methods and eligible generated callable bindings supply the same invocation selector. Bare sends return accessor or method results, while parentheses invoke selected behavior once. Neither form reifies a method. Tear-off would add an explicit operation to obtain the method itself as a callable, requiring dedicated syntax and rules for bound versus unbound methods.

**Planned approach:** A **name suffix operator** (tentatively *\$*) that produces a bound method reference:

```
class Sorter() {
 compare(a, b) { a.lessThan(b) }
 process(items) {
 items.sortBy(compare$) // tear-off – passes bound method as
 ↪ callable
 items.forEach(describe$) // equivalent to fun(x) { describe(x) }
 }
}
```

The *\$* suffix on a name means: “don’t invoke the method — return a callable that, when applied, dispatches to this method on the current receiver.”

#### 16.12.1 Name Suffix Summary

VarioHyve uses name suffixes to distinguish four operations on a name:

Suffix	Syntax	Meaning
. (dot)	<code>a.name</code>	ordinary zero-argument send; parentheses request invocation at their arity
! (bang)	<code>a.name! (v)</code>	setter — synthesized for writable slots, convention for mutating/effectful methods; <b>only valid on selectors, not on slot/parameter/local names</b>
() (parens)	<code>a.name()</code>	invoke zero-argument behavior once: exact method or eligible callable binding
\$ (dollar)	<code>a.name\$</code>	tear-off — return bound method as callable, do not invoke

### 16.12.2 Semantics

A tear-off `receiver.method$` produces a **bound method object** — a callable that captures both the method body and the receiver:

```
p = Point(3, 4)
mover = p.move$ // bound method – captures p and move
mover(1, 2) // equivalent to p.move(1, 2)
```

The bound method object implements `(apply, N)` which dispatches the original method on the captured receiver with the given arguments.

### 16.12.3 Design Considerations

- **Bound vs unbound:** `obj.method$` produces a *bound* method (receiver captured). *Unbound* method references (without a receiver) may be supported via the mirror API or a separate syntax (TBD).
- **Arity:** The bound method inherits the original method's arity. `compare$` from a `compare(a, b)` method produces a callable with `(apply, 2)`.
- **Identity:** Whether two tear-offs of the same method on the same receiver are `==` is TBD. Structural equality (same method, same receiver) is likely but has allocation implications.
- **Language extension vs reflection:** Tear-off is planned as a grammar extension (adding `$` or another symbol to the IDENT rule or as a postfix operator) rather than a reflection-only capability. This keeps it lightweight and syntactically natural. The mirror API may also expose tear-off functionality for reflective use cases, but the primary mechanism should be a language-level construct.

### 16.13 Concurrency and Parallelism

**Status:** Explicitly out of scope for the base VM. Concurrency is a composition-layer concern.

The VarioHyve base VM is **purely deterministic and sequential**. There is no notion of threads, coroutines, channels, atomic operations, or shared mutable state in the base language. A single VarioHyve program executes as a single-threaded CESK reduction sequence — every step is deterministic and locally analyzable.

This is a deliberate design choice, not a limitation.

#### 16.13.1 The Semantic Language Interface (SLI)

VarioHyve addresses concurrency through a **layered architecture** inspired by the Gamine Semantic Language Interface:

Semantic Composition VM (concurrency model, scheduling, sync)	
Composes multiple Base VM instances via the SLI according to a concurrency model	
Semantic Language Interface (SLI) (deterministic API exposed by each Base VM to the composition layer)	
VarioHyve Base VM (sequential, deterministic CESK machine) (one per "process" / "actor" / "agent")	

**Layer 0 — Base VM.** A single CESK machine. Purely sequential. Exposes a Semantic Language Interface: a deterministic API that the composition layer uses to drive execution. The SLI provides operations like:

- `step()` — advance the machine by one reduction step
- `inject(message)` — enqueue an external message into the machine's input
- `extract()` — retrieve output messages or results
- `snapshot()` / `restore(state)` — capture and restore machine state (for replay, migration, or speculative execution)
- `inspect()` — query machine state (for monitoring, debugging, abstract interpretation)

The exact SLI API is TBD and will be refined through implementation. The key property is that the SLI is **deterministic and side-effect-free from the base VM's perspective** — all external interaction is mediated through explicit inject/extract operations.

**Layer 1 — Semantic Composition VM.** A meta-level runtime that manages multiple Base VM instances and composes them according to a chosen concurrency model. The Composition VM is not VarioHyve — it is external infrastructure (potentially written in Python,

Java, Rust, or VarioHyve itself running in its own Base VM). Different Composition VM implementations can provide different concurrency semantics:

Concurrency Model	Composition Strategy
<b>Actor model</b>	One Base VM per actor. Messages between actors go through <code>inject/extract</code> . The Composition VM is the mailbox + scheduler.
<b>CSP (channels)</b>	One Base VM per process. Channel operations are <code>inject/extract</code> calls mediated by the Composition VM.
<b>Dataflow</b>	Each Base VM computes a pure function. The Composition VM wires outputs to inputs.
<b>Parallel simulation</b>	Multiple Base VMs run model fragments. The Composition VM manages time synchronization (e.g., DEVS, HLA).
<b>Speculative execution</b>	<code>snapshot()</code> a Base VM, run two branches, <code>restore()</code> the loser.

The Composition VM decides scheduling, synchronization, message routing, and failure handling. The Base VM is unaware of concurrency — it just steps, injects, and extracts.

### 16.13.2 Why This Layered Approach

- 1. The base stays simple.** No locks, no atomics, no memory model complexity, no race conditions, no need for concurrent GC. The CESK machine is a pure function from state to state. This is essential for the abstract interpretation story — AAM-style analysis requires deterministic semantics.
- 2. Concurrency models are swappable.** The same VarioHyve program can run under an actor model, a CSP model, or a dataflow model — the difference is in the Composition VM, not the language. This is critical for a system-of-systems lingua franca: different subsystems may require different concurrency semantics, and the federation layer must accommodate all of them.
- 3. Replay and debugging are free.** A deterministic base VM with `snapshot/restore` enables time-travel debugging, deterministic replay, and speculative execution without any language-level support for these features.
- 4. Cross-boundary composition is natural.** The [Alien/Expat model](#) sits at the SLI boundary. When a VarioHyve Base VM sends a message to an Alien, the SLI extracts the message; the Composition VM routes it to the foreign runtime (or another Base VM); the response comes back via `inject`. The Alien/Expat protocol is the message-level API; the SLI is the machine-level API.

### 16.13.3 Connection to the Alien/Expat Model

The Alien/Expat boundary and the SLI are complementary:

- **Alien/Expat** defines how individual foreign operation requests cross the boundary — argument marshalling, identity, adapter dispatch and failures. Message forwarding is one adapter policy.
- **SLI** defines how entire *machines* interact — stepping, message injection, state capture.

In this future architecture, an Alien operation request goes through the SLI's extract mechanism. It may request foreign message forwarding, an explicit capability operation on an opaque handle, or another adapter-defined interaction; no shared object ontology is implied. The Composition VM receives the extracted message, routes it to the appropriate foreign runtime (or another Base VM), gets the result, and injects it back. From the Base VM's perspective, the Alien just sent a message and got a reply — it has no awareness of the concurrency or routing that happened in between.

#### 16.13.4 Prior Art

- **Gamine Semantic Language Interface** — the direct inspiration for this layered approach; separation of base execution from semantic composition
- **Erlang/OTP** — actor model with isolated processes and message passing; each process is effectively a sequential VM
- **E language** — communicating event loops (vats); each vat is single-threaded, concurrency is inter-vat
- **DEVS** (Discrete Event System Specification) — formal framework for modeling and simulation with well-defined composition semantics
- **Orleans / Virtual Actors** — actor model where the runtime manages actor lifecycle and placement transparently

#### 16.13.5 Roadmap

Phase	What
<b>v1</b>	Sequential Base VM only. No SLI, no composition.
<b>v2</b>	Define and implement the SLI API. Single Base VM exposed through the SLI to a host (Python/Java).
<b>v3</b>	Composition VM prototype: actor-model composition of multiple Base VMs.
<b>Future</b>	Additional concurrency models; formal semantics of composition; abstract interpretation across composed VMs.